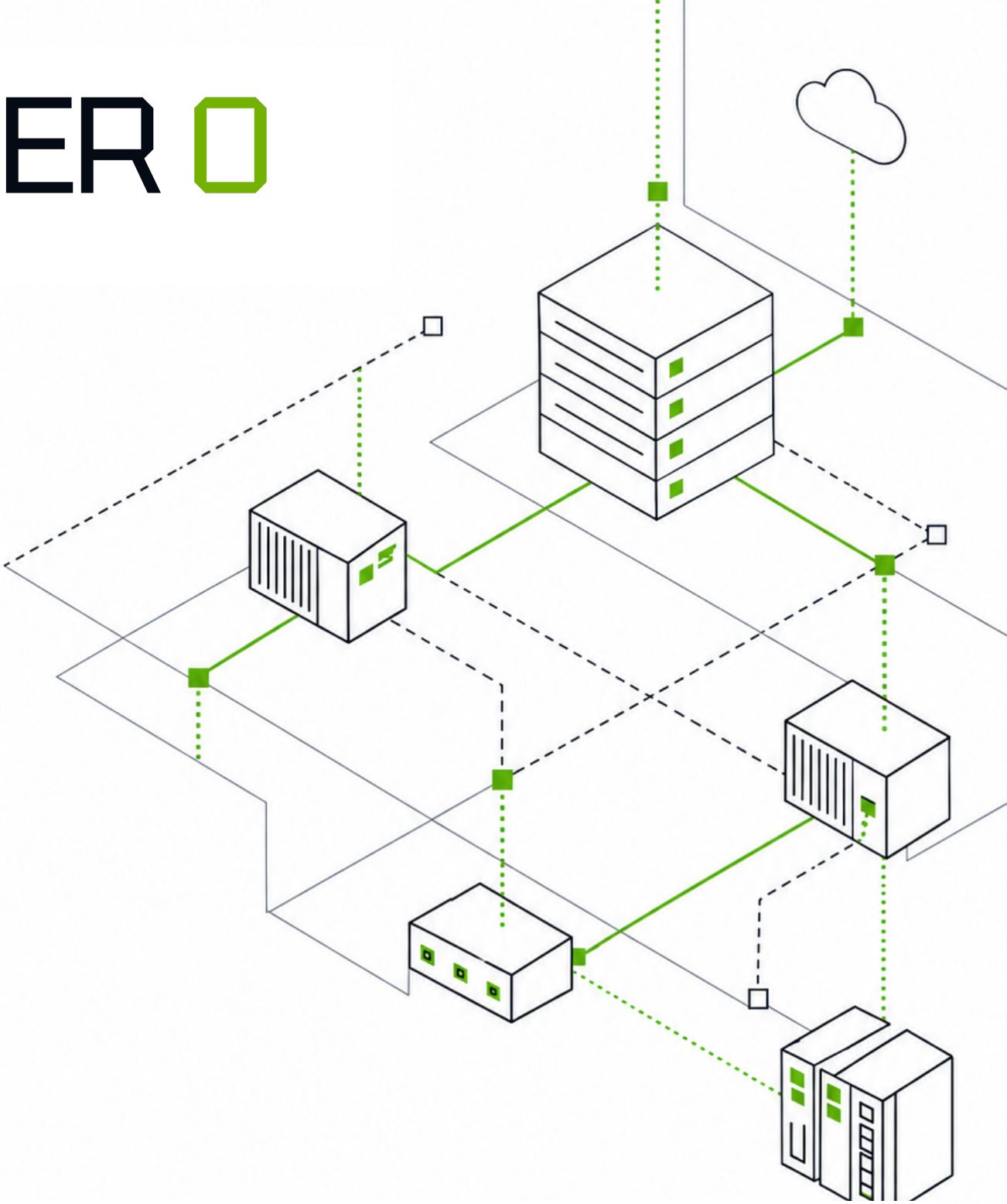
TIER 0



Guía técnica de Tier0

Índice

PRIMEROS PASOS

1. Elegir la mejor versión
2. Instalación
3. Probar Demo Factory en Tier0

USO DE TIER0

1. Base de datos de fábrica
2. Preparar la base de datos
3. Gestionar datos del centro al edge
4. Trabajar con datos de fábrica
5. Crear apps
6. Construir Digital Twins
7. Analizar datos
8. Operar Tier0 con Agents

BUENAS PRÁCTICAS

1. Modelar datos de fábrica
2. Conectar protocolos industriales
3. Seguridad y resiliencia
4. Crear apps de analítica
5. Crear flujos de datos con Agent
6. Mostrar datos con Digital Twin

REFERENCIA

1. Referencia de API
2. Referencia de comandos CLI
3. Skills integradas de Builder
4. Gestión de usuarios
5. Seguridad de red y datos de Tier0 Cloud
6. Jerarquía de equipos ISA-95
7. Standard Deployment Plan
8. Plan de despliegue de alta disponibilidad
9. SLA y límites de alta disponibilidad
10. Lista estándar de puertos
11. Runbook de operaciones

APÉNDICE

1. Referencia completa de OpenAPI

Primeros pasos

Elegir la mejor versión

Edge, Cloud o Enterprise: posicionamiento, planes y matriz de capacidades para elegir tu edición de Tier0.

Tier0 está disponible en tres ediciones. Cada edición tiene sus propias funciones, planes y capacidades.

■ OPEN SOURCE

Edge

La base UNS en una sola máquina. Completamente tuya.

> Para evaluación técnica, PoC y equipos cómodos operando software open source por su cuenta.

Apache-2.0 GRATIS

Integración de datos UNS, almacenamiento histórico y despliegue Docker en una sola máquina.

Clonar en GitHub ->

■ SAAS GESTIONADO

Cloud La mayoría de los equipos empiezan aquí

La plataforma completa, operada por nosotros.

> Apps, notebooks y launchpad desde el primer día, sin infraestructura que operar.

Builder $199/SEAT/MO

Solo generación de apps.

Growth ★ $20,000/YR

Hasta 5 edges. Para una sola fábrica con pocas apps y necesidad de empezar rápido.

Scale $38,000/YR

Hasta 10 edges. Para usuarios con varias fábricas y muchas apps.

Iniciar la prueba de 7 días ->

■ DESPLIEGUE PRIVADO

Enterprise

La plataforma completa, bajo tus condiciones.

> Para soberanía de datos, escala, gobernanza y supervisión empresarial.

Base $10,000/YR

Base de datos unificada. Un pequeño número de apps de propósito único.

Plus $20,000/YR

Instancia única. Integración de datos para una sola fábrica y apps para varios casos de uso.

Cluster ★ $39,900+/YR

Multiinstancia. Varias fábricas, muchas apps y gestión centralizada en nube privada.

Hablar con el equipo ->

Add-ons: Se pueden agregar edge nodes y applications a los planes existentes según tus requisitos. Para más detalles, consulta tier0.app/pricing.

> ⓘ **Explicación de términos especiales**
> En los planes Cloud, un edge es un nodo de conexión que se comunica con tu Tier0 en la nube. Puede ser un Edge Tier0, un gateway o una PC industrial.

Matriz de capacidades

Capacidad	Edge	Cloud	Enterprise
UNS / Modelado de datos	✓ UNS en una sola máquina	✓ Growth / Scale	✓ Base / Plus / Cluster
Protocolos industriales	✓ MQTT	✓ Growth / Scale: MQTT, REST, i3X, OPC UA	✓ Base: MQTT; Plus / Cluster: MQTT, REST, i3X, OPC UA
UNS Agent	×	✓ Growth / Scale	×
Notebook (análisis avanzado)	×	✓ Growth / Scale	✓ Plus / Cluster
Vision	×	✓ Scale	✓ Plus / Cluster
Anchor	×	✓ Scale	✓ Cluster
App Builder + Template Library	×	✓ Builder / Growth / Scale	×
LaunchPad / My Apps	×	✓ Builder / Growth / Scale	✓ Base / Plus / Cluster
Auditoría / logs de apps y sistema	×	✓ Growth / Scale	✓ Plus / Cluster; SIEM en Cluster
HA / multiinstancia / gobernanza	×	×	✓ Cluster
Operaciones	Tú	FREEZONEX	Tú, con soporte

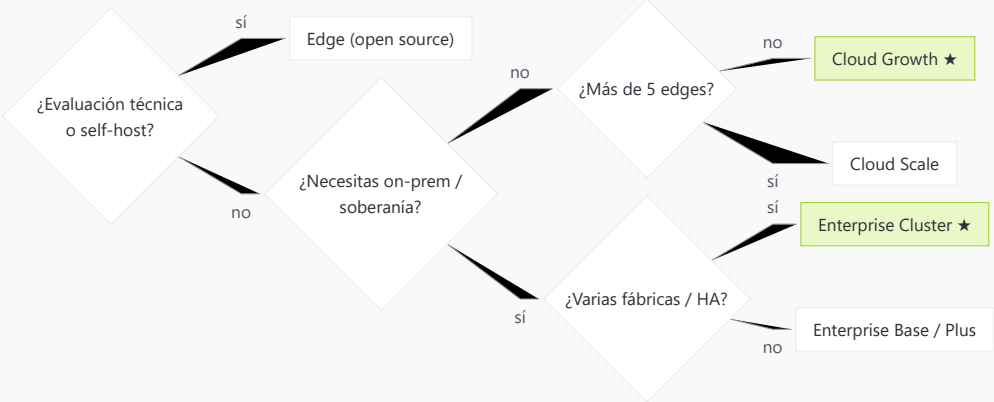
Requisitos de hardware de Edge

> **Si quieres usar Edge**
> Edge está pensado para evaluación técnica y requiere experiencia operativa. Asegúrate de que tu entorno cumpla los siguientes requisitos de hardware antes de usarlo.

	Mínimo	Recomendado
CPU	4 cores	8 cores
Memory	8 GB	16 GB
Disk	100 GB (1000 IOPS)	1 TB
OS	Ubuntu 24.04, Windows 10/11 (Docker)	-

Árbol de decisión

¿Todavía no sabes qué opción elegir? Sigue la ruta para decidir.



Siguiente

- Crear apps sobre UNS - Crea aplicaciones industriales con datos UNS.

- **Instalación** - La prueba Cloud de 7 días incluye la plataforma completa.

PRIMEROS PASOS

Instalación

Instala Tier0 según la edición.

Tier0 se ofrece en tres ediciones:

- **Edge** - Despliegue open source, ejecutado localmente con Docker.
- **Cloud** - SaaS gestionado, sin infraestructura que preparar.
- **Enterprise** - Despliegue privado para entornos empresariales.

Consulta Elegir la mejor versión para encontrar la edición que mejor se ajusta a tus requisitos.

Instalar Tier0

PESTAÑA: CLOUD

Solicita una cuenta de prueba antes de explorar Tier0 Cloud.

1. Registra tu cuenta en Register Tier0.
2. Ingresa en la ventana emergente el código de verificación enviado a tu correo.
3. Serás redirigido a la página **Login**. Inicia sesión con la cuenta registrada.
4. Ingresa la información correspondiente y haz clic en **Continue**.
5. Selecciona **Create a New Workspace**.
6. Crea un workspace y comienza la prueba.

PESTAÑA: ENTERPRISE

Requisitos

- Despliegue de un solo nodo
  - Mínimo: 4C / 8G / 100G
  - Recomendado: 8C / 16G / 500G
- Despliegue estándar
  - Mínimo: 4 VM

    - Línea base requerida por tipo de VM:
        - App: 8C / 32G / 100G
        - Database: 4C / 8G / 500G
        - Witness: 2C / 4G / 40G
- Despliegue de alta disponibilidad
    - Mínimo: 6 VM
    - Línea base requerida por tipo de VM:
        - App (Swarm): 8C / 32G / 100G x 3
        - Database: 4C / 8G / 500G x 2
        - Witness: 2C / 4G / 40G

> **Los requisitos de hardware pueden variar**
> Los requisitos de hardware de Enterprise varían según el escenario de despliegue.

> **Nota**
> La información detallada del despliegue está disponible en el **Plan de despliegue estándar** y el **Plan de despliegue de alta disponibilidad**.

Operaciones

1. Obtén el paquete de tu proveedor y verifícalo.

```
sha256sum -c tier0-enterprise-offline-standard-
YYYYmmddTHHMMSS.tar.gz.sha256
```

2. Descomprime el paquete.

```
tar -xzf tier0-enterprise-offline-standard-YYYYmmddTHHMMSS.tar.gz
cd tier0-enterprise-offline-standard-YYYYmmddTHHMMSS
```

3. Edita `install.env` según los detalles del despliegue y luego inicia el despliegue.

```
vi install.env
```

```
bash install.sh
```

Operaciones comunes de mantenimiento

- Revisa el entorno de despliegue sin instalar Enterprise.

```
./install.sh --check-only
```

- Limpia todos los nodos después de un despliegue fallido.

```
./full-cleanup.sh --env install.env --yes
```

> Este comando borra todos los datos de runtime de Tier0 y las copias de seguridad de la base de datos, pero conserva las VM, el sistema operativo y el software Docker instalado.

- Después de la instalación se genera un reporte de despliegue.

```
runtime/install-reports/<timestamp>/deployment-report.md
```

> No entregues el proyecto si el reporte contiene algún elemento **FAIL**. Contacta primero con soporte.

Referencias de mantenimiento

- Lista estándar de puertos
- Guía de mantenimiento y resolución de problemas
- Instrucciones sobre límites de HA

PESTAÑA: EDGE

Requisitos

- CPU mínimo de 4 núcleos, 8 GB de RAM, 100 GB de disco

- Ubuntu 24.04 o Windows 10/11 con Docker y Git

Operaciones

1. Clona el repositorio Tier0-Edge desde GitHub en tu servidor y entra al proyecto.

```
git clone https://github.com/FREEZONEX/Tier0-Edge
cd Tier0-Edge/deploy
```

2. Configura en `.env.default` los parámetros de la tabla según tu entorno.

```
cp .env.default .env
vi .env
:wq!
```

Parámetros que deben configurarse

Campo	Tipo	Descripción	Ejemplo
VOLUMES_PATH	string	Ruta de almacenamiento para los datos del proyecto	`/srv/tier0/data` en Linux; `D:/tier0/data` o `/d/tier0/data` en Windows Bash
ENTRANCE_DOMAIN/ENTRANCE_PORT	string	Endpoint de acceso a Tier0 (IP/dominio + puerto)	`192.168.1.10` / `8088`
LANGUAGE	enum	zh-CN / en-US	`en-US`
ADMIN_INIT_PASSWORD	string	Contraseña inicial del usuario administrador	Una contraseña privada y segura

Campo	Tipo	Descripción	Ejemplo
COMPOSE_PROJECT_NAME/ PORT_OFFSET	string	Aísla varias instancias en el mismo host	`edge_lab` / `100`

3 Ejecuta el script de instalación y revisa el estado de la instalación.

```
bash bin/install.sh
bash bin/compose.sh ps # Revisa el estado de los contenedores
bash bin/compose.sh logs -f backend # Revisa los logs del contenedor backend
curl -fsS http://127.0.0.1:8088/readyz # Revisa la readiness del servicio backend
```

> ⚠ **Precaución**
> No edites `deploy/.env.runtime`; contiene secretos generados y valores de runtime resueltos.

4 Accede a Tier0 con el endpoint configurado y la cuenta de administrador predeterminada:

- Usuario: `tier0`
- Contraseña: el valor de `ADMIN_INIT_PASSWORD` en `.env` o `tier0` si no se configuró.

Siguiente

- Probar Demo Factory - Una planta precargada para explorar.
- Elegir la mejor versión

PRIMEROS PASOS

Probar Demo Factory en Tier0

Explora una fábrica precargada y funcional en tu workspace de prueba Cloud: namespace, flows, notebooks y apps.

La forma más rápida de entender Tier0 es explorar una fábrica que ya funciona.

Contexto de negocio

La fábrica en Tier0 recibe órdenes desde ERP, divide las órdenes, programa la producción, envía el programa al departamento de producción y recibe datos de avance de producción mientras avanza el trabajo.



Conocer el proceso

1 En Tier0, ve a **UNS** y revisa los detalles de las órdenes que vienen de ERP.

- `DemoFactory/ERP/ProductionOrders/State/UpsertProductionOrder`: Órdenes.
- `DemoFactory/ERP/ProductionOrders/State/OrderList`: Snapshot de la lista actual de órdenes.

> 🚀 **¿Cómo se recopilan estos datos en UNS?**
> Ve a **Flows** > **Source Flow** > **DemoFactory-Flow** para revisar los detalles de recolección de datos.

2. (Opcional) Ve a **Launchpad**, abre la aplicación **Work Order Management** y crea órdenes en la página **Production Orders**.
3. En **Work Order Management**, divide órdenes y programa planes de producción para las órdenes de trabajo divididas en la página **Plans**.
4. Envía el plan al departamento de producción y revisa los detalles del plan en los siguientes topics de **UNS**.

   - `DemoFactory/ERP/WorkOrderPlan/Metric/SplitCount`: Número de workorders después de la división.
   - `DemoFactory/ERP/WorkOrderPlan/State/PlanStatus`: Estado actual del plan.
   - `DemoFactory/ERP/WorkOrderPlan/State/WorkOrderList`: Lista de workorders después de programar el plan de producción.

> ⓘ **Nota**
> La aplicación escribe directamente los plans y workorders de vuelta en **UNS**.

5. Revisa los datos de avance de producción en la página **Plan Details** de la aplicación.

> 🚀 **¿De dónde vienen los detalles?**
> Los datos de avance de producción vienen del departamento de producción, se recopilan mediante **DemoFactory-Flow** en **Source Flow**, se publican en **UNS** y se muestran en la página **Plan Details**.
>
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/CurrentWorkOrder`: La orden en proceso.
> - `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/State/WorkOrderStatus`: El estado de ejecución de la orden actual.

- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Target_Qty`: La cantidad objetivo de producción de la orden actual.
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Produced_Qty`: La cantidad de productos ya completados de la orden actual.
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Defect_Qty`: La cantidad de defectos de la orden actual.
- `DemoFactory/Site_01/Production/Line_01/WorkOrderExecution/Metric/Completion_Rate`: La tasa de finalización de la orden actual.

Siguiente

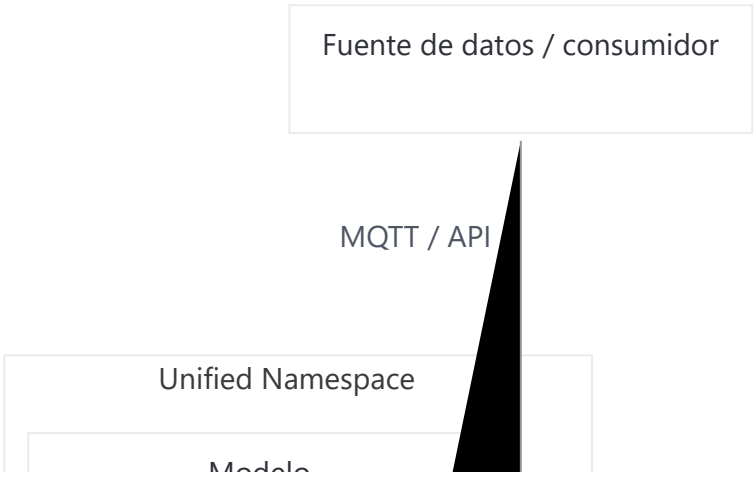
- **Elegir la mejor versión** - Compara cada versión y elige la que mejor se adapta a ti
- **Conceptos de UNS** - Comprende el modelado de datos en Unified Namespace

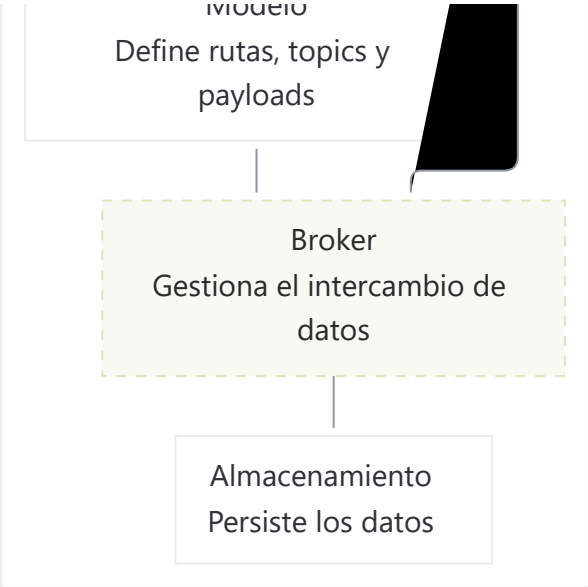
Base de datos de fábrica

El modelo Unified Namespace: Metric, State, Action y cómo UNS almacena datos de fábrica.

En Tier0, los datos de fábrica se organizan en modelos claros con estructura de árbol basados en la metodología UNS (Unified Namespace), formando una base de datos unificada.

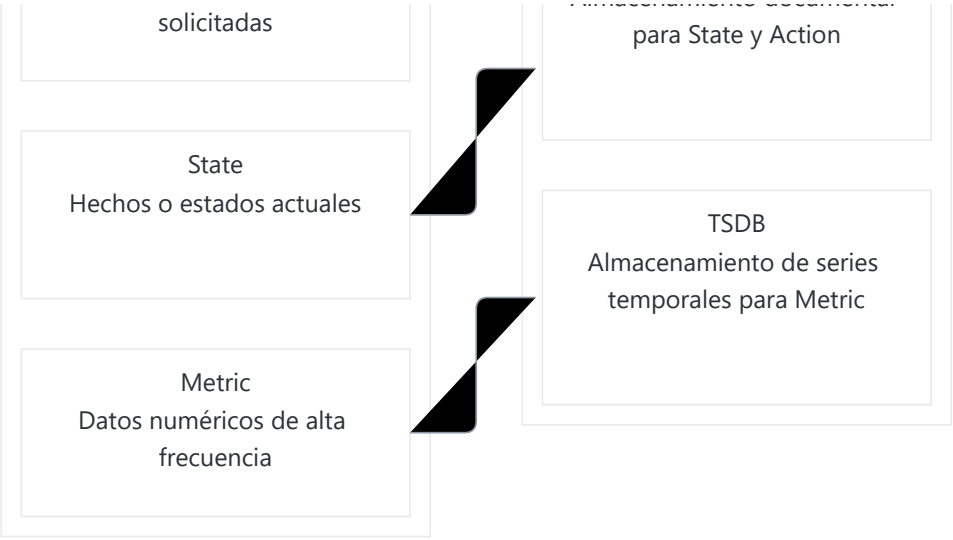
- El broker recibe y envía datos a través de topics del modelo.





- Los modelos se clasifican en tres tipos: **Metric**, **State** y **Action**.
- Los datos se almacenan en diferentes bases de datos según el tipo de modelo.
    - **Action/State**: JSONB (PostgreSQL)
    - **Metric**: datos en tiempo real (TSDB)





Model

> **¿Qué guía debo seguir al crear un modelo?**
> Elige el enfoque de modelado según lo que representan los datos.
>
> - **Physical Data**: Datos en tiempo real de máquinas físicas. Se organizan normalmente según el estándar **ISA-95** y dependen mucho de la jerarquía de ubicación.
> - **Business Data**: Datos relacionados con requisitos de producción, programas, tareas y despacho. Normalmente se organizan por **dominio de negocio y objeto**.

Metric, State, Action

Los datos industriales cambian con distintas frecuencias y sirven para distintos propósitos, por eso Tier0 separa los modelos UNS en tres tipos fijos.

Tipo	Por qué	Ejemplo
`METRIC`	Diseñado para datos de alta frecuencia que se recopilan continuamente y	Temperatura/presión/voltaje de máquina

Tipo	Por qué	Ejemplo
	necesitan almacenamiento de series temporales.	
`STATE`	Diseñado para datos que cambian con menor frecuencia y describen la condición o hecho actual de un objeto.	Estado operativo de una máquina
`ACTION`	Diseñado para datos de comando de baja frecuencia que registran lo que debe ejecutar un sistema externo u operador.	Iniciar lote, detener máquina

Path/Topic, Payload

Cada modelo UNS contiene un path, un topic y un payload. Juntos forman una estructura de **árbol** que transporta contexto de negocio y datos industriales.

- **Path/Topic**: El path representa la propiedad de los datos, y el topic transporta el payload del mensaje. Juntos definen la ***model address***.
- **Payload**: El payload del mensaje que contiene el topic. Es JSON en formato **key-value** y define los ***model data***.

Por ejemplo, un modelo en UNS se ve como el árbol siguiente:

PESTAÑA: PHYSICAL DATA

```
Factory_A
└── Site_01
    └── SMT_Line_1
        └── Metric
            └── Machine_001
```

- **Path**: `Factory_A/Site_01/SMT_Line_1/Metric`
- **Topic**: `/Machine_001` .
- **Payload**

```
"temperature": 85,
"vibration": 2.8,
```

PESTAÑA: BUSINESS DATA

```
Factory_A
└── ERP
    └── ProductionOrders
        └── State
            └── OrderList
```

- **Path**: `Factory_A/ERP/ProductionOrders/State`
- **Topic**: `/OrderList` .
- **Payload**

```
"orders": "0123569876",
"updated_at": 2026-7-13,
```

> **Para crear un modelo**
> Consulta Cómo crear un modelo de datos.

Broker

En Tier0, el broker es el componente de la plataforma que recibe y envía datos por MQTT u OpenAPI según los modelos de datos.

- **Por MQTT**: El path completo de cada nodo en el árbol del modelo funciona como dirección del topic para pasar el payload del mensaje.
- **Por Open API**: Ejecuta comandos CRUD sobre modelos de datos mediante solicitudes HTTP.

> **Referencia de API**
> Para detalles de API, consulta **Referencia de API**.

Storage

Tier0 usa PostgreSQL para almacenar datos. Los distintos tipos de datos cambian a distintas velocidades y tienen distintos requisitos de almacenamiento.

Tipo de dato	Almacenamiento	Descripción
Metric	Time-series database	Optimizado para mediciones de alta frecuencia como temperatura y presión. Cada topic incluye campos predeterminados: `timestamp` y `quality`.
State	PostgreSQL	Almacena estados de máquinas o sistemas en un formato flexible `JSONB`.
Action	PostgreSQL	Almacena comandos o acciones en un formato flexible `JSONB`.

Siguiente

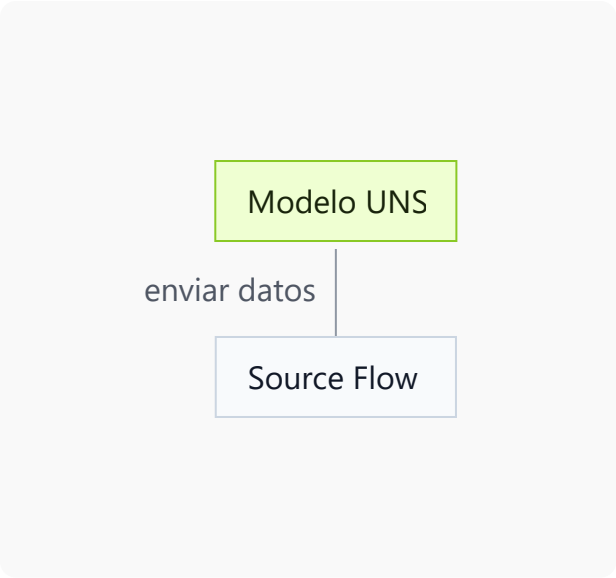
- Conectar datos a UNS - Cómo modelar tu planta en UNS y conectar datos al modelo.
- Trabajar con datos UNS - Cómo operar sobre tu modelo UNS mediante MQTT y APIs.

USO DE TIER0

Preparar la base de datos

Modela tu namespace e incorpora datos con Source Flows. Disponible en todas las ediciones.

Primero construye un modelo en **UNS** y luego conecta los datos mediante **Source Flow** al modelo, formando una base de datos para su uso posterior.



Construir un modelo de datos

Define la jerarquía de datos como un mapa de árbol según:

- La ubicación física de la que provienen los datos (ISA-95);
- La lógica de negocio que siguen los datos.

Construir modelos manualmente

1. Diseña la estructura del modelo y añade las rutas en orden antes de añadir el topic.
2. Configura **Topic Type** entre **Metric**, **State** y **Action** según las características de los datos.
3. Selecciona **Mock Data** para enviar datos simulados al modelo y **Enable History** para guardar los datos en la base de datos.

> **Nota**
> Se creará un flow en **Flows** para enviar datos simulados al modelo.

4. Añade campos de datos al topic con los tipos correctos mediante **Auto Parsing** o **Pre-defined**.
    - **Auto Parsing**: Convierte automáticamente campos desde texto JSON por lotes.
    - **Pre-defined**: Configura manualmente los campos del topic uno por uno.

Importar modelos

> **Consejo**
> Usa LLMs como ChatGPT para ayudar a importar modelos.

1. Envía el JSON de plantilla de **Import** a la IA y usa un prompt similar.

```
Genera un modelo UNS usado para xx en la planta xx, incluyendo xx
equipos y fuentes de datos, según la plantilla.
```

2. Importa el resultado generado en UNS.

Conectar datos a UNS

Características del flujo de datos

- Basado en Node-RED.
- Cada flow termina con un nodo `mqtt out`. Funciona como un MQTT client para publicar datos en el broker UNS.
- El broker UNS está integrado en los nodos `mqtt out` y `mqtt in` con el mismo nombre que el flow.
- Cuando se usa un modelo UNS como topic, los datos van directamente al modelo correspondiente en **UNS**.

Pasos

1. Usa nodos según el tipo de fuente de datos para crear un flow y termínalo con un nodo `mqtt out`.

> **Si necesitas nodos que no están disponibles**
> Instala nodos de la comunidad de Node-RED según tus requisitos. Los nodos de la comunidad suelen comenzar con `node-red-contrib`.

2. Asegúrate de que el **Server** del nodo `mqtt out` esté configurado como el broker UNS y que el topic sea un modelo de **UNS**.

Gestión de versiones de flow

Guarda versiones del flujo de datos para poder volver a una versión anterior o iniciar un nuevo flow desde una versión histórica.

> **Precaución**
> La reversión de versiones no está disponible en la edición Edge.

Siguiente

- Crear apps con datos de fábrica - Crea aplicaciones industriales con datos UNS.
- Analizar datos - Analiza datos UNS con Marimo Notebook y Python.

USO DE TIER0

Gestionar datos del centro al edge

Gestiona sin interrupciones los datos de los nodos edge conectados desde un nodo central.

En una fábrica real, los datos suelen recopilarse mediante dispositivos edge y gestionarse desde un nodo central para reducir el uso del ancho de banda de red y mejorar la seguridad de los datos. Tier0 ofrece una solución de Fleet Management para gestionar desde un nodo central los datos que se encuentran en los nodos edge.

Conectar nodos edge al nodo central

1. Primero, crea nodos Fleet en el nodo central.

   > **Nota**
   > El número de nodos edge que se pueden conectar depende de la licencia.

2. Utiliza el token generado por el nodo central para conectar cada nodo edge.

Sincronizar datos entre el edge y el centro

- Los datos UNS solo se pueden seleccionar y sincronizar desde los nodos edge al nodo central.
- Desde el nodo central, puedes asignar usuarios a los nodos edge para gestionarlos.

Siguiente

- Modelar datos de fábrica — Aprende a modelar datos de fábrica en UNS.
- Preparar la base de datos — Modela tu namespace y conecta los datos de origen.

USO DE TIER0

Trabajar con datos de fábrica

Opera datos UNS con MQTT y la API, con la referencia de API.

Puedes operar datos de UNS mediante MQTT y APIs.

MQTT

Los MQTT clients pueden conectarse al broker UNS con credentials generadas en Tier0.

> **Nota**
> Puedes usar MQTT clients como **MQTTX**, **MQTT Explorer** y **Node-RED** para conectarte al broker y publicar/suscribirte a messages.

Generar MQTT Credentials

En la pestaña **Edge** de Tier0, crea una nueva credential y guárdala.

Conectar MQTT Clients

> **Consejo**
> Los siguientes clients se usan como ejemplos para mostrar cómo usar las credentials para conectarse al broker UNS.

PESTAÑA: MQTTX

1. Install and open MQTTX.
2. Agrega una conexión con la información del broker UNS y las credentials generadas.

```
{
  "Host": "mqtt.tier0.dev",
  "Port": 8883,
  "Client ID": "<generated client ID>",
  "Username": "<generated username>",
  "Password": "<generated password>"
```

```
}
```

PESTAÑA: MQTT EXPLORER

1. Install and open MQTT Explorer.
2. Agrega una nueva conexión con la información del broker UNS y las credentials generadas.

```
{
  "Protocol": "mqtt://",
  "Host": "mqtt.tier0.dev",
  "Port": 8883,
  "Username": "<generated username>",
  "Password": "<generated password>"
}
```

3. Change the client ID to the credential ID in **ADVANCED** before connecting.

PESTAÑA: NODE-RED

- Usa **Event Flow** - módulo de procesamiento de datos en Tier0 basado en Node-RED.
  1. Crea un **Event Flow** en **Flows** y empieza con un node `mqtt in`.
  2. Select the UNS broker as **Server** (same name as the flow) and set **Topic** to a UNS model in the node.
- Usa una **instancia independiente de Node-RED**
  1. Instala y abre la instancia de Node-RED.
  2. Agrega un server en el node `mqtt in` con la siguiente información.

```
{
  {/* under Connection */}
  "Server": "mqtt://mqtt.tier0.dev",
  "Port": 8883,
  "Client ID": "<generated clientId>",
  {/* under Security */}
  "Username": "<generated username>",
  "Password": "<generated password>"
```

```
}
```

Probar conexiones

1. Asegúrate de que el client status sea normal.
2. Copia un topic de modelo UNS y publica un message en él mediante el client.

API

Usa la **REST API** para operar datos de UNS.

> 🚀 **Cómo usar la API**
> El base path de OpenAPI es `/openapi/v1`, por ejemplo `https://your-workspace.tier0.app/openapi/v1`, y la API key se pasa en el header `x-api-key`.

Obtener API Key

Las API keys están en **Settings** > **API Keys**. Según el account type, los tipos de key permitidos son distintos.

- **Service Key**: Solo la pueden crear **Admin** y **Owner**.
- **Personal Key**: La puede crear cualquier persona, y su permission scope es el mismo que el de la cuenta.

> ⚠ **Permiso de key vs permiso de cuenta**
> El permiso de cuenta prevalece sobre el permiso de key. Cuando la key está configurada como **Full Access**, los datos no disponibles para la cuenta siguen sin poder accederse con esa key.

Ejemplos

- `/uns/create`

  Crea un path o topic de UNS. Usa `namespace` para definir el node tree.

PESTAÑA: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/create`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    namespace: [
      {
        name: 'DemoFactory',
        type: 'PATH',
        children: [
          {
            name: 'Site_01',
            type: 'PATH',
            children: [
              {
                name: 'Production',
                type: 'PATH',
                children: [
                  {
                    name: 'Produced_Qty',
                    type: 'TOPIC',
                    topicType: 'METRIC',
                    displayName: 'Produced Quantity',
                    description: 'Produced quantity from the production
line.',
                    enableHistory: true,
                    fields: [
                      { name: 'value', type: 'DOUBLE', unit: 'pcs' },
                    ],
                  },
                ],
              },
            ],
          },
        ],
      },
    ],
  }),
});

const result = await response.json();
```

```
console.log(result.data.results);
```

PESTAÑA: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/create",
    headers={"x-api-key": api_key},
    json={
        "namespace": [
            {
                "name": "DemoFactory",
                "type": "PATH",
                "children": [
                    {
                        "name": "Site_01",
                        "type": "PATH",
                        "children": [
                            {
                                "name": "Production",
                                "type": "PATH",
                                "children": [
                                    {
                                        "name": "Produced_Qty",
                                        "type": "TOPIC",
                                        "topicType": "METRIC",
                                        "displayName": "Produced
Quantity",
                                        "description": "Produced quantity
from the production line.",
                                        "enableHistory": True,
                                        "fields": [
                                            {"name": "value", "type":
"DOUBLE", "unit": "pcs"},
                                        ],
                                    },
                                ],
                            },
                        ],
                    },
                ],
            },
```

```
        ],
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"]["results"])
```

PESTAÑA: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/create" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "namespace": [
      {
        "name": "DemoFactory",
        "type": "PATH",
        "children": [
          {
            "name": "Site_01",
            "type": "PATH",
            "children": [
              {
                "name": "Production",
                "type": "PATH",
                "children": [
                  {
                    "name": "Produced_Qty",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "displayName": "Produced Quantity",
                    "description": "Produced quantity from the production
line.",
                    "enableHistory": true,
                    "fields": [
                      { "name": "value", "type": "DOUBLE", "unit": "pcs"
}
                    ]
                  }
                ]
              }
            ]
          }
        ]
      }
```

```
    ]
  }'
```

- `/uns/read`

  Lee el valor más reciente y los metadatos de uno o varios UNS topics.

PESTAÑA: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/read`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    include_leaf_value: true,
    include_metadata: true,
  }),
});

const result = await response.json();
console.log(result.data.results);
```

PESTAÑA: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/read",
    headers={"x-api-key": api_key},
    json={
```

```
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "include_leaf_value": True,
        "include_metadata": True,
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"]["results"])
```

PESTAÑA: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/read" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "include_leaf_value": true,
    "include_metadata": true
  }'
```

- `/uns/update`

  Actualiza un UNS node por `path`. Usa `updateMask` para especificar qué campos deben cambiar.

PESTAÑA: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/update`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
```

```
  },
  body: JSON.stringify({
    path: 'DemoFactory/Site_01/Production/Produced_Qty',
    displayName: 'Produced Quantity',
    description: 'Updated production output quantity.',
    updateMask: ['displayName', 'description'],
  }),
});

const result = await response.json();
console.log(result.data);
```

PESTAÑA: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]

response = requests.post(
    f"{base_url}/uns/update",
    headers={"x-api-key": api_key},
    json={
        "path": "DemoFactory/Site_01/Production/Produced_Qty",
        "displayName": "Produced Quantity",
        "description": "Updated production output quantity.",
        "updateMask": ["displayName", "description"],
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

PESTAÑA: CURL

```
curl -X POST "$TIER0_BASE_URL/uns/update" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
```

```
    "path": "DemoFactory/Site_01/Production/Produced_Qty",
    "displayName": "Produced Quantity",
    "description": "Updated production output quantity.",
    "updateMask": ["displayName", "description"]
  }'
```

- `/uns/delete`

  Elimina uno o varios UNS models. Usa `hard_delete: false` para mover los models a la papelera.

PESTAÑA: JAVASCRIPT

```
const BASE_URL = process.env.TIER0_BASE_URL;
const API_KEY = process.env.TIER0_API_KEY;

const response = await fetch(`${BASE_URL}/uns/delete`, {
  method: 'POST',
  headers: {
    'content-type': 'application/json',
    'x-api-key': API_KEY,
  },
  body: JSON.stringify({
    topics: [
      'DemoFactory/Site_01/Production/Produced_Qty',
    ],
    hard_delete: false,
  }),
});

const result = await response.json();
console.log(result.data);
```

PESTAÑA: PYTHON

```
import os
import requests

base_url = os.environ["TIER0_BASE_URL"]
api_key = os.environ["TIER0_API_KEY"]
```

```python
response = requests.post(
    f"{base_url}/uns/delete",
    headers={"x-api-key": api_key},
    json={
        "topics": [
            "DemoFactory/Site_01/Production/Produced_Qty",
        ],
        "hard_delete": False,
    },
    timeout=30,
)
response.raise_for_status()

result = response.json()
print(result["data"])
```

PESTAÑA: CURL

```bash
curl -X POST "$TIER0_BASE_URL/uns/delete" \
  -H "content-type: application/json" \
  -H "x-api-key: $TIER0_API_KEY" \
  -d '{
    "topics": [
      "DemoFactory/Site_01/Production/Produced_Qty"
    ],
    "hard_delete": false
  }'
```

Referencia de API

- Referencia OpenAPI de Tier0

> ⚠ **Limitaciones de API key en Tier0 Edge**
> La API key obtenida desde Tier0 Edge solo es válida para las API de UNS y Flows.

Siguiente

- Crear apps sobre UNS — Crea aplicaciones industriales con datos UNS.
- Analizar datos — Analiza datos UNS con Marimo Notebook y Python.

USO DE TIER0

Crear apps

Crea aplicaciones industriales mediante conversaciones con IA, usando datos de fábrica, control de acceso, gestión de versiones y despliegue.

Crea aplicaciones industriales personalizadas mediante conversaciones con IA. Las aplicaciones pueden usar datos de fábrica e incluir capacidades integradas como control de acceso basado en roles, gestión de versiones y despliegue.

Crear una app de Tier0

Antes de crear una aplicación, crea un project.

- El project se usa para clasificar aplicaciones y gestionar el control de acceso.
- Las aplicaciones dentro de un mismo project pueden funcionar como módulos relacionados de un sistema complejo.
- Los datos de cada project son independientes de los datos de otros projects.

Escribir el prompt

Para que el agent genere una aplicación que se ajuste mejor a tus requisitos, asegúrate de que el prompt cubra ***al menos*** los siguientes aspectos.

- **Propósito de la aplicación**: ¿Cuál es el objetivo de la aplicación?

  ```
  Crea una aplicación de mantenimiento de equipos para técnicos de fábrica.
  ```

- **Datos de la aplicación**: ¿Qué datos usa la aplicación y dónde están?

  PESTAÑA: PLATFORM PLAN

  Las aplicaciones pueden consumir datos de UNS y publicar datos en UNS indicando el UNS topic en el prompt.

  ```
  ```

```
La aplicación debe usar el estado de los equipos, los registros de
alarmas y el historial de mantenimiento del modelo UNS xxx, y
publicar actualizaciones en el UNS topic xxx.
```

PESTAÑA: BUILDER PLAN

La aplicación no se conecta a UNS. Los datos generados durante el uso de la aplicación se almacenan en la base de datos.

- **Función de la aplicación**: ¿Qué debe hacer la aplicación?

```
La aplicación debe permitir a los usuarios supervisar el estado de los
equipos, enviar solicitudes de mantenimiento y seguir el progreso de las
reparaciones.
```

> ⚠ **Cuando estás en Builder**
> El plan **Builder** actualmente no admite UNS, pero los datos simulados están disponibles. Para necesidades de integración de datos más específicas, contacta con nuestro equipo en sales@freezonex.io.

> 🚀 **Para crear aplicaciones más rápido**
> - Las plantillas de aplicaciones industriales comunes están disponibles en **Library**. Puedes usar una plantilla y modificar sus funciones para crear una aplicación mucho más rápido.
> - Usa las **Skills integradas de Builder** para ayudarte a crear aplicaciones.
> - Usa `/clear` para borrar el contexto y `/compact` para comprimirlo cuando la conversación sea demasiado larga.

Control de acceso

El control de acceso basado en roles se aplica tanto a los projects como a las aplicaciones.

- Niveles de acceso del project
  - **Owner**: Acceso completo al project.

    - **Can Edit**: Acceso completo al project, excepto eliminarlo.
    - **Can Launch**: Solo puede ver el project en **Launchpad** y usar las aplicaciones permitidas.
- Roles de la aplicación
    - El agent los genera automáticamente según la lógica de negocio de la aplicación cuando no se definen explícitamente. También se pueden reutilizar roles existentes de otras aplicaciones cuando sus responsabilidades coinciden.
    - Solo pueden asignarse a miembros del project.
    - La aplicación queda abierta a todos los miembros del project si el prompt especifica explícitamente que no se requieren roles.

> 🚀 **Si quieres asignar tus propios roles**
> Indica los roles y el alcance de permisos en el prompt. El agent los comparará con los roles existentes generados desde otras aplicaciones. Si hay un conflicto, el agent te pedirá confirmación.

> ⚠ **La asignación de roles de aplicación es obligatoria para aplicaciones controladas por roles**
> Para acceder a una aplicación controlada por roles en **Launchpad**, debes tener roles de aplicación asignados. Incluso un workspace/project owner no puede acceder a la aplicación sin un rol de aplicación.

Gestión de versiones

Las aplicaciones existen en Tier0 en 3 estados.

- **Draft**: El estado más reciente de la aplicación. Los cambios en esta versión no afectan a otras versiones si no se despliega.
- **Snapshot**: Guarda la aplicación en el momento de crear el snapshot. Desplegar una versión snapshot reemplaza la versión **Active** actual.
- **Active**: La versión actual de la aplicación en línea.

> 🚀 **Cambio de versión**
> - Al desplegar un draft, la aplicación active se actualiza a una versión más nueva.

- Al desplegar un snapshot, la aplicación active vuelve a una versión anterior.

Exportar a entornos externos

Cuando la aplicación esté lista, se puede exportar desde la página **Preview** tanto para terceros externos como para Tier0 Enterprise.

- **Download Source Code**: Exporta el código fuente de la aplicación para desarrollo secundario.
- **Export Bundle**: Exportación específica para importación local en **Tier0 Enterprise**.

> 🚀 **Descargar el código fuente de un snapshot**
> Descarga el snapshot después de guardarlo para obtener el código fuente de esa versión.

Siguiente

- Analizar datos - Analiza datos UNS con Marimo Notebook y Python.
- Mejor práctica: conectar protocolos industriales - Ejemplos de conexión de fuentes de datos comunes.

USO DE TIER0

Construir Digital Twins

Vincula datos UNS en vivo a modelos 3D y compón una escena completa de fábrica para visualización de Digital Twin.

Los datos de fábrica conectados a Tier0 se pueden visualizar como un Digital Twin con movimientos y entorno realistas. El proceso va desde mapear datos a modelos 3D hasta combinar esos modelos para crear una escena completa de fábrica.

> ⓘ **¿Buscas dashboards 2D?**
> Esta página trata sobre **visualización 3D de Digital Twin** — cómo mapear datos UNS a modelos 3D y componer escenas de fábrica. Para **dashboards analíticos 2D** (gráficos, tablas, paneles KPI) construidos con Notebook, consulta **Building Analytics Apps**.

Mapear datos a modelos 3D

1. Después de subir un modelo en **Model**, selecciona los nodos que se mapearán con datos UNS para movimientos en tiempo real.
2. Añade instancias con distintos datos UNS sobre el modelo para simular distintas máquinas.
3. Vincula los datos al modelo y usa **Preview** para comprobar si se mueve y rota cuando cambian los datos.
    - **Position**: El modelo se mueve en la dirección seleccionada del sistema de coordenadas.
    - **Rotation**: El modelo rota alrededor del punto de origen de las coordenadas.
4. Usa **Data Tagging** para añadir etiquetas destinadas a mostrar información estática.

Visualizar datos

1. Añade desde **Instance Library** las instancias que deberían existir en la escena.
2. Usa las herramientas superiores para mover, escalar y rotar los modelos hasta ajustarlos a la escena, y para cambiar la perspectiva de visualización y la posición de coordenadas.

3. La posición del modelo se puede confirmar con **Object** en la esquina derecha, donde se muestran los datos UNS vinculados.
4. El viewport y la iluminación sirven para ajustar detalles del entorno.

Uso compartido externo

Solo las instancias se pueden compartir, ya sea por modelo o por escena.

- **Share Model**: Genera un enlace de la plataforma Tier0 donde se muestra el modelo con datos UNS en vivo.
- **Share Scene**: Genera un enlace de la plataforma Tier0 donde se muestran el modelo y su escena, con datos UNS en vivo y ajustes del entorno.

USO DE TIER0

Analizar datos

Consulta datos vivos del namespace en Tier0 Notebook y crea análisis interactivos y apps de machine learning. Disponible en Cloud y Enterprise.

Marimo Notebook se usa en Tier0 para análisis avanzado de datos con scripts de **Python**, a partir de los datos de fábrica modelados y conectados a UNS.

> **Antes de empezar**
> Los datos conectados a modelos UNS con **Enable History** activado están disponibles como una fuente de datos existente en **Notebook**.

1. Los datos UNS están en **VARIABLES** > **PostgreSQL** > **tenant_id** > **uns**.
2. Usa **@** para incluir fuentes de datos al usar **GENERATE WITH AI**. La búsqueda de fuentes de datos admite coincidencia difusa.
3. Guarda versiones de tu análisis para futuros rollbacks.

> **AI assist en Notebook**
> - AI assist está disponible en Tier0 Cloud.
> - En Tier0 Enterprise debes proporcionar una key para AI assist.

Siguiente

- Crear apps sobre UNS - Crea aplicaciones industriales con datos UNS.
- Conceptos UNS - Comprende el modelado de datos en Unified Namespace.

USO DE TIER0

Operar Tier0 con Agents

Usa AI agents para operar Tier0 mediante Skills y comandos CLI.

Tier0 proporciona Skills y comandos CLI que permiten a los AI agents operar la plataforma.

Trabaja en Tier0 con tu AI agent preferido.

Claude Code Codex Gemini CLI Cursor GitHub Copilot

Instalar Skills y CLI

Elige el instalador que coincida con tu entorno.

Con Node.js

Si Node.js 16 o posterior está disponible, usa `npx` para instalar el CLI y los skills.

```
npx -y @tier0/cli@latest install
```

Esto ejecuta el instalador completo y hace dos cosas:

- Instala el binario CLI `tier0` en `~/.tier0/bin/`.
- Instala la base integrada de Tier0 Skill y la sincroniza con agents detectados como Codex, Claude Code y más.

Sin Node.js

1 Instala el binario CLI `tier0`.

PESTAÑA: CURL

```
curl -fsSL https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.sh | bash
```

PESTAÑA: IWR

```
iwr https://raw.githubusercontent.com/FREEZONEX/Tier0-cli/main/install.ps1 | iex
```

2 Descarga los Tier0 Skills más recientes y sincronízalos con los agents detectados.

```
tier0 skills update
```

3 Verifica la versión y el estado de salud del Skill instalado.

```
tier0 skills status
```

Gestionar Skills

- Comprueba la versión y el estado de salud del Skill instalado:

```
tier0 skills status
```

- Repara una base integrada de Skill ausente o dañada:

```
tier0 skills install
```

- Actualiza al Tier0 Skill más reciente:

```

```

```
tier0 skills update
```

Cómo trabajan juntos Skills y CLI

Tier0 CLI proporciona comandos ejecutables, y las Skills guían a los agents para elegir y usar esos comandos de forma segura.

> **Nota**
> Tier0 Skills están diseñadas para activarse automáticamente mediante prompts y guiar a los agents sobre qué hacer y qué comandos usar.

Ejemplo

- Prompt: Comprueba la temperatura actual de `Plant/Line1/Metric/Temperature`.
- Skill
  - `tier0-uns`
  - Indica al agent que el valor actual debe leerse desde un topic path completo y selecciona la operación de lectura UNS adecuada.
- CLI: `tier0 uns read Plant/Line1/Metric/Temperature --json`

Skills

Skill	Propósito
`tier0`	Enruta tareas de Tier0 al skill adecuado y proporciona reglas compartidas de configuración, autenticación y seguridad.
`tier0-uns`	Guía a los agents en operaciones UNS, como browse, read, write, history, search, create, update, delete y restore.
`tier0-flow`	Guía a los agents en la gestión de SourceFlow y EventFlow, como inspección,

Skill	Propósito
	exportación, creación, actualización, despliegue y eliminación.
`tier0-mqtt`	Guía a los agents en la gestión de credenciales MQTT, publicación y suscripciones en tiempo real acotadas.
`tier0-files`	Guía a los agents en operaciones de archivos de Tier0, como upload, download, generación de URL de acceso y delete.

> **Usa un agent para leer el skill repo**
> Pide a tu agent que lea el **Tier0 Skill repo** para ver los detalles.

Operaciones CLI

Configurar la instancia operativa

> **Nota**
> Cloud usa `https://tier0.dev` por defecto y no necesita configuración de base URL.

Los despliegues privados deben configurar la base URL antes del login. Una vez configurada, todos los comandos siguientes se ejecutarán sobre esa instancia.

```
tier0 config --base-url https://your-tier0.example.com
tier0 login # interactive browser login
```

Autenticación

PESTAÑA: CON API KEY

1. Configura la API key obtenida de la instancia Tier0 operativa.

```
tier0 config --api-key <api-key>
```

2. Verifica la identidad y permisos de la API key.

```
tier0 auth whoami --json
```

3. Realiza otras operaciones con el CLI `tier0`.

PESTAÑA: SIN API KEY

1. Obtén el enlace de autenticación en navegador.

```
tier0 login --no-wait --json
```

2. Completa la autenticación en el navegador accediendo al `verification_url` devuelto.
3. Consulta el resultado del login con el `setup_code` devuelto.

```
tier0 login --setup-code <setup_code>
```

> ⚠ **Importante**
> Ejecuta `config` para definir la instancia operativa antes de configurar una API key o iniciar el login en navegador; de lo contrario, el CLI puede operar sobre la instancia equivocada.

Previsualizar operaciones

Usa `--dry-run` para validar y previsualizar operaciones de escritura compatibles sin enviar una solicitud a Tier0.

Operaciones compatibles

- UNS: `write/create/update/delete/restore`
- Flow: `create/update/delete/deploy`
- API: `tier0 api`
- MQTT: `auth create/auth delete/publish`
- Files: `delete`

Ejemplos

```
tier0 uns write --topic demo --value '{"value":1}' --dry-run
tier0 flow deploy --id 1 --flows-file flows.json --dry-run --json
tier0 api /openapi/v1/uns/write --body-file body.json --dry-run --json
tier0 mqtt auth create --name my-client --save my-client --dry-run --json
tier0 assets delete --file-path workspace/path/to/file.csv --dry-run --json
```

> ⓘ **Nota**
> La opción `--json` genera la solicitud en formato JSON.

> ⚠ **El request body debe contener JSON válido**
> El CLI ya no intenta adivinar ni reescribir JSON mal formado si no se proporciona JSON válido.

Previsualizaciones devueltas

> ⓘ **Nota**
> Los headers y API keys no se muestran en la salida de dry-run.

```
{
  "ok": true,
  "dry_run": true,
  "data": {
    "api": [
```

```
      {"method": "POST", "url": "https://tier0.dev/openapi/v1/uns/write", "body":
{}}
    ]
  }
}
```

Manejar errores

Con `--json`, los fallos de comando devuelven un objeto JSON con el mensaje y el código de error.

```
{"ok":false,"error":
{"type":"validation","subtype":"invalid_argument","param":"--qos","message":"-
-qos must be 0, 1, or 2"}}
```

Desinstalar Skills y CLI

- Comandos de desinstalación

```
# Elimina el CLI de Tier0 y conserva la configuración local y los skills.
npx @tier0/cli@latest uninstall

# Elimina el CLI de Tier0 y borra la configuración local, credenciales y caché,
pero conserva los skills.
npx @tier0/cli@latest uninstall --purge

# Elimina los skills
npx @tier0/cli@latest uninstall --remove-skills

# Elimina todo lo anterior
npx @tier0/cli@latest uninstall --purge --remove-skills
```

- Desinstalación global de NPM

```
npm uninstall -g @tier0/cli
```

- Limpieza manual

```
rm -rf ~/.tier0/bin/tier0 ~/.tier0/skills
npx -y --package=skills -- skills remove tier0 -y -g
```

Referencia CLI

Referencia CLI de Tier0 lista todos los comandos CLI.

Siguiente

- Trabajar con datos de fábrica — Cómo operar sobre tu modelo de datos de fábrica mediante MQTT y APIs.

Buenas prácticas

Modelar datos de fábrica

Diseño del Demo UNS: evitar topic explosion y mejores prácticas de payload structure.

Cómo está estructurado Demo UNS

> **Qué hacer primero**
> Antes de modelar los datos, responde estas preguntas.
>
> - ¿Qué datos se consumirán desde UNS?
> - ¿Qué datos se publicarán de vuelta a UNS?

Datos físicos

Los datos industriales de máquinas y equipos con ubicaciones físicas se modelan normalmente según el estándar ISA-95.

Modelo demo

PESTAÑA: ISA-95 HIERARCHY

```
Enterprise (Level 4)
└── Site (Level 3/4)
    └── Area (Level 3)
        └── Work Center / Process Cell (Level 2/3)
            └── Equipment / Unit (Level 1/2)
                └── Control Module / Device (Level 0/1)
```

PESTAÑA: TREE STRUCTURE

```
DemoFactory
└── Site_01
    └── Production
        └── Line_01
```

```
            └── WorkOrderExecution
                ├── Metric
                │   ├── Target_Qty
                │   ├── Produced_Qty
                │   ├── Defect_Qty
                │   └── Completion_Rate
                └── State
                    ├── CurrentWorkOrder
                    └── WorkOrderStatus
```

PESTAÑA: JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_9e50bbcd833d",
      "type": "PATH",
      "children": [
        {
          "name": "Site_01",
          "alias": "DemoFactory__6d1fcad14e86",
          "type": "PATH",
          "children": [
            {
              "name": "Production",
              "alias": "__Site_01_Pro_91523d2ad304",
              "type": "PATH",
              "children": [
                {
                  "name": "Line_01",
                  "alias": "__Production__acd5841c1ada",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "WorkOrderExecution",
                      "alias": "__Line_01_Wor_54904ff56e91",
                      "type": "PATH",
                      "children": [
                        {
                          "name": "Metric",
                          "alias": "metric_233fd84305c4",
                          "type": "PATH",
                          "children": [
                            {
                              "name": "Target_Qty",
                              "alias": "Target_Qty_8646b092e543",
```

```
                            "type": "TOPIC",
                            "topicType": "METRIC",
                            "description": "Target quantity for the current
Work Order.",
                            "fields": [
                              {
                                "name": "production_order_id",
                                "type": "STRING"
                              },
                              {
                                "name": "plan_id",
                                "type": "STRING"
                              },
                              {
                                "name": "plan_number",
                                "type": "STRING"
                              },
                              {
                                "name": "work_order_id",
                                "type": "STRING"
                              },
                              {
                                "name": "value",
                                "type": "DOUBLE"
                              },
                              {
                                "name": "unit",
                                "type": "STRING"
                              },
                              {
                                "name": "timestamp",
                                "type": "DATETIME"
                              }
                            ],
                            "enableHistory": "TRUE",
                            "mockData": "FALSE"
                          },
                          {
                            "name": "Produced_Qty",
                            "alias": "Produced_Qty_1f8cd3e11f77",
                            "type": "TOPIC",
                            "topicType": "METRIC",
                            "description": "Produced quantity for the
current Work Order.",
                            "fields": [
                              {
                                "name": "production_order_id",
                                "type": "STRING"
                              },
                              {
                                "name": "plan_id",
```

```
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
      },
      {
        "name": "Defect_Qty",
        "alias": "Defect_Qty_b48fb3563ec1",
        "type": "TOPIC",
        "topicType": "METRIC",
        "description": "Defect quantity for the current
Work Order.",
        "fields": [
          {
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
```

```
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
        ],
        "enableHistory": "TRUE",
        "mockData": "FALSE"
      },
      {
        "name": "Completion_Rate",
        "alias": "Completion_R_7294e38e1530",
        "type": "TOPIC",
        "topicType": "METRIC",
        "description": "Completion percentage for the
current Work Order.",
        "fields": [
          {
            "name": "production_order_id",
            "type": "STRING"
          },
          {
            "name": "plan_id",
            "type": "STRING"
          },
          {
            "name": "plan_number",
            "type": "STRING"
          },
          {
            "name": "work_order_id",
            "type": "STRING"
          },
          {
            "name": "value",
            "type": "DOUBLE"
          },
          {
            "name": "unit",
            "type": "STRING"
          },
          {
            "name": "timestamp",
            "type": "DATETIME"
          }
```

```
                ],
                "enableHistory": "TRUE",
                "mockData": "FALSE"
              }
            ]
          },
          {
            "name": "State",
            "alias": "state_539a0f331fe0",
            "type": "PATH",
            "children": [
              {
                "name": "CurrentWorkOrder",
                "alias": "CurrentWorkO_a638320784dd",
                "type": "TOPIC",
                "topicType": "STATE",
                "description": "Current active Work Order on
Line_01.",
                "fields": [
                  {
                    "name": "production_order_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_id",
                    "type": "STRING"
                  },
                  {
                    "name": "plan_number",
                    "type": "STRING"
                  },
                  {
                    "name": "work_order_id",
                    "type": "STRING"
                  },
                  {
                    "name": "target_line",
                    "type": "STRING"
                  },
                  {
                    "name": "workstation",
                    "type": "STRING"
                  },
                  {
                    "name": "product_code",
                    "type": "STRING"
                  },
                  {
                    "name": "target_qty",
                    "type": "DOUBLE"
                  },
```

```
                {
                  "name": "unit",
                  "type": "STRING"
                },
                {
                  "name": "started_at",
                  "type": "DATETIME"
                },
                {
                  "name": "updated_at",
                  "type": "DATETIME"
                }
              ],
              "enableHistory": "TRUE",
              "mockData": "FALSE"
            },
            {
              "name": "WorkOrderStatus",
              "alias": "WorkOrderSta_e24e3bf113d1",
              "type": "TOPIC",
              "topicType": "STATE",
              "description": "Execution status of the current
Work Order.",
              "fields": [
                {
                  "name": "production_order_id",
                  "type": "STRING"
                },
                {
                  "name": "plan_id",
                  "type": "STRING"
                },
                {
                  "name": "plan_number",
                  "type": "STRING"
                },
                {
                  "name": "work_order_id",
                  "type": "STRING"
                },
                {
                  "name": "status",
                  "type": "STRING"
                },
                {
                  "name": "reason",
                  "type": "STRING"
                },
                {
                  "name": "updated_at",
                  "type": "DATETIME"
```

```
                        }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                }
            ]
        }
    ]
}
```

Datos de negocio

Para objetos de planificación y proceso de negocio, como órdenes de producción y planes de trabajo en un ERP, modela los datos según su contexto de negocio.

> **How to recognize business data**
> Datos que no tienen una ubicación física.

Guía de modelado de datos de negocio

```
Factory / Business Domain / Business Object / Topic Type / Topic
```

- **Business Domain**: Confirma el proceso o sistema de negocio al que pertenecen los datos.

```
ERP
MES
WMS
QMS
```

```
EAM
LIMS
Planning
Inventory
Maintenance
Quality
Logistics
...
```

- **Business Object**: El objeto gestionado principal dentro del dominio de negocio.

```
ProductionOrders
WorkOrderPlan
MaterialInventory
QualityInspection
MaintenanceOrders
PurchaseOrders
Shipments
...
```

- **Topic Type**: Elige el tipo de dato entre **Metric**, **Action** y **State**.

Topic Type	Used for	Examples
STATE	Estado actual o hechos actuales	Estado de orden de trabajo, estado de orden, estado de equipo
METRIC	Valores numéricos que cambian continuamente	Temperatura, velocidad, producción, OEE, consumo de energía
ACTION	Comandos para ejecutores externos	Iniciar equipo, detener equipo, restablecer alarma, solicitar despacho de orden de trabajo

Modelo demo

PESTAÑA: TREE STRUCTURE

```
DemoFactory
└── ERP
    ├── ProductionOrders
    │   └── State
    │       ├── UpsertProductionOrder
    │       └── OrderList
    └── WorkOrderPlan
        ├── Metric
        │   └── SplitCount
        └── State
            ├── PlanStatus
            └── WorkOrderList
```

PESTAÑA: JSON


```
{
  "namespace": [
    {
      "name": "DemoFactory",
      "alias": "DemoFactory_f925c1ec5041",
      "type": "PATH",
      "children": [
        {
          "name": "ERP",
          "alias": "DemoFactory__e504805b1973",
          "type": "PATH",
          "children": [
            {
              "name": "ProductionOrders",
              "alias": "__ERP_Product_1e7984ac3a1d",
              "type": "PATH",
              "children": [
                {
                  "name": "State",
                  "alias": "state_c2616c8698fa",
                  "type": "PATH",
                  "children": [
                    {
                      "name": "UpsertProductionOrder",
                      "alias": "UpsertProduc_d5f166da35c6",
                      "type": "TOPIC",
                      "topicType": "STATE",
                      "description": "Event for creating or updating a
Production Order from WOM or ERP simulation.",
```

```
      "fields": [
        {
          "name": "event_id",
          "type": "STRING"
        },
        {
          "name": "event_type",
          "type": "STRING"
        },
        {
          "name": "source",
          "type": "STRING"
        },
        {
          "name": "production_order_id",
          "type": "STRING"
        },
        {
          "name": "order_number",
          "type": "STRING"
        },
        {
          "name": "product_code",
          "type": "STRING"
        },
        {
          "name": "planned_qty",
          "type": "DOUBLE"
        },
        {
          "name": "unit",
          "type": "STRING"
        },
        {
          "name": "status",
          "type": "STRING"
        },
        {
          "name": "due_date",
          "type": "DATETIME"
        },
        {
          "name": "updated_at",
          "type": "DATETIME"
        }
      ],
      "enableHistory": "TRUE",
      "mockData": "FALSE"
    },
    {
      "name": "OrderList",
```

```
                    "alias": "OrderList_11fbca51a2bd",
                    "type": "TOPIC",
                    "topicType": "STATE",
                    "description": "Retained compatibility snapshot of
current Production Orders.",
                    "fields": [
                      {
                        "name": "orders",
                        "type": "STRING"
                      },
                      {
                        "name": "updated_at",
                        "type": "DATETIME"
                      }
                    ],
                    "enableHistory": "TRUE",
                    "mockData": "FALSE"
                  }
                ]
              }
            ]
          },
          {
            "name": "WorkOrderPlan",
            "alias": "__ERP_WorkOrd_19f04960daf5",
            "type": "PATH",
            "children": [
              {
                "name": "Metric",
                "alias": "metric_17258b809a4f",
                "type": "PATH",
                "children": [
                  {
                    "name": "SplitCount",
                    "alias": "SplitCount_032e86f5fbfc",
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "description": "Number of executable Work Orders
generated from a Work Order Plan.",
                    "fields": [
                      {
                        "name": "production_order_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_id",
                        "type": "STRING"
                      },
                      {
                        "name": "plan_number",
                        "type": "STRING"
```

```
                    },
                    {
                      "name": "split_count",
                      "type": "INTEGER"
                    },
                    {
                      "name": "updated_at",
                      "type": "DATETIME"
                    }
                  ],
                  "enableHistory": "TRUE",
                  "mockData": "FALSE"
                }
              ]
            },
            {
              "name": "State",
              "alias": "state_884cee548bf7",
              "type": "PATH",
              "children": [
                {
                  "name": "PlanStatus",
                  "alias": "PlanStatus_2e21c1327c49",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Current lifecycle status of a Work Order
Plan.",
                  "fields": [
                    {
                      "name": "production_order_id",
                      "type": "STRING"
                    },
                    {
                      "name": "plan_id",
                      "type": "STRING"
                    },
                    {
                      "name": "plan_number",
                      "type": "STRING"
                    },
                    {
                      "name": "status",
                      "type": "STRING"
                    },
                    {
                      "name": "updated_at",
                      "type": "DATETIME"
                    }
                  ],
                  "enableHistory": "TRUE",
                  "mockData": "FALSE"
```

```
                    },
                    {
                      "name": "WorkOrderList",
                      "alias": "WorkOrderLis_887b92f26fc2",
                      "type": "TOPIC",
                      "topicType": "STATE",
                      "description": "Split result list generated from a sent
Work Order Plan.",
                      "fields": [
                        {
                          "name": "production_order_id",
                          "type": "STRING"
                        },
                        {
                          "name": "plan_id",
                          "type": "STRING"
                        },
                        {
                          "name": "plan_number",
                          "type": "STRING"
                        },
                        {
                          "name": "split_count",
                          "type": "INTEGER"
                        },
                        {
                          "name": "lines",
                          "type": "STRING"
                        },
                        {
                          "name": "updated_at",
                          "type": "DATETIME"
                        }
                      ],
                      "enableHistory": "TRUE",
                      "mockData": "FALSE"
                    }
                  ]
                }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

Cómo evitar la explosión de topics

Tier0 uses 3 fixed topic types: `Metric`, `Action`, and `State` to avoid topic explosion.

- **Usa `Metric` para datos en tiempo real de alta frecuencia**

  Coloca mediciones y telemetría de alta frecuencia bajo `Metric` en lugar de ampliar el namespace para cada detalle de señal.

PESTAÑA: BAD

```
Factory_A
└── SMT_Line_1
    └── Machine_001
        ├── Temperature_Value
        ├── Temperature_Unit
        ├── Temperature_Status
        └── Speed_Value
```

PESTAÑA: BETTER

```
Factory_A
└── SMT_Line_1
    └── Metric
        └── Machine_001
```

Model fields:

```
{
  "temperature": 80,
  "temperature_unit": "Celsius",
  "temperature_status": "Normal",
  "speed": 300
}
```

- **Usa `State` para datos de estado de baja frecuencia**

Coloca estados de baja frecuencia, snapshots y estado de negocio actual bajo `State`. Evita crear un topic nuevo para cada entidad cambiante, como trabajos, lotes u órdenes.

PESTAÑA: BAD

```
Factory_A
└── SMT_Line_1
    └── State
        ├── Job_001
        ├── Job_002
        └── Job_003
```

PESTAÑA: BETTER

```
Factory_A
└── SMT_Line_1
    └── State
        └── Machine_001
```

Model fields:

```
{
  "current_job": "Job_003",
  "product": "Product_A"
}
```

- **Usa `Action` para datos de acción de baja frecuencia**

  Coloca comandos, solicitudes y acciones de operador bajo `Action`. Mantén las entradas de la acción como campos de un topic en lugar de crear un topic por parámetro.

PESTAÑA: BAD

```
Factory_A
```

```
└── SMT_Line_1
    └── Machine_001
        ├── Start_Command
        ├── Target_Height
        └── Light_On
```

PESTAÑA: BETTER

```
Factory_A
└── SMT_Line_1
    └── Action
        └── StartJob
```

Model fields:

```
{
  "target_height": 12.5,
  "light_on": true,
  "requested_by": "operator"
}
```

Estructura del payload

Al estructurar el topic payload, considera tanto los datos que se consumen como los datos que se escriben de vuelta.

- **Action**

  Los payloads de Action incluyen automáticamente `_timestamp`, que registra cuándo se almacenan los datos en la base de datos de Tier0.

PESTAÑA: START_JOB

```
{
  "light_turn_on": true,
```

```
  "height_adjust": 12.5,
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

PESTAÑA: STOP_JOB

```
{
  "water_pump_off": true,
  "window_close": true,
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

- **Metric**

  Los payloads de Metric incluyen automáticamente `_timestamp`, que registra cuándo se almacenan los datos en la base de datos de Tier0, y `_quality`, que representa la calidad de los datos.

PESTAÑA: ERP / SPLITCOUNT

```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
  "split_count": 3,
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z",
  "_quality": "Good"
}
```

PESTAÑA: SITE_01 / TARGET_QTY

```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
```

```
  "work_order_id": "WO-001",
  "value": 1200,
  "unit": "pcs",
  "timestamp": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z",
  "_quality": "Good"
}
```

- **State**

  Los payloads de State incluyen automáticamente `_timestamp`, que registra cuándo se almacenan los datos en la base de datos de Tier0.

PESTAÑA: ERP / UPSERTPRODUCTIONORDER



```
{
  "event_id": "EVT-001",
  "event_type": "UPSERT",
  "source": "ERP",
  "production_order_id": "PO-20260710-001",
  "order_number": "PO-001",
  "product_code": "SKU-1001",
  "planned_qty": 1200,
  "unit": "pcs",
  "status": "Released",
  "due_date": "2026-07-12T00:00:00Z",
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z"
}
```

PESTAÑA: SITE_01 / WORKORDERSTATUS



```
{
  "production_order_id": "PO-20260710-001",
  "plan_id": "PLAN-001",
  "plan_number": "WOP-20260710-001",
  "work_order_id": "WO-001",
  "status": "Running",
  "reason": "In production",
  "updated_at": "2026-07-10T10:00:00Z",
  "_timestamp": "2026-07-10T10:00:00Z"
```

```
}
```

BUENAS PRÁCTICAS

Conectar protocolos industriales

Conecta OPC UA, Modbus y otros dispositivos de protocolo típicos a UNS con Node-RED, con referencias reales de flow JSON.

Esta sección usa tipos de datos comunes, incluidos OPC UA, Modbus y API, como ejemplos para demostrar el proceso de conectar fuentes de datos a **UNS** mediante **Source Flow**.

OPC UA

Configurar Data Flow

1. Inicia sesión en Tier0, ve a **Flow** y crea un nuevo **Source Flow**.
2. Agrega los nodos **inject**, **OPC UA Client** y **mqtt out** al flow.
3. Conéctalos en orden y haz doble clic en cada nodo para configurar los detalles.

- **inject**

  1. Haz clic en **add** en la esquina inferior izquierda para agregar una property.
  2. Configura las properties con los siguientes elementos:

```
{
  "msg.payload": "[multiple data tags]",
  "msg.topic": "multiple"
}
```

- **OPC UA Client**

```
"Endpoint": "<OPC UA server address and port>", //e.g.
opc.tcp://127.0.0.1:4850
"Action": "SUBSCRIBE",
"Interval": "<data collecting interval>" //e.g. 10 seconds
```

- **mqtt out**

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

4 Despliega y ejecuta el flow.

Referencia del flow

Mostrar OPC UA flow JSON


```
[
    {
        "id": "fn_mapper_01",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "ISA95 Mapping (Air Compressor)",
        "func": "// OPC UA NodeId → ISA-95 MQTT Topic Mapping\n\nconst mapping
= {\n    \"ns=2;i=2\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/outlet_temperature\
",\n        field: \"temperature\"\n    },\n    \"ns=2;i=3\": {\n
topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/discharge_pressure\
",\n        field: \"pressure\"\n    },\n    \"ns=2;i=4\": {\n        topic:
\"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_
Air_System/Air_Compressor_Station/Air_Compressor_01/Metric/vibration_rms\",\n
field: \"vibration\"\n    }\n};\n\nlet value = msg.payload;\n\n// OPC UA
structure normalize\nif (value && typeof value === \"object\") {\n    if
(value.value?.value !== undefined) {\n        value = value.value.value;\n
} else if (value.value !== undefined) {\n        value = value.value;\n
}\n}\n\nconst m = mapping[msg.topic];\nif (!m) return null;\n\nmsg.topic =
m.topic;\n\nmsg.payload = {\n    [m.field]: Number(value)\n};\n\nreturn msg;",
        "outputs": 1,
        "timeout": "",
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 340,
        "y": 320,
        "wires": [
            [
                "mqtt_out_01"
```

```
            ]
        ]
    },
    {
        "id": "mqtt_out_01",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "MQTT Air Compressor Metrics",
        "topic": "",
        "qos": "0",
        "retain": "false",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 570,
        "y": 200,
        "wires": []
    },
    {
        "id": "1f242deefa9fdba9",
        "type": "OpcUa-Client",
        "z": "b002a07d1163c3de",
        "endpoint": "298ef5d50e9dc411",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 220,
        "y": 220,
        "wires": [
            [
                "fn_mapper_01"
            ],
            [],
```

```
            []
        ]
    },
    {
        "id": "7be5f021ed1f10ef",
        "type": "inject",
        "z": "b002a07d1163c3de",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "multiple",
        "payload": "[{\"nodeId\":\"ns=2;i=2\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=3\",\"datatype\":\"Float\"},
{\"nodeId\":\"ns=2;i=4\",\"datatype\":\"Float\"}]",
        "payloadType": "json",
        "x": 110,
        "y": 160,
        "wires": [
            [
                "1f242deefa9fdba9"
            ]
        ]
    },
    {
        "id": "broker-djnwpg00y370",
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
```

```
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "298ef5d50e9dc411",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4841",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "a003de8af8e62c75",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-opcua": "0.2.339"
        }
    }
]
```


Modbus

Configurar Data Flow

1. En el flow, agrega los nodos **modbus-read** y **mqtt out**.
2. Haz doble clic en los nodos y configura la información requerida.
    - **modbus-read**
        - **Settings**
            - **FC**: código de función que especifica la acción Modbus, por ejemplo FC 3 = leer registros holding.
            - **Address**: dirección inicial del registro que se va a leer (normalmente empieza en 0).
            - **Quantity**: The number of consecutive registers to read.
            - **Poll Rate**: frecuencia con la que el nodo consulta el dispositivo Modbus (por ejemplo, cada 10 segundos).

        - **Server**: referencia a un Modbus server configurado (IP, port, protocol, etc.).
    - **Server**
        - **Host**: IP del Modbus server.
        - **Port**: port del Modbus server.
- **mqtt out**

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

3. Despliega y ejecuta el flow.

Referencia del flow

<details>
<summary>Mostrar Modbus flow JSON</summary>


```
[
    {
        "id": "d3becb19e0d762f4",
        "type": "modbus-read",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic": "",
        "showStatusActivities": false,
        "logIOActivities": false,
        "showErrors": false,
        "showWarnings": true,
        "unitid": "1",
        "dataType": "HoldingRegister",
        "adr": "0",
        "quantity": "1",
        "rate": "10",
        "rateUnit": "s",
        "delayOnStart": false,
        "startDelayTime": "",
        "server": "c665b74952f6cf5a",
        "useIOFile": false,
        "ioFile": "",
        "useIOForPayload": false,
        "emptyMsgOnFail": false,
```

```
        "x": 130,
        "y": 500,
        "wires": [
            [
                "2987638967f632b4",
                "09b8538e09d0766a"
            ],
            []
        ]
    },
    {
        "id": "b068c119928c029a",
        "type": "mqtt out",
        "z": "b002a07d1163c3de",
        "name": "status",
        "topic":
"global_plant/Smart_Manufacturing_Group/Suzhou_Plant/Utility_Area/Compressed_A
ir_System/Air_Compressor_Station/Air_Compressor_01/State/compressor_status",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djnwpg00y370",
        "x": 510,
        "y": 540,
        "wires": []
    },
    {
        "id": "2987638967f632b4",
        "type": "debug",
        "z": "b002a07d1163c3de",
        "name": "debug 3",
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 260,
        "y": 600,
        "wires": []
    },
    {
        "id": "09b8538e09d0766a",
        "type": "function",
        "z": "b002a07d1163c3de",
        "name": "function 1",
```

```
        "func": "const value = msg.payload;\n\n// 1️⃣ verify\nif (value === undefined || value === null) {\n    node.warn(\"Empty Modbus payload\");\n return null;\n}\n\n// 2️⃣ convert to number\nconst status_code = Number(value);\n\n// 3️⃣ mapping to status\nconst statusMap = {\n    0: \"maintenance\",\n    1: \"running\",\n    2: \"standby\",\n    3: \"debug\",\n    4: \"fault\"\n};\n\n// 4️⃣ filtering\nif (!(status_code in statusMap)) {\n    node.warn(\"Invalid status_code: \" + status_code);\n return null;\n}\n\n// 5️⃣ contextualized output\nmsg.payload = {\n status_code: status_code,\n    status_label: statusMap[status_code],\n _valid: true,\n    _ts: new Date().toISOString()\n};\n\n// 6️⃣ optional  set topic\nmsg.topic = \"v1/aircompressor01/state\";\n\nreturn msg;",
        "outputs": 1,
        "timeout": 0,
        "noerr": 0,
        "initialize": "",
        "finalize": "",
        "libs": [],
        "x": 320,
        "y": 500,
        "wires": [
            [
                "b068c119928c029a"
            ]
        ]
    },
    {
        "id": "c665b74952f6cf5a",
        "type": "modbus-client",
        "name": "modbus",
        "clienttype": "tcp",
        "bufferCommands": true,
        "stateLogEnabled": false,
        "queueLogEnabled": false,
        "failureLogEnabled": true,
        "tcpHost": "172.31.151.237",
        "tcpPort": "5020",
        "tcpType": "DEFAULT",
        "serialPort": "/dev/ttyUSB",
        "serialType": "RTU-BUFFERD",
        "serialBaudrate": 9600,
        "serialDatabits": 8,
        "serialStopbits": 1,
        "serialParity": "none",
        "serialConnectionDelay": 100,
        "serialAsciiResponseStartDelimiter": "0x3A",
        "unit_id": 1,
        "commandDelay": 1,
        "clientTimeout": 1000,
        "reconnectOnTimeout": true,
        "reconnectTimeout": 2000,
        "parallelUnitIdsAllowed": true,
```

```
        "showErrors": false,
        "showWarnings": true,
        "showLogs": true
    },
    {
        "id": "broker-djnwpg00y370",
        "type": "mqtt-broker",
        "z": "b002a07d1163c3de",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
        "clientid": "355424069147520",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "5cc203d0788c324d",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-modbus": "5.43.0"
        }
    }
]
```


API

Configurar Data Flow

1. En el flow, agrega los nodos **inject**, **http request** y **mqtt out**.
2. Haz doble clic en los nodos y configura la información requerida.
    - **http request**


```
"Method": "<API method>",
```

```
"URL": "<API URL>",
"Return": "a parsed JSON object",
"Headers": "<API headers>"
```

- **mqtt out**

```
"Server": "<embedded UNS broker>", //same name as the flow
"Topic": "<UNS model>",
```

3. Despliega y ejecuta el flow.

Referencia del flow

Mostrar API flow JSON

```
[
    {
        "id": "46273fcbb98afa11",
        "type": "inject",
        "z": "932355c6886aaf27",
        "name": "",
        "props": [
            {
                "p": "payload"
            },
            {
                "p": "topic",
                "vt": "str"
            }
        ],
        "repeat": "",
        "crontab": "",
        "once": false,
        "onceDelay": 0.1,
        "topic": "",
        "payload": "",
        "payloadType": "date",
        "x": 170,
        "y": 380,
        "wires": [
            [
                "3aca156076ad6dd1"
            ]
```

```
        ]
    },
    {
        "id": "3aca156076ad6dd1",
        "type": "http request",
        "z": "932355c6886aaf27",
        "name": "",
        "method": "GET",
        "ret": "obj",
        "paytoqs": "ignore",
        "url": "http://127.0.0.1:1880/api/wms/picking-zone-a/status",
        "tls": "",
        "persist": false,
        "proxy": "",
        "insecureHTTPParser": false,
        "authType": "",
        "senderr": false,
        "headers": [],
        "x": 380,
        "y": 380,
        "wires": [
            [
                "83fd65cf177c2ddf"
            ]
        ]
    },
    {
        "id": "83fd65cf177c2ddf",
        "type": "mqtt out",
        "z": "932355c6886aaf27",
        "name": "",
        "topic": "v1/Suzhou_Plant/Assembly-Area-1/Line-01/Station-
Screwing/Metric/torque",
        "qos": "",
        "retain": "",
        "respTopic": "",
        "contentType": "",
        "userProps": "",
        "correl": "",
        "expiry": "",
        "broker": "broker-djm245hxt9d9",
        "x": 780,
        "y": 440,
        "wires": []
    },
    {
        "id": "broker-djm245hxt9d9",
        "type": "mqtt-broker",
        "name": "emqx:1883",
        "broker": "emqx",
        "port": "1883",
```

```
        "clientid": "355039328257392",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    }
]
```

Otros aspectos que conviene saber

- Puedes arrastrar un nodo **function** para limpiar los datos según sea necesario antes de publicar en **UNS**.
- Puedes usar un nodo **debug** para revisar los datos en el momento.
- Para detalles de nodos, consulta **Node-RED flows**.

BUENAS PRÁCTICAS

Seguridad y resiliencia

Resumen de los controles de identidad, acceso a datos, disponibilidad del despliegue y recuperación de Tier0.

Tier0 protege los datos desde dos perspectivas:

- **Seguridad**: Controla quién puede acceder a la plataforma, las aplicaciones y los datos.
- **Resiliencia**: Abarca la continuidad del servicio y la recuperación de datos tras un fallo.

Ámbito de seguridad por edición

Edición	Cobertura de seguridad
Cloud	Control de acceso a la plataforma y acceso autenticado a MQTT y las API.
Enterprise	Control de acceso a la plataforma y los datos, además de disponibilidad del despliegue, copias de seguridad y recuperación para despliegues privados.
Edge (código abierto)	Conexión y modelado de datos locales. Su broker MQTT integrado permite suscripciones sin autenticación.

Consulta Elegir el mejor producto para conocer el enfoque de cada edición.

> ⓘ **Seguridad de Tier0 Cloud**
> Para conocer los límites de red, el aislamiento de inquilinos, la protección de datos y las operaciones de seguridad del servicio Cloud de producción, consulta **Seguridad de red y datos de Tier0 Cloud**. Este documento solo se aplica a Tier0 Cloud.

Seguridad

Identidad y control de acceso

Área	Mecanismo de Tier0	Detalles
Acceso a la plataforma	• Cloud controla el acceso mediante la pertenencia a un Workspace. • Enterprise controla el acceso mediante cuentas de usuario.	Gestión de usuarios
Acceso de agentes	Los agentes se autentican con una clave de API y utilizan Tier0 CLI para acceder a la plataforma y operarla.	Autenticación de CLI
Acceso a proyectos y aplicaciones	Tier0 utiliza RBAC para controlar el acceso a las aplicaciones.	Control de acceso

> **Nota**
> La pertenencia a un Workspace no crea automáticamente una cuenta. Cada usuario debe registrar su cuenta e iniciar sesión. El registro de la cuenta requiere una verificación en dos pasos.

Control de acceso a los datos

Acceso	Mecanismo de Tier0	Detalles
MQTT	• Los clientes se autentican con credenciales generadas por Tier0. • TLS y WebSocket seguro cifran los datos en tránsito.	Credenciales y conexiones MQTT

Acceso	Mecanismo de Tier0	Detalles
REST API	El acceso a los datos requiere una clave generada por Tier0.	Referencia de API

Resiliencia

Disponibilidad del despliegue

Enterprise ofrece actualmente dos planes de despliegue para los nodos centrales:

Plan	Límite de disponibilidad	Detalles
Despliegue estándar	• Permite la conmutación por error entre las bases de datos primaria y en espera. • La aplicación Enterprise y EMQX siguen siendo puntos únicos de fallo.	Plan de despliegue estándar
Despliegue de alta disponibilidad	• Añade nodos Enterprise activo y en espera y define la conmutación por error de la base de datos, la caché, el almacenamiento de objetos y los archivos gestionados del entorno de ejecución. • No garantiza cero interrupciones ni	Plan de despliegue de alta disponibilidad

Plan	Límite de disponibilidad	Detalles
	cero pérdida de datos.	

> **Nota**
> Las limitaciones detalladas de disponibilidad se describen en **SLA y límites de alta disponibilidad**.

Copias de seguridad y recuperación

Enterprise permite realizar copias de seguridad y recuperar bases de datos. Después de la recuperación, comprueba el estado del despliegue, verifica las funciones básicas y confirma que los usuarios pueden iniciar sesión en la plataforma.

- Para las copias de seguridad, consulta **Runbook de operaciones**.
- Para los requisitos de verificación de la recuperación, consulta los requisitos de simulacros en **SLA y límites de alta disponibilidad**.

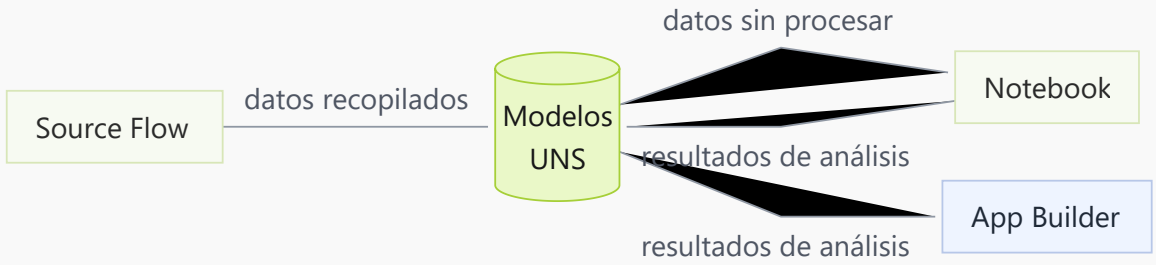
> **Tiempo de respuesta**
> El tiempo de respuesta habitual es de 3 días laborables.

BUENAS PRÁCTICAS

Crear apps de analítica

Crear una app de analítica con Notebook usando el caso Aramco Bowtie (datos anonimizados).

Tier0 usa Marimo Notebook para análisis avanzado de datos con Python. Esta guía usa una aplicación Bowtie como ejemplo.



Contexto del ejemplo

La corrosión en refinería es un proceso complejo influido por múltiples factores operativos. Este ejemplo muestra una evaluación de riesgo de corrosión en tiempo real que combina datos de proceso para estimar la probabilidad de corrosión y apoyar el mantenimiento proactivo.

Obtener datos raw

1 En Tier0, ve a **UNS** e importa el modelo de datos.

Mostrar JSON del modelo UNS

```
{
  "notes":
"type:PATH|TOPIC,topicType:STATE|ACTION|METRIC,schema.type:INTEGER|STRING
|FLOAT|DOUBLE|BOOLEAN|LONG|DATETIME",
  "namespace": [
    {
      "name": "Refinery",
      "type": "PATH",
      "children": [
        {
```

```
          "name": "CDU_Plant",
          "type": "PATH",
          "children": [
            {
              "name": "State",
              "type": "PATH",
              "topicType": "STATE",
              "children": [
                {
                  "name": "Corrosion_Risk",
                  "type": "TOPIC",
                  "topicType": "STATE",
                  "description": "Current corrosion risk state inferred
by Bayesian Network",
                  "enableHistory": "TRUE",
                  "schema": [
                    {
                      "name": "risk_state",
                      "type": "STRING"
                    },
                    {
                      "name": "previous_state",
                      "type": "STRING"
                    },
                    {
                      "name": "model_confidence",
                      "type": "FLOAT"
                    },
                    {
                      "name": "timestamp",
                      "type": "DATETIME"
                    }
                  ]
                }
              ]
            },
            {
              "name": "Metric",
              "type": "PATH",
              "topicType": "METRIC",
              "children": [
                {
                  "name": "Corrosion_Monitoring",
                  "type": "TOPIC",
                  "topicType": "METRIC",
                  "description": "Real-time process measurements for CDU
atmospheric overhead corrosion monitoring",
                  "enableHistory": "TRUE",
                  "mockData": "FALSE",
                  "schema": [
                    {
```

```
        "name": "d103_chloride",
        "type": "FLOAT"
      },
      {
        "name": "d103_ph",
        "type": "FLOAT"
      },
      {
        "name": "wash_water_flow",
        "type": "FLOAT"
      },
      {
        "name": "desalter_salt_ptb",
        "type": "FLOAT"
      },
      {
        "name": "desalter_bsw",
        "type": "FLOAT"
      },
      {
        "name": "wash_water_rate",
        "type": "FLOAT"
      },
      {
        "name": "rrd_ph",
        "type": "FLOAT"
      },
      {
        "name": "rrd_chloride",
        "type": "FLOAT"
      },
      {
        "name": "rrd_total_iron",
        "type": "FLOAT"
      },
      {
        "name": "sap_evidence",
        "type": "STRING"
      },
      {
        "name": "injection_quill_fail",
        "type": "BOOLEAN"
      },
      {
        "name": "timestamp",
        "type": "DATETIME"
      }
    ]
  },
  {
    "name": "Corrosion_Analysis",
```

```
                    "type": "TOPIC",
                    "topicType": "METRIC",
                    "description": "Posterior probabilities generated by
the Bayesian Network",
                    "enableHistory": "TRUE",
                    "mockData": "FALSE",
                    "schema": [
                      {
                        "name": "p_normal",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "p_developing",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "p_confirmed",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "total_corrosion_risk",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "lopc_probability",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "shutdown_probability",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "escalation_probability",
                        "type": "DOUBLE"
                      },
                      {
                        "name": "timestamp",
                        "type": "DATETIME"
                      }
                    ]
                  }
                ]
              }
            ]
          }
        ]
      }
    ]
}
```

2 Ve a **Flows**, crea un **Source Flow** para conectar los datos sin procesar y publicarlos en **UNS**.

Mostrar JSON de Source Flow


```
[
    {
        "id": "6173469ed5064ccf",
        "type": "tab",
        "label": "corrosion",
        "disabled": false,
        "info": ""
    },
    {
        "id": "ad20b016baf7247f",
        "type": "OpcUa-Client",
        "z": "6173469ed5064ccf",
        "endpoint": "66bd118c33eb12f8",
        "action": "subscribe",
        "deadbandtype": "a",
        "deadbandvalue": 1,
        "time": 10,
        "timeUnit": "s",
        "certificate": "n",
        "localfile": "",
        "localkeyfile": "",
        "securitymode": "None",
        "securitypolicy": "None",
        "useTransport": false,
        "maxChunkCount": 1,
        "maxMessageSize": 8192,
        "receiveBufferSize": 8192,
        "sendBufferSize": 8192,
        "setstatusandtime": false,
        "keepsessionalive": false,
        "name": "",
        "x": 280,
        "y": 440,
        "wires": [
            [
                "ac9195bb4863dbdd",
                "5a3d61b2ae40dcb6"
            ],
            [],
            []
        ]
    },
    {
```

```
            "id": "b6746ed066ba0e1a",
            "type": "inject",
            "z": "6173469ed5064ccf",
            "name": "",
            "props": [
                {
                    "p": "payload"
                },
                {
                    "p": "topic",
                    "vt": "str"
                }
            ],
            "repeat": "",
            "crontab": "",
            "once": false,
            "onceDelay": 0.1,
            "topic": "multiple",
            "payload": "[   {     \"name\": \"D103_CHLORIDE\",
\"nodeId\": \"ns=2;i=3\",     \"datatype\": \"Float\"   },   {
\"name\": \"D103_PH\",     \"nodeId\": \"ns=2;i=4\",     \"datatype\":
\"Float\"   },   {     \"name\": \"WASH_WATER_FLOW\",     \"nodeId\":
\"ns=2;i=5\",     \"datatype\": \"Float\"   },   {     \"name\":
\"DESALTER_SALT_PTB\",     \"nodeId\": \"ns=2;i=6\",     \"datatype\":
\"Float\"   },   {     \"name\": \"DESALTER_BSW\",     \"nodeId\":
\"ns=2;i=7\",     \"datatype\": \"Float\"   },   {     \"name\":
\"WASH_WATER_RATE\",     \"nodeId\": \"ns=2;i=8\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_PH\",     \"nodeId\":
\"ns=2;i=9\",     \"datatype\": \"Float\"   },   {     \"name\":
\"RRD_CHLORIDE\",     \"nodeId\": \"ns=2;i=10\",     \"datatype\":
\"Float\"   },   {     \"name\": \"RRD_TOTAL_IRON\",     \"nodeId\":
\"ns=2;i=11\",     \"datatype\": \"Float\"   },   {     \"name\":
\"SAP_EVIDENCE\",     \"nodeId\": \"ns=2;i=12\",     \"datatype\":
\"UInt16\"   },   {     \"name\": \"INJECTION_QUILL_FAIL\",
\"nodeId\": \"ns=2;i=13\",     \"datatype\": \"Boolean\"   } ]",
            "payloadType": "json",
            "x": 90,
            "y": 440,
            "wires": [
                [
                    "ad20b016baf7247f"
                ]
            ]
        },
        {
            "id": "ac9195bb4863dbdd",
            "type": "debug",
            "z": "6173469ed5064ccf",
            "name": "debug 2",
            "active": true,
            "tosidebar": true,
```

```
            "console": false,
            "tostatus": false,
            "complete": "false",
            "statusVal": "",
            "statusType": "auto",
            "x": 490,
            "y": 480,
            "wires": []
        },
        {
            "id": "5a3d61b2ae40dcb6",
            "type": "function",
            "z": "6173469ed5064ccf",
            "name": "function 1",
            "func": "const TOPIC =
\"Refinery/CDU_Plant/Metric/Corrosion_Monitoring\";\n\nconst nodeMap =
{\n    \"ns=2;i=3\": { field: \"d103_chloride\", type: \"float\" },\n
\"ns=2;i=4\": { field: \"d103_ph\", type: \"float\" },\n    \"ns=2;i=5\":
{ field: \"wash_water_flow\", type: \"float\" },\n    \"ns=2;i=6\": {
field: \"desalter_salt_ptb\", type: \"float\" },\n    \"ns=2;i=7\": {
field: \"desalter_bsw\", type: \"float\" },\n    \"ns=2;i=8\": { field:
\"wash_water_rate\", type: \"float\" },\n    \"ns=2;i=9\": { field:
\"rrd_ph\", type: \"float\" },\n    \"ns=2;i=10\": { field:
\"rrd_chloride\", type: \"float\" },\n    \"ns=2;i=11\": { field:
\"rrd_total_iron\", type: \"float\" },\n    \"ns=2;i=12\": { field:
\"sap_evidence\", type: \"sap\" },\n    \"ns=2;i=13\": { field:
\"injection_quill_fail\", type: \"boolean\" }\n};\n\nconst sapMap = {\n
0: \"none\",\n    1: \"work_order\",\n    2:
\"inspection_finding\"\n};\n\nfunction normalizeNodeId(v) {\n    if (!v)
return \"\";\n    if (typeof v === \"string\") return v;\n    if
(v.toString) return v.toString();\n    return \"\";\n}\n\nfunction
extractValue(payload) {\n    // OPC UA DataValue: payload.value.value\n
if (payload && payload.value && payload.value.value !== undefined) {\n
return payload.value.value;\n    }\n\n    // Some OPC UA nodes output
Variant directly.\n    if (payload && payload.value !== undefined) {\n
return payload.value;\n    }\n\n    return payload;\n}\n\nfunction
extractNodeId(msg) {\n    return normalizeNodeId(\n        msg.nodeId
||\n        msg.topic ||\n        msg.payload?.nodeId ||\n
msg.payload?.nodeid\n    );\n}\n\nconst nodeId =
extractNodeId(msg);\nconst spec = nodeMap[nodeId];\n\nif (!spec) {\n
node.warn(`Unknown OPC UA nodeId: ${nodeId}`);\n    return
null;\n}\n\nlet value = extractValue(msg.payload);\n\nif (spec.type ===
\"float\") {\n    value = Number(value);\n} else if (spec.type ===
\"boolean\") {\n    value = Boolean(value);\n} else if (spec.type ===
\"sap\") {\n    value = sapMap[Number(value)] || \"unknown\";\n}\n\nlet
latest = context.get(\"latest\") || {};\nlatest[spec.field] =
value;\nlatest.timestamp = new
Date().toISOString();\ncontext.set(\"latest\", latest);\n\n// Publish
only after all 11 fields have been received at least once.\nconst
requiredFields = Object.values(nodeMap).map(x => x.field);\nconst ready =
requiredFields.every(field => latest[field] !== undefined);\n\nif
```

```
(!ready) {\n    node.status({\n        fill: \"yellow\",\n        shape:
\"ring\",\n        text: `waiting ${Object.keys(latest).length - 1}/11`\n
});\n    return null;\n}\n\nmsg.topic = TOPIC;\nmsg.payload = {\n
d103_chloride: latest.d103_chloride,\n    d103_ph: latest.d103_ph,\n
wash_water_flow: latest.wash_water_flow,\n    desalter_salt_ptb:
latest.desalter_salt_ptb,\n    desalter_bsw: latest.desalter_bsw,\n
wash_water_rate: latest.wash_water_rate,\n    rrd_ph: latest.rrd_ph,\n
rrd_chloride: latest.rrd_chloride,\n    rrd_total_iron:
latest.rrd_total_iron,\n    sap_evidence: latest.sap_evidence,\n
injection_quill_fail: latest.injection_quill_fail,\n    timestamp:
latest.timestamp\n};\n\nnode.status({\n    fill: \"green\",\n    shape:
\"dot\",\n    text: \"published corrosion monitoring\"\n});\n\nreturn
msg;",
          "outputs": 1,
          "timeout": 0,
          "noerr": 0,
          "initialize": "",
          "finalize": "",
          "libs": [],
          "x": 370,
          "y": 580,
          "wires": [
              [
                  "aa15be33a84fb812",
                  "4fdd68dda3ee5c9e"
              ]
          ]
      },
      {
          "id": "aa15be33a84fb812",
          "type": "mqtt out",
          "z": "6173469ed5064ccf",
          "name": "",
          "topic": "",
          "qos": "",
          "retain": "",
          "respTopic": "",
          "contentType": "",
          "userProps": "",
          "correl": "",
          "expiry": "",
          "broker": "broker-djx75cwgiywq",
          "x": 550,
          "y": 580,
          "wires": []
      },
      {
          "id": "4fdd68dda3ee5c9e",
          "type": "debug",
          "z": "6173469ed5064ccf",
          "name": "debug 3",
```

```
        "active": true,
        "tosidebar": true,
        "console": false,
        "tostatus": false,
        "complete": "false",
        "statusVal": "",
        "statusType": "auto",
        "x": 450,
        "y": 680,
        "wires": []
    },
    {
        "id": "66bd118c33eb12f8",
        "type": "OpcUa-Endpoint",
        "endpoint": "opc.tcp://172.31.151.237:4850",
        "secpol": "None",
        "secmode": "None",
        "none": true,
        "login": false,
        "usercert": false,
        "usercertificate": "",
        "userprivatekey": ""
    },
    {
        "id": "broker-djx75cwgiywq",
        "type": "mqtt-broker",
        "z": "6173469ed5064ccf",
        "name": "corrosion",
        "broker": "emqx",
        "port": "1883",
        "clientid": "357356352615696",
        "usetls": false,
        "protocolVersion": "4",
        "keepalive": "60",
        "cleansession": true,
        "birthTopic": "",
        "birthQos": "0",
        "birthPayload": "",
        "closeTopic": "",
        "closeQos": "0",
        "closePayload": "",
        "willTopic": "",
        "willQos": "0",
        "willPayload": ""
    },
    {
        "id": "c5201f1ad482b182",
        "type": "global-config",
        "env": [],
        "modules": {
            "node-red-contrib-opcua": "0.2.339"
```

```
            }
        }
    ]
```

Crear una app de analítica en Notebook

1. En Tier0, ve a **Notebook** y crea un notebook nuevo.
2. Abre el notebook y agrega las siguientes celdas para analizar datos de **UNS**.
    - Celda 1 - Importa las dependencias necesarias y configura el broker MQTT, los topics y los ajustes de ejecución.

Mostrar código

```
@app.cell
def _():
    """Imports and runtime configuration."""
    import json
    import math
    import os
    import threading
    import time
    from datetime import datetime, timezone

    import marimo as mo
    import pandas as pd
    import paho.mqtt.client as mqtt

    MQTT_BROKER_HOST = os.getenv("TIER0_MQTT_HOST", "enterprise-new-
dev.tier0.dev")
    MQTT_BROKER_PORT = int(os.getenv("TIER0_MQTT_PORT", "1883"))
    MQTT_USERNAME = os.getenv("TIER0_MQTT_USERNAME",
"357890055321856")
    MQTT_PASSWORD = os.getenv("TIER0_MQTT_PASSWORD",
"caa6b34242002a3dff4948dd73ce8a")
    MQTT_CLIENT_ID = os.getenv(
        "TIER0_MQTT_CLIENT_ID",
        "357890055321856_bn",
    )

    SOURCE_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Monitoring"
    ANALYSIS_TOPIC = "Refinery/CDU_Plant/Metric/Corrosion_Analysis"
    RISK_TOPIC = "Refinery/CDU_Plant/State/Corrosion_Risk"
```

```
    SOURCE_QOS = int(os.getenv("TIER0_SOURCE_QOS", "1"))
    OUTPUT_QOS = int(os.getenv("TIER0_OUTPUT_QOS", "1"))
    AUTOSTART = os.getenv("TIER0_BN_AUTOSTART", "true").lower() in {
        "1", "true", "yes", "on"
    }
    ROOT_CONFIDENCE = float(os.getenv("TIER0_BN_ROOT_CONFIDENCE",
"0.85"))

    def utc_now_iso() -> str:
        return (
            datetime.now(timezone.utc)
            .isoformat(timespec="milliseconds")
            .replace("+00:00", "Z")
        )

    mo.md(
        """
        # CDU Corrosion Bayesian Analysis

        This notebook subscribes to a complete corrosion-monitoring
snapshot,
        performs one Bayesian inference for each new source message,
and publishes
        the analysis results back to the UNS.
        """
    )
    return (
        ANALYSIS_TOPIC,
        AUTOSTART,
        MQTT_BROKER_HOST,
        MQTT_BROKER_PORT,
        MQTT_CLIENT_ID,
        MQTT_PASSWORD,
        MQTT_USERNAME,
        OUTPUT_QOS,
        RISK_TOPIC,
        ROOT_CONFIDENCE,
        SOURCE_QOS,
        SOURCE_TOPIC,
        datetime,
        json,
        math,
        mo,
        mqtt,
        os,
        pd,
        threading,
        time,
        timezone,
        utc_now_iso,
```

```
    )
```

- Celda 2 - Define la topology de Bayesian Network, los espacios de estado, los generadores CPT y las reglas de probabilidad.

Mostrar código

```
@app.cell
def _(math, pd):
    """Static Bayesian-network definition and CPT generators."""
    import itertools

    from pgmpy.factors.discrete import TabularCPD
    from pgmpy.inference import VariableElimination
    from pgmpy.models import DiscreteBayesianNetwork

    STATE_SPACES = {
        "STATE_3": ["LOW", "NORMAL", "HIGH"],
        "STATE_YN": ["NO", "YES"],
        "PB2_3STATE": ["FAIL", "IL_OK", "SL_OK"],
        "TOP_EVENT_STATE": ["NORMAL", "DEVELOPING", "CONFIRMED"],
    }

    NODE_STATE_SPACE = {
        "T1_HighChlorideLoad": "STATE_3",
        "T2_ChemProtectionFail": "STATE_3",
        "T3_WashWaterLow": "STATE_3",
        "PB1_Desalter_OK": "STATE_YN",
        "PB2_WashQty_State": "PB2_3STATE",
        "PB3_Neutralization_OK": "STATE_YN",
        "MB1_ESD_OK": "STATE_YN",
        "MB2_TSV_OK": "STATE_YN",
        "TE_CorrosionState": "TOP_EVENT_STATE",
        "EV_TOTIRON_High": "STATE_YN",
        "EV_SAP_Positive": "STATE_YN",
        "EV_QuillFailure": "STATE_YN",
        "C1_LOPC": "STATE_YN",
        "C2_UnplannedShutdown": "STATE_YN",
        "C3_Escalation": "STATE_YN",
    }

    EDGES = [
        ("T1_HighChlorideLoad", "TE_CorrosionState"),
        ("T2_ChemProtectionFail", "TE_CorrosionState"),
        ("T3_WashWaterLow", "TE_CorrosionState"),
        ("PB1_Desalter_OK", "TE_CorrosionState"),
        ("PB2_WashQty_State", "TE_CorrosionState"),
```

```
        ("PB3_Neutralization_OK", "TE_CorrosionState"),
        ("TE_CorrosionState", "EV_TOTIRON_High"),
        ("TE_CorrosionState", "EV_SAP_Positive"),
        ("TE_CorrosionState", "EV_QuillFailure"),
        ("TE_CorrosionState", "C1_LOPC"),
        ("MB1_ESD_OK", "C1_LOPC"),
        ("MB2_TSV_OK", "C1_LOPC"),
        ("TE_CorrosionState", "C2_UnplannedShutdown"),
        ("MB1_ESD_OK", "C2_UnplannedShutdown"),
        ("C1_LOPC", "C3_Escalation"),
        ("MB1_ESD_OK", "C3_Escalation"),
        ("MB2_TSV_OK", "C3_Escalation"),
    ]

    def states_for(node_id: str) -> list[str]:
        return STATE_SPACES[NODE_STATE_SPACE[node_id]]

    def softmax(values: list[float]) -> list[float]:
        maximum = max(values)
        exps = [math.exp(value - maximum) for value in values]
        total = sum(exps)
        return [value / total for value in exps]

    THREAT_SCORE = {"LOW": 0.0, "NORMAL": 1.0, "HIGH": 2.0}
    PB1_OK_BONUS = {"NO": 0.8, "YES": -0.6}
    PB3_OK_BONUS = {"NO": 1.0, "YES": -0.7}
    PB2_STATE_BONUS = {"FAIL": 1.2, "IL_OK": 0.3, "SL_OK": -0.6}

    def top_event_distribution(t1, t2, t3, pb1, pb2, pb3) ->
dict[str, float]:
        score = (
            THREAT_SCORE[t1]
            + THREAT_SCORE[t2]
            + THREAT_SCORE[t3]
            + PB1_OK_BONUS[pb1]
            + PB2_STATE_BONUS[pb2]
            + PB3_OK_BONUS[pb3]
        )
        probabilities = softmax(
            [
                2.2 - score,
                -0.2 + 0.6 * score,
                -1.2 + 0.8 * score,
            ]
        )
        return dict(zip(STATE_SPACES["TOP_EVENT_STATE"],
probabilities))

    TE_PARENTS = [
        "T1_HighChlorideLoad",
        "T2_ChemProtectionFail",
```

```
        "T3_WashWaterLow",
        "PB1_Desalter_OK",
        "PB2_WashQty_State",
        "PB3_Neutralization_OK",
    ]

    TE_PARENT_SPACES = [states_for(parent) for parent in TE_PARENTS]
    te_rows = []
    for combination in itertools.product(*TE_PARENT_SPACES):
        distribution = top_event_distribution(*combination)
        te_rows.append(
            [
                *combination,
                distribution["NORMAL"],
                distribution["DEVELOPING"],
                distribution["CONFIRMED"],
            ]
        )
    TE_CPT_DF = pd.DataFrame(
        te_rows,
        columns=TE_PARENTS
        + ["P_NORMAL", "P_DEVELOPING", "P_CONFIRMED"],
    )

    def evidence_yes_probability(te_state: str, kind: str) -> float:
        mappings = {
            "TOTIRON": {"NORMAL": 0.15, "DEVELOPING": 0.65,
"CONFIRMED": 0.85},
            "SAP": {"NORMAL": 0.08, "DEVELOPING": 0.35, "CONFIRMED":
0.75},
            "QUILL": {"NORMAL": 0.03, "DEVELOPING": 0.15,
"CONFIRMED": 0.55},
        }
        return mappings[kind][te_state]

    def lopc_yes_probability(te: str, mb1: str, mb2: str) -> float:
        base = {"NORMAL": 0.02, "DEVELOPING": 0.08, "CONFIRMED":
0.25}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0) + (0.18
if mb2 == "NO" else 0))

    def shutdown_yes_probability(te: str, mb1: str) -> float:
        base = {"NORMAL": 0.05, "DEVELOPING": 0.35, "CONFIRMED":
0.60}[te]
        return min(0.98, base + (0.10 if mb1 == "NO" else 0))

    def escalation_yes_probability(lopc: str, mb1: str, mb2: str) ->
float:
        if lopc == "NO":
            return 0.01
```

```
        return min(0.98, 0.10 + (0.18 if mb1 == "NO" else 0) + (0.35
if mb2 == "NO" else 0))

    def peaked_prior(node_id: str, center_state: str, confidence:
float) -> list[float]:
        states = states_for(node_id)
        confidence = min(max(float(confidence), 0.0), 1.0)
        if center_state not in states:
            return [1.0 / len(states)] * len(states)
        if len(states) == 1:
            return [1.0]
        remainder = (1.0 - confidence) / (len(states) - 1)
        probabilities = [remainder] * len(states)
        probabilities[states.index(center_state)] = confidence
        return probabilities

    def binary_cpd(child: str, parent: str, yes_probability_fn):
        parent_states = states_for(parent)
        yes_values = [float(yes_probability_fn(state)) for state in
parent_states]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=[parent],
            evidence_card=[len(parent_states)],
            state_names={child: states_for(child), parent:
parent_states},
        )

    def binary_multi_cpd(child: str, parents: list[str],
yes_probability_fn):
        parent_spaces = [states_for(parent) for parent in parents]
        combinations = list(itertools.product(*parent_spaces))
        yes_values = [float(yes_probability_fn(*combination)) for
combination in combinations]
        return TabularCPD(
            variable=child,
            variable_card=2,
            values=[[1.0 - value for value in yes_values],
yes_values],
            evidence=parents,
            evidence_card=[len(space) for space in parent_spaces],
            state_names={
                child: states_for(child),
                **{parent: states_for(parent) for parent in
parents},
            },
        )
```

```
    return (
        DiscreteBayesianNetwork,
        EDGES,
        NODE_STATE_SPACE,
        STATE_SPACES,
        TE_CPT_DF,
        TE_PARENTS,
        TabularCPD,
        VariableElimination,
        binary_cpd,
        binary_multi_cpd,
        escalation_yes_probability,
        evidence_yes_probability,
        itertools,
        lopc_yes_probability,
        peaked_prior,
        shutdown_yes_probability,
        states_for,
    )
```

- Celda 3 - Valida el payload MQTT entrante y convierte los valores continuos del proceso en estados de Bayesian Network.

Mostrar código

```
@app.cell
def _(ROOT_CONFIDENCE, utc_now_iso):
    """Source validation, discretization, and one-snapshot
analysis."""
    import hashlib

    REQUIRED_SOURCE_FIELDS = [
        "d103_chloride",
        "d103_ph",
        "wash_water_flow",
        "desalter_salt_ptb",
        "desalter_bsw",
        "wash_water_rate",
        "rrd_ph",
        "rrd_chloride",
        "rrd_total_iron",
        "sap_evidence",
        "injection_quill_fail",
        "timestamp",
    ]

    def parse_boolean(value) -> bool:
```

```
        if isinstance(value, bool):
            return value
        if isinstance(value, (int, float)):
            return bool(value)
        if isinstance(value, str):
            normalized = value.strip().lower()
            if normalized in {"true", "1", "yes", "y", "on"}:
                return True
            if normalized in {"false", "0", "no", "n", "off"}:
                return False
        raise ValueError(f"invalid boolean value: {value!r}")

    def normalize_sap(value) -> str:
        normalized = str(value).strip().lower()
        if normalized in {"0", "none", "normal", "", "nan"}:
            return "None"
        if normalized in {"1", "work_order", "work order", "minor",
"warning"}:
            return "Minor corrosion observed"
        if normalized in {
            "2",
            "inspection_finding",
            "inspection finding",
            "confirmed",
            "abnormal",
        }:
            return "Confirmed corrosion / thickness loss"
        raise ValueError(f"unsupported sap_evidence value:
{value!r}")

    def normalize_source_payload(payload: dict) -> dict:
        if not isinstance(payload, dict):
            raise TypeError("source MQTT payload must be a JSON
object")

        lower = {str(key).lower(): value for key, value in
payload.items()}
        missing = [field for field in REQUIRED_SOURCE_FIELDS if
field not in lower]
        if missing:
            raise ValueError(f"source payload is missing fields:
{missing}")

        normalized = {
            "d103_chloride": float(lower["d103_chloride"]),
            "d103_ph": float(lower["d103_ph"]),
            "wash_water_flow": float(lower["wash_water_flow"]),
            "desalter_salt_ptb": float(lower["desalter_salt_ptb"]),
            "desalter_bsw": float(lower["desalter_bsw"]),
            "wash_water_rate": float(lower["wash_water_rate"]),
            "rrd_ph": float(lower["rrd_ph"]),
```

```
            "rrd_chloride": float(lower["rrd_chloride"]),
            "rrd_total_iron": float(lower["rrd_total_iron"]),
            "sap_evidence": normalize_sap(lower["sap_evidence"]),
            "injection_quill_fail":
parse_boolean(lower["injection_quill_fail"]),
            "timestamp": str(lower["timestamp"]).strip(),
        }

        numeric_ranges = {
            "d103_chloride": (0.0, 1000.0),
            "d103_ph": (0.0, 14.0),
            "wash_water_flow": (0.0, 10000.0),
            "desalter_salt_ptb": (0.0, 1000.0),
            "desalter_bsw": (0.0, 100.0),
            "wash_water_rate": (0.0, 100.0),
            "rrd_ph": (0.0, 14.0),
            "rrd_chloride": (0.0, 10000.0),
            "rrd_total_iron": (0.0, 10000.0),
        }
        for field, (minimum, maximum) in numeric_ranges.items():
            value = normalized[field]
            if not minimum <= value <= maximum:
                raise ValueError(f"{field} out of accepted range:
{value}")

        if not normalized["timestamp"]:
            raise ValueError("timestamp cannot be empty")
        return normalized

    def source_message_id(payload: dict) -> str:
        # Timestamp is the primary event key. Hash protects against
two different
        # snapshots accidentally sharing the same timestamp.
        ordered = "|".join(str(payload[field]) for field in
REQUIRED_SOURCE_FIELDS)
        return hashlib.sha256(ordered.encode("utf-8")).hexdigest()

    def discretize_source(payload: dict) -> tuple[dict, dict]:
        t1 = (
            "LOW"
            if payload["d103_chloride"] < 5.0
            else "NORMAL"
            if payload["d103_chloride"] <= 20.0
            else "HIGH"
        )

        chemical_failure_score = int(payload["d103_ph"] < 5.5) +
int(
            payload["d103_chloride"] > 10.0
        )
        t2 = (
```

```
            "HIGH"
            if chemical_failure_score == 2
            else "NORMAL"
            if chemical_failure_score == 1
            else "LOW"
        )

        t3 = (
            "HIGH"
            if payload["wash_water_flow"] < 6.0
            else "NORMAL"
            if payload["wash_water_flow"] <= 10.0
            else "LOW"
        )

        pb1 = (
            "YES"
            if payload["desalter_salt_ptb"] <= 1.0
            and payload["desalter_bsw"] <= 0.2
            else "NO"
        )

        pb2 = (
            "FAIL"
            if payload["wash_water_rate"] < 4.0
            else "IL_OK"
            if payload["wash_water_rate"] < 6.0
            else "SL_OK"
        )

        pb3 = (
            "YES"
            if payload["rrd_ph"] >= 5.5 and payload["rrd_chloride"]
<= 10.0
            else "NO"
        )

        root_centers = {
            "T1_HighChlorideLoad": t1,
            "T2_ChemProtectionFail": t2,
            "T3_WashWaterLow": t3,
            "PB1_Desalter_OK": pb1,
            "PB2_WashQty_State": pb2,
            "PB3_Neutralization_OK": pb3,
            # The source model currently has no MB fields. Keep both
mitigations
            # available by default, matching the previous notebook
UI defaults.
            "MB1_ESD_OK": "YES",
            "MB2_TSV_OK": "YES",
        }
```

```
        evidence = {
            "EV_TOTIRON_High": "YES" if payload["rrd_total_iron"] >=
10.0 else "NO",
            "EV_SAP_Positive": "YES" if payload["sap_evidence"] !=
"None" else "NO",
            "EV_QuillFailure": "YES" if
payload["injection_quill_fail"] else "NO",
        }
        return root_centers, evidence

    return (
        REQUIRED_SOURCE_FIELDS,
        discretize_source,
        normalize_source_payload,
        parse_boolean,
        source_message_id,
    )
```

- Celda 4 - Construye una Bayesian Network para el snapshot actual, ejecuta inferencia y genera payloads de salida.

Mostrar código

```
@app.cell
def _(
    DiscreteBayesianNetwork,
    EDGES,
    ROOT_CONFIDENCE,
    STATE_SPACES,
    TE_CPT_DF,
    TE_PARENTS,
    TabularCPD,
    VariableElimination,
    binary_cpd,
    binary_multi_cpd,
    discretize_source,
    escalation_yes_probability,
    evidence_yes_probability,
    itertools,
    lopc_yes_probability,
    peaked_prior,
    shutdown_yes_probability,
    states_for,
    utc_now_iso,
):
```

```
    """Build a fresh BN for the current snapshot and execute
inference."""

    def factor_probability(factor, variable: str, state: str) ->
float:
        states = factor.state_names[variable]
        values = list(map(float, factor.values))
        return float(values[states.index(state)])

    def build_model(root_centers: dict):
        model = DiscreteBayesianNetwork(EDGES)

        root_cpds = []
        for node_id, center_state in root_centers.items():
            states = states_for(node_id)
            probabilities = peaked_prior(node_id, center_state,
ROOT_CONFIDENCE)
            root_cpds.append(
                TabularCPD(
                    variable=node_id,
                    variable_card=len(states),
                    values=[[probability] for probability in
probabilities],
                    state_names={node_id: states},
                )
            )

        te_values = [
            TE_CPT_DF["P_NORMAL"].astype(float).tolist(),
            TE_CPT_DF["P_DEVELOPING"].astype(float).tolist(),
            TE_CPT_DF["P_CONFIRMED"].astype(float).tolist(),
        ]
        te_cpd = TabularCPD(
            variable="TE_CorrosionState",
            variable_card=3,
            values=te_values,
            evidence=TE_PARENTS,
            evidence_card=[len(states_for(parent)) for parent in
TE_PARENTS],
            state_names={
                "TE_CorrosionState":
STATE_SPACES["TOP_EVENT_STATE"],
                **{parent: states_for(parent) for parent in
TE_PARENTS},
            },
        )

        evidence_cpds = [
            binary_cpd(
                "EV_TOTIRON_High",
                "TE_CorrosionState",
```

```
                lambda state: evidence_yes_probability(state,
"TOTIRON"),
            ),
            binary_cpd(
                "EV_SAP_Positive",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"SAP"),
            ),
            binary_cpd(
                "EV_QuillFailure",
                "TE_CorrosionState",
                lambda state: evidence_yes_probability(state,
"QUILL"),
            ),
        ]

        consequence_cpds = [
            binary_multi_cpd(
                "C1_LOPC",
                ["TE_CorrosionState", "MB1_ESD_OK", "MB2_TSV_OK"],
                lopc_yes_probability,
            ),
            binary_multi_cpd(
                "C2_UnplannedShutdown",
                ["TE_CorrosionState", "MB1_ESD_OK"],
                shutdown_yes_probability,
            ),
            binary_multi_cpd(
                "C3_Escalation",
                ["C1_LOPC", "MB1_ESD_OK", "MB2_TSV_OK"],
                escalation_yes_probability,
            ),
        ]

        model.add_cpds(*root_cpds, te_cpd, *evidence_cpds,
*consequence_cpds)
        if not model.check_model():
            raise RuntimeError("Bayesian network model validation
failed")
        return model

    def analyze_snapshot(payload: dict, previous_risk_state: str |
None = None) -> dict:
        root_centers, evidence = discretize_source(payload)
        model = build_model(root_centers)
        inference = VariableElimination(model)

        q_te = inference.query(["TE_CorrosionState"],
evidence=evidence, show_progress=False)
```

```
        q_c1 = inference.query(["C1_LOPC"], evidence=evidence,
show_progress=False)
        q_c2 = inference.query(["C2_UnplannedShutdown"],
evidence=evidence, show_progress=False)
        q_c3 = inference.query(["C3_Escalation"], evidence=evidence,
show_progress=False)

        p_normal = factor_probability(q_te, "TE_CorrosionState",
"NORMAL")
        p_developing = factor_probability(q_te, "TE_CorrosionState",
"DEVELOPING")
        p_confirmed = factor_probability(q_te, "TE_CorrosionState",
"CONFIRMED")
        probabilities = {
            "NORMAL": p_normal,
            "DEVELOPING": p_developing,
            "CONFIRMED": p_confirmed,
        }
        risk_state = max(probabilities, key=probabilities.get)
        analysis_timestamp = utc_now_iso()

        analysis_payload = {
            "p_normal": p_normal,
            "p_developing": p_developing,
            "p_confirmed": p_confirmed,
            "total_corrosion_risk": p_developing + p_confirmed,
            "lopc_probability": factor_probability(q_c1, "C1_LOPC",
"YES"),
            "shutdown_probability": factor_probability(
                q_c2, "C2_UnplannedShutdown", "YES"
            ),
            "escalation_probability": factor_probability(q_c3,
"C3_Escalation", "YES"),
            "timestamp": analysis_timestamp,
        }

        risk_payload = {
            "risk_state": risk_state,
            "previous_state": previous_risk_state or risk_state,
            "model_confidence": max(probabilities.values()),
            "timestamp": analysis_timestamp,
        }

        return {
            "source_timestamp": payload["timestamp"],
            "root_states": root_centers,
            "evidence": evidence,
            "analysis_payload": analysis_payload,
            "risk_payload": risk_payload,
            "risk_state": risk_state,
        }
```

```
    return analyze_snapshot, build_model, factor_probability
```

- Celda 5 - Se suscribe al source topic, procesa cada snapshot entrante, ejecuta el análisis Bayesian y publica los resultados.

Mostrar código

```
@app.cell
def _(
    ANALYSIS_TOPIC,
    AUTOSTART,
    MQTT_BROKER_HOST,
    MQTT_BROKER_PORT,
    MQTT_CLIENT_ID,
    MQTT_PASSWORD,
    MQTT_USERNAME,
    OUTPUT_QOS,
    RISK_TOPIC,
    SOURCE_QOS,
    SOURCE_TOPIC,
    analyze_snapshot,
    json,
    mqtt,
    normalize_source_payload,
    source_message_id,
    threading,
    utc_now_iso,
):
    """Long-running MQTT subscriber and publisher service."""

    class CorrosionBNService:
        def __init__(self):
            self.lock = threading.RLock()
            self.client = None
            self.running = False
            self.connected = False
            self.last_message_id = None
            self.last_source_timestamp = None
            self.last_analysis_timestamp = None
            self.previous_risk_state = None
            self.processed_count = 0
            self.duplicate_count = 0
            self.error_count = 0
            self.last_error = None
            self.last_result = None
            self.connection_rc = None
```

```
        def _create_client(self):
            # Use MQTT 3.1.1 and callback API v1 for broad
compatibility with
            # EMQX and different paho-mqtt releases bundled in
Notebook images.
            kwargs = {
                "client_id": MQTT_CLIENT_ID,
                "clean_session": True,
                "protocol": mqtt.MQTTv311,
                "transport": "tcp",
            }
            try:
                client = mqtt.Client(

callback_api_version=mqtt.CallbackAPIVersion.VERSION1,
                    **kwargs,
                )
            except (AttributeError, TypeError):
                client = mqtt.Client(**kwargs)

            client.username_pw_set(MQTT_USERNAME, MQTT_PASSWORD)
            client.on_connect = self._on_connect
            client.on_disconnect = self._on_disconnect
            client.on_message = self._on_message
            client.reconnect_delay_set(min_delay=1, max_delay=30)
            client.enable_logger()
            return client

        def _on_connect(self, client, userdata, flags, rc,
properties=None):
            code = int(rc)
            with self.lock:
                self.connected = code == 0
                self.connection_rc = code
                self.last_error = None if code == 0 else f"MQTT
connection rejected, rc={code}"
            if code == 0:
                sub_rc, mid = client.subscribe(SOURCE_TOPIC,
qos=SOURCE_QOS)
                if sub_rc != mqtt.MQTT_ERR_SUCCESS:
                    with self.lock:
                        self.last_error = f"MQTT subscribe failed,
rc={sub_rc}"
                    return
                print(f"[BN SERVICE] connected and subscribed to
{SOURCE_TOPIC}")
            else:
                print(f"[BN SERVICE ERROR] connection rejected rc=
{code}")
```

```
        def _on_disconnect(self, client, userdata, rc,
properties=None):
            with self.lock:
                self.connected = False
                self.connection_rc = int(rc) if rc is not None else
None
                if self.running and rc:
                    self.last_error = f"MQTT disconnected
unexpectedly, rc={rc}"
            print(f"[BN SERVICE] disconnected: {rc}")

        def _publish_json(self, topic: str, payload: dict):
            info = self.client.publish(
                topic,
                payload=json.dumps(payload, separators=(",", ":"),
ensure_ascii=False),
                qos=OUTPUT_QOS,
                retain=False,
            )
            info.wait_for_publish(timeout=10)
            if info.rc != mqtt.MQTT_ERR_SUCCESS:
                raise RuntimeError(f"MQTT publish failed for
{topic}: rc={info.rc}")

        def _on_message(self, client, userdata, msg):
            try:
                raw_payload = json.loads(msg.payload.decode("utf-
8"))
                payload = normalize_source_payload(raw_payload)
                message_id = source_message_id(payload)

                with self.lock:
                    if message_id == self.last_message_id:
                        self.duplicate_count += 1
                        return
                    previous_risk_state = self.previous_risk_state

                result = analyze_snapshot(payload,
previous_risk_state)

                # Publish only after both payloads have been
generated successfully.
                self._publish_json(ANALYSIS_TOPIC,
result["analysis_payload"])
                self._publish_json(RISK_TOPIC,
result["risk_payload"])

                with self.lock:
                    self.last_message_id = message_id
                    self.last_source_timestamp =
payload["timestamp"]
```

```
                    self.last_analysis_timestamp =
result["analysis_payload"]["timestamp"]
                    self.previous_risk_state = result["risk_state"]
                    self.processed_count += 1
                    self.last_result = result
                    self.last_error = None

                print(
                    "[BN SERVICE] processed",
                    payload["timestamp"],
                    "->",
                    result["risk_state"],
                )
            except Exception as exc:
                with self.lock:
                    self.error_count += 1
                    self.last_error = f"{type(exc).__name__}: {exc}"
                print("[BN SERVICE ERROR]", self.last_error)

        def start(self):
            with self.lock:
                if self.running:
                    return False
                if not MQTT_PASSWORD:
                    raise RuntimeError(
                        "TIER0_MQTT_PASSWORD is empty. Set it before
starting the service."
                    )
                self.client = self._create_client()
                self.running = True
            try:
                # Synchronous connect surfaces DNS, socket, and
authentication
                # failures immediately instead of leaving
running=True forever.
                rc = self.client.connect(
                    MQTT_BROKER_HOST,
                    MQTT_BROKER_PORT,
                    keepalive=30,
                )
                if rc != mqtt.MQTT_ERR_SUCCESS:
                    raise RuntimeError(f"MQTT connect() failed, rc=
{rc}")
                self.client.loop_start()
                return True
            except Exception as exc:
                with self.lock:
                    self.running = False
                    self.connected = False
                    self.last_error = f"{type(exc).__name__}: {exc}"
                    self.client = None
```

```
                raise

        def stop(self):
            with self.lock:
                if not self.running:
                    return False
                client = self.client
                self.running = False
                self.connected = False
                self.client = None
            try:
                client.disconnect()
            finally:
                client.loop_stop()
            return True

        def restart(self):
            self.stop()
            return self.start()

        def status(self) -> dict:
            with self.lock:
                return {
                    "running": self.running,
                    "connected": self.connected,
                    "connection_rc": self.connection_rc,
                    "client_id": MQTT_CLIENT_ID,
                    "broker": f"{MQTT_BROKER_HOST}:
{MQTT_BROKER_PORT}",
                    "source_topic": SOURCE_TOPIC,
                    "analysis_topic": ANALYSIS_TOPIC,
                    "risk_topic": RISK_TOPIC,
                    "processed_count": self.processed_count,
                    "duplicate_count": self.duplicate_count,
                    "error_count": self.error_count,
                    "last_source_timestamp":
self.last_source_timestamp,
                    "last_analysis_timestamp":
self.last_analysis_timestamp,
                    "previous_risk_state": self.previous_risk_state,
                    "last_error": self.last_error,
                }

    # Reuse a previous service instance when the cell is re-
executed, preventing
    # duplicate subscribers inside the same Python process.
    existing = globals().get("_CORROSION_BN_SERVICE")
    if existing is not None:
        try:
            existing.stop()
        except Exception:
```

```
            pass

    CORROSION_BN_SERVICE = CorrosionBNService()
    globals()["_CORROSION_BN_SERVICE"] = CORROSION_BN_SERVICE

    AUTOSTART_ERROR = None
    if AUTOSTART:
        try:
            CORROSION_BN_SERVICE.start()
        except Exception as exc:
            AUTOSTART_ERROR = f"{type(exc).__name__}: {exc}"

    return AUTOSTART_ERROR, CORROSION_BN_SERVICE, CorrosionBNService
```

- Celda 6 - Muestra el estado de conexión MQTT y las estadísticas del servicio.

Mostrar código

```
@app.cell
def _(AUTOSTART_ERROR, CORROSION_BN_SERVICE, mo, pd):
    """Service status and usage instructions."""
    status = CORROSION_BN_SERVICE.status()

    blocks = [
        mo.md("## MQTT analysis service"),
        pd.DataFrame([status]),
    ]
    if status.get("last_error"):
        blocks.append(mo.md(f"### Connection / processing
error\n`{status['last_error']}`"))
    elif status.get("running") and not status.get("connected"):
        blocks.append(mo.md("### Connecting\nThe MQTT client has
started but has not completed the broker connection yet."))

    if AUTOSTART_ERROR:
        blocks.append(
            mo.md(
                f"""
                ### Service not started

                `{AUTOSTART_ERROR}`

                Set the MQTT password in the Notebook environment
and run this cell again:

                ~~~python
                import os
```

```
                # MQTT credentials are already configured in this
notebook.
                ~~~
                """
            )
        )
    else:
        blocks.append(
            mo.md(
                """
                The service starts automatically and processes one
complete source
                snapshot for every new MQTT message. No manual
analysis trigger or SQL
                polling is required.

                Available controls in Python:

                ~~~python
                CORROSION_BN_SERVICE.status()
                CORROSION_BN_SERVICE.stop()
                CORROSION_BN_SERVICE.start()
                CORROSION_BN_SERVICE.restart()
                ~~~
                """
            )
        )

    mo.vstack(blocks)
    return status,
```

- Celda 7 - Muestra el resultado más reciente del análisis Bayesian.

Mostrar código

```
@app.cell
def _(CORROSION_BN_SERVICE, mo):
    """Display the most recent analysis result when one is
available."""
    latest = CORROSION_BN_SERVICE.last_result
    if latest is None:
        mo.md("### Latest result\nNo source message has been
processed yet.")
    else:
        mo.vstack(
            [
                mo.md("### Latest source snapshot result"),
```

```
                mo.md(f"- Source timestamp:
`{latest['source_timestamp']}`"),
                mo.md(f"- Risk state: `{latest['risk_state']}`"),
                mo.md("#### Root states"),
                latest["root_states"],
                mo.md("#### Evidence"),
                latest["evidence"],
                mo.md("#### Corrosion analysis payload"),
                latest["analysis_payload"],
                mo.md("#### Corrosion risk payload"),
                latest["risk_payload"],
            ]
        )
    return


if __name__ == "__main__":
    app.run()
```

3. Ejecuta todas las celdas y ve a **UNS** para comprobar los resultados.

Crear Bow-tie App

1. En Tier0, ve a **Builder**.
2. Introduce los requisitos de la aplicación en el cuadro de diálogo y comienza la construcción.

> ⚠ **Lo que debe indicarse**
> Recuerda indicar al agent que use datos de UNS y proporcionar topic paths específicos. De lo contrario, el agent puede tener problemas para mostrar los datos correctos.

Mostrar prompt de Builder

```
    Crea una aplicación industrial moderna de Bow-Tie Analysis para
la corrosión atmosférica superior de CDU.

    Usa los siguientes topics de UNS como las únicas fuentes de datos:

    - Refinery/CDU_Plant/Metric/Corrosion_Monitoring
    - Refinery/CDU_Plant/Metric/Corrosion_Analysis
```

- Refinery/CDU_Plant/State/Corrosion_Risk

La aplicación debe actualizarse automáticamente cuando lleguen nuevos mensajes de UNS.

La aplicación debe ayudar a los operadores a entender el riesgo actual de corrosión, las amenazas contribuyentes, las condiciones de las barreras y las posibles consecuencias.

Crea un dashboard de una sola página con:

1. Un diagrama Bow-Tie como sección principal:
- Amenazas a la izquierda
- Barreras preventivas entre las amenazas y el evento principal
- Corrosión como evento principal central
- Barreras mitigadoras después del evento principal
- Consecuencias a la derecha

2. Un resumen de riesgo destacado que muestre:
- Estado de riesgo actual
- Confianza del modelo
- Probabilidades normal, en desarrollo y confirmada
- Riesgo total de corrosión

3. Un panel compacto de datos de proceso en vivo que muestre valores clave de Corrosion_Monitoring, incluidos chloride, pH, wash water flow, desalter performance, wash water rate, total iron, SAP evidence e injection quill status.

4. Indicadores de consecuencias para:
- Pérdida de contención primaria
- Parada no planificada
- Riesgo de escalamiento

5. Reglas visuales de estado:
- Verde para condiciones normales o saludables
- Ámbar/amarillo para riesgo en desarrollo o condiciones de advertencia
- Rojo para riesgo confirmado, barreras fallidas o consecuencias graves
- Gris cuando los datos no están disponibles
- Azul solo para interacción y resaltados informativos

6. Interacción:
- Al seleccionar una amenaza, barrera o consecuencia, se debe mostrar su estado actual, los valores de proceso relacionados y una breve explicación.
- Actualiza la interfaz automáticamente cuando lleguen nuevos datos de UNS.
- Muestra la marca de tiempo de los datos más recientes y el estado de conexión/suscripción desde el SDK integrado.

```
    Requisitos de diseño:

    Mantén el lenguaje visual y el estilo existentes en toda la
aplicación.

    Usa un estilo moderno de dashboard de operaciones industriales:
    - Paleta azul grisácea oscura en lugar de negro puro.
    - Paneles por capas con gradientes sutiles.
    - Diseño basado en tarjetas con esquinas grandes, de alrededor de 16-
20px.
    - Bordes finos para separar elementos en lugar de sombras pesadas.
    - Profundidad suave y efectos de brillo moderados.
    - Espaciado limpio con padding generoso y alineación consistente.
    - IBM Plex Sans para el texto del cuerpo.
    - Space Grotesk para los encabezados.
    - Tarjetas compactas y densas en información, con jerarquía visual
clara.
    - Etiquetas de sección en mayúsculas con mayor espaciado entre
letras.
    - Una interfaz profesional de refinería / SCADA, no un dashboard de
analítica empresarial.
    - Ilustraciones planas y gráficos SVG en lugar de elementos
skeuomórficos.
    - Transiciones y efectos hover suaves y sutiles.

    La apariencia general debe sentirse limpia, moderna, técnica y
operativa.

    Evita:
    - Fondos negro puro
    - Glassmorphism
    - Estilo cyberpunk
    - Iluminación neón
    - Efectos demasiado decorativos
    - Estilo de dashboard de analítica empresarial

    Usa un estilo limpio y profesional de operaciones de refinería, con
jerarquía clara, tarjetas compactas, etiquetas legibles y layout
responsive.

    Usa los campos de payload exactamente como están definidos en los
modelos UNS correspondientes. No inventes campos adicionales, valores
calculados ni datos mock.
```

3 Once the application is complete after certain rounds of refining, deploy it.

4 Go to **Launchpad**, open the application and check.

BUENAS PRÁCTICAS

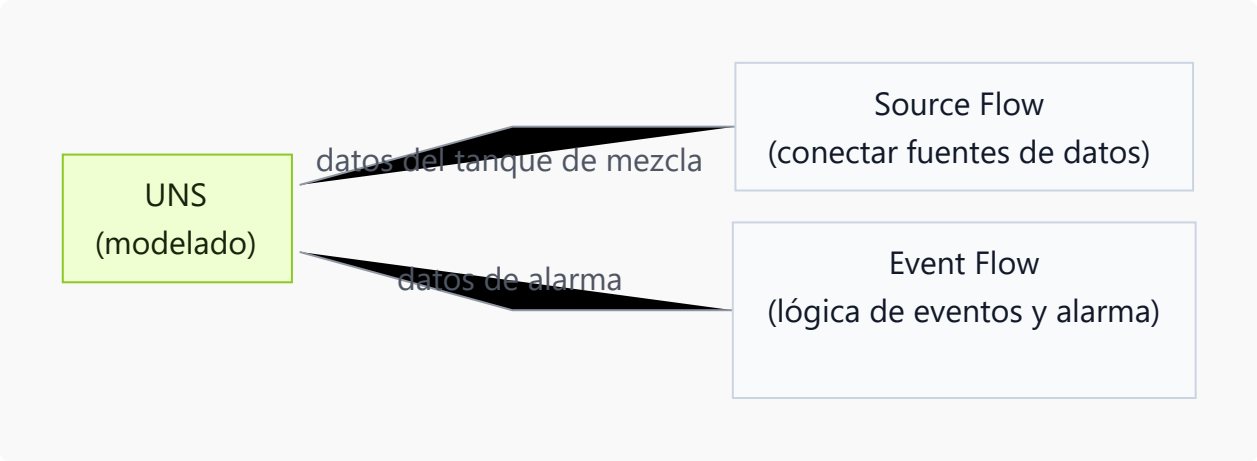
Crear flujos de datos con Agent

Usa UNS Agent para crear workflows que cubren modelado de datos, conexión de datos y eventos de datos.

En lugar de crear modelos, conexiones y eventos paso a paso en distintos módulos, UNS Agent gestiona el workflow mediante conversación.

Contexto del workflow

Un tanque de mezcla combina materiales mientras los calienta. Crea un modelo de datos y recopila temperatura, nivel de agua y estado del calentador para determinar si el tanque se sobrecalienta y activar una alarma.



Cómo crear el workflow

1. Inicia una conversación con UNS Agent en **UNS** usando `full_access`.
2. Introduce el prompt para crear los modelos UNS.

```
Crea un modelo de datos que represente la temperatura, el nivel de agua
y el estado del calentador de un tanque de mezcla. Coloca temperatura y
nivel en un metric topic, y el estado del calentador en un state topic.
```

3 Cuando confirmes que el modelo está completo, introduce el prompt para que el Agent conecte los datos correspondientes al modelo.

```
Crea un Source Flow para enviar datos a estos 2 topics y simula datos
razonables.
```

> **Cuando tengas fuentes de datos reales**
> Indica al Agent la información de la fuente de datos para que realice la conexión.

4 Introduce el prompt para crear un evento con la siguiente lógica.

- Advertencia de bajo nivel de agua: se activa cuando el nivel de agua es inferior al 20% y el calentador está encendido.
- Alarma de calentamiento en seco: se activa cuando el nivel de agua es inferior al 15%, la temperatura supera 90°C y el calentador está encendido.

```
Crea un Event Flow para la siguiente lógica de alarma:
  - Warning: activar cuando level < 20 y heater_status = true.
  - Critical: activar cuando level < 15, temperature > 90 y heater_status =
true.
  - Limpiar la alarma activa cuando level > 25 o heater_status = false.
Crea un nuevo topic bajo el modelo UNS existente para recibir el resultado
de la alarma. Incluye el nivel de alarma, el mensaje, el estado activo y la
marca de tiempo en la salida.
```

5 Comprueba los resultados en **UNS**.

> **Nota**
> Después de cada paso, puedes seguir hablando con el Agent para ajustar el resultado.

BUENAS PRÁCTICAS

Mostrar datos con Digital Twin

Visualiza datos UNS en modelos y escenas 3D en tiempo real.

La función digital twin de Tier0 permite ver datos UNS en vivo sobre modelos 3D dentro de escenas realistas.

Visualizar la escena de fábrica

1. Después de subir un modelo en **Model**, selecciona los nodos que se mapearán con datos UNS para movimientos en tiempo real.
2. Añade instancias con distintos datos UNS al modelo para simular distintas máquinas.
3. En **Scene**, añade las instancias existentes desde **Instance Library**.
4. Usa las herramientas superiores para mover, escalar y rotar modelos hasta que encajen en la escena, y ajusta la perspectiva y la posición de coordenadas.
5. Confirma la posición del modelo y los datos UNS vinculados en **Object**.

Referencia

Referencia de API

Aspectos esenciales de la integración de API y una referencia OpenAPI legible por máquinas para Tier0.

Primeros pasos

- Usa `https://<your-tier0-host>/openapi/v1` como URL base.
- Envía `x-api-key` a los endpoints OpenAPI excepto a `/assets/files/public`. Los archivos marcados como `public` se pueden descargar sin una clave de API.
- Llama primero a `POST /auth/whoami` para confirmar la identidad y los permisos efectivos de la clave de API.

> ⓘ **Referencia OpenAPI**
> Abre la **Referencia de API** interactiva para consultar esquemas de solicitud, esquemas de respuesta y solicitudes ejecutables.

Gestión de resultados

Las operaciones por lotes de UNS pueden fallar parcialmente incluso con respuestas HTTP `200`. Revisa cada resultado antes de asumir que la operación tuvo éxito.

Rutas HTTP In personalizadas en Flows

En las API desarrolladas dentro de **Source Flow** y **Event Flow**, la entrada y la respuesta las define el Flow desplegado.

- **Source Flow** usa `/flow/source/` y es público de forma predeterminada. Añade control de acceso de entrada o a nivel de Flow antes de exponer comportamiento sensible.
- **Event Flow** usa `/flow/event/` y requiere `x-api-key` con permiso `flow.read`.

> ⓘ **Ejemplo de API personalizada en Source Flow o Event Flow**
> Por ejemplo, si la URL de la API es `/integrations/echo` y necesitas un nodo `HTTP In` para desarrollar la API:

- La ruta de `HTTP In` es `/integrations/echo`; Tier0 completa el resto.
- La URL de llamada externa es `https://tier0.example.com/flow/source/integrations/echo` o `https://tier0.example.com/flow/event/integrations/echo`.

Solución de problemas

- `401` / `403`: Comprueba la clave de API, el alcance y los permisos del propietario.
- `404`: Comprueba el método HTTP, el prefijo de ruta, la URL del nodo y el despliegue.
- `No response`: Asegúrate de que cada ruta del Flow llega a un nodo `HTTP Response` sin descartar las propiedades originales de la solicitud y la respuesta.

REFERENCIA

Referencia de comandos CLI

Referencia de comandos del CLI y del agent skill.

Buscar comandos, opciones o ejemplos...

Opciones globales Estas opciones se pueden usar con cualquier comando `tier0` .

`--json` Genera JSON sin procesar `--debug` Muestra detalles de solicitud/respuesta HTTP

Operaciones básicas

`tier0 doctor`

Diagnostica la conexión y autenticación local del CLI de Tier0

Uso

```
tier0 doctor
```

Notas

- Comprueba la configuración local, la conectividad OpenAPI, la presencia de API key y la identidad de la API key.

Ejemplo

```
tier0 doctor
```

`tier0 config`

Ver o actualizar configuración

Uso

```
tier0 config
```

Opciones

`--base-url`
Configurar la base URL de la plataforma

`--api-key`
Configurar la API key (alternativa a tier0 login)

`--lang`
Flag de compatibilidad obsoleto; la salida del CLI solo está en inglés
obsoleto

Ejemplos

```
tier0 config
```

```
tier0 config --base-url https://tier0.dev
```

```
tier0 config --api-key sk-per-xxxxxx
```

`tier0 login`

Autenticarse mediante Device Flow

Uso

```
tier0 login
```

Opciones

`--no-wait`

Imprime la URL de autorización y sale sin sondeo

`--setup-code`

Sondea un setup code existente en lugar de crear uno nuevo

`--base-url`

Sobrescribir la base URL de la plataforma

Notas

- Sin `--no-wait`, el CLI consulta cada 5 segundos durante un máximo de 10 minutos.
- Con `--no-wait` `--json`, el CLI devuelve `verification_url`, `setup_code` y `expires_in`.

Ejemplos

```
tier0 login
```

```
tier0 login --no-wait --json
```

```
tier0 login --setup-code <setup_code>
```

`tier0 api`

Llamar directamente a un API endpoint

Uso

```
tier0 api <endpoint>
```

Opciones

`--body`

Cadena JSON del cuerpo de la solicitud

`--body-file`

Leer el cuerpo de la solicitud desde un archivo

`--method`

Método HTTP (GET|POST|PUT|DELETE)
predeterminado: `POST`

`--dry-run`

Previsualizar la solicitud sin ejecutarla

Notas

- `--body` y `--body-file` son mutuamente excluyentes.

Ejemplos

```
tier0 api /openapi/v1/uns/browse --body '{"path":"/"}'
```

```
tier0 api /openapi/v1/uns/read --body '{"topics":["demo"]}'
```

```
tier0 api /openapi/v1/uns/write --body-file body.json
```

`tier0 upgrade`

Actualizar el CLI a la última versión

Uso

```
tier0 upgrade
```

Opciones

`--dry-run`

Comprobar actualizaciones sin instalar

`-v, --version`

Actualizar a una versión específica (predeterminado: latest)

Ejemplos

```
tier0 upgrade
```

```
tier0 upgrade --dry-run
```

`tier0 uninstall`

Desinstalar tier0 CLI

Uso

```
tier0 uninstall
```

Opciones

`--purge`

Eliminar también el archivo de configuración (credenciales)

`--remove-skills`

Eliminar también la Tier0 Skill de los AI agents detectados

`--keep-skills`

Conservar agent skills (obsoleto; ahora es el comportamiento predeterminado)
obsoleto

Notas

- Las Agent Skills y la configuración se conservan de forma predeterminada.
- El directorio local de skills integradas se elimina durante la desinstalación del CLI.

Ejemplos

```
tier0 uninstall
```

```
tier0 uninstall --purge
```

```
tier0 uninstall --remove-skills
```

```
tier0 uninstall --purge --remove-skills
```

`tier0 version`

Mostrar versión

Uso

```
tier0 version
```

Ejemplo

```
tier0 version
```

Skills

`tier0 skills install`

Instalar o reparar la Skill integrada en este CLI

Uso

```
tier0 skills install
```

Opciones

`--force`

Reemplazar la Skill activa con la línea base integrada

`--no-sync`

Instalar localmente sin sincronizar con los AI agents detectados

Ejemplos

```
tier0 skills install
```

```
tier0 skills install --force
```

`tier0 skills list`

Listar skills instaladas

alias: ls

Uso

```
tier0 skills list
```

Ejemplo

```
tier0 skills list
```

`tier0 skills update`

Actualizar skills a la última versión

alias: upgrade

Uso

```
tier0 skills update
```

Opciones

`--dry-run`
Comprobar actualizaciones sin instalar

Ejemplos

```
tier0 skills update
```

```
tier0 skills update --dry-run
```

`tier0 skills version`

Mostrar información de versión de las skills

alias: ver

Uso

```
tier0 skills version
```

Ejemplo

```
tier0 skills version
```

`tier0 skills status`

Mostrar el estado de la Skill activa e integrada

Uso

```
tier0 skills status
```

Ejemplo

```
tier0 skills status
```

`tier0 skills sync`

Sincronizar la Skill activa con los AI agents detectados

Uso

```
tier0 skills sync
```

Ejemplo

```

```

```
tier0 skills sync
```

Autenticación

`tier0 auth whoami`

Mostrar la identidad de la API key actual

Uso

```
tier0 auth whoami
```

Notas

- Muestra usuario, workspace, nombre de key, tipo de key, roles y permisos.

Ejemplos

```
tier0 auth whoami
```

```
tier0 auth whoami --json
```

UNS

`tier0 uns browse`

Explorar el árbol de namespace de UNS

Uso

```
tier0 uns browse
```

Opciones

`-p, --path`
Ruta para explorar en el árbol UNS
predeterminado: `/`

`-d, --max-depth`
Profundidad máxima de recursión (0 = ilimitado)
predeterminado: `1`

`--include-metadata`
Incluir metadatos del nodo

`--include-leaf-value`
Incluir valores de nodos hoja

Ejemplos

```
tier0 uns browse
```

```
tier0 uns browse --path /devices
```

```
tier0 uns browse --path / --max-depth 2
```

`tier0 uns search`

Buscar topics de UNS

Uso

```
tier0 uns search
```

Opciones

`-k, --keyword`
Buscar por palabra clave de nombre

`--path-prefix`
Filtrar por prefijo de ruta
predeterminado: `/`

`--topic-type`
Filtrar por topic type

`--page`
Número de página
predeterminado: `1`

`-l, --size`
Tamaño de página (resultados máximos)
predeterminado: `20`

`--include-metadata`
Incluir metadatos del nodo

`--include-leaf-value`
Incluir valores de nodos hoja

Ejemplos

```
tier0 uns search --keyword temp
```

```
tier0 uns search --path-prefix /devices --size 50
```

```
tier0 uns search --keyword temp --include-metadata
```

`tier0 uns read`

Leer el valor actual de topics de UNS

Uso

```
tier0 uns read [topic...]
```

```
tier0 uns read --topic <topic>
```

Opciones

`-t, --topic`

Nombre(s) de topic para leer (repetible; también se aceptan argumentos posicionales)
repetible

`--include-metadata`

Incluir metadatos del topic (topicType, fields, description)

`--include-leaf-value`

Incluir valores de nodos hoja

Notas

- Se requiere al menos un topic mediante argumentos posicionales o `--topic`.

Ejemplos

```
tier0 uns read demo
```

```
tier0 uns read --topic demo
```

```

```

```
tier0 uns read temp humidity
```

```
tier0 uns read --topic sensor1 --include-metadata
```

`tier0 uns write`

Escribir un valor en un topic de UNS

Uso

```
tier0 uns write
```

Opciones

`-t, --topic`
Nombre del topic en el que escribir (obligatorio)
obligatorio

`-v, --value`
Cadena de valor JSON (mutuamente excluyente con --file)

`-f, --file`
Leer valor desde archivo (mutuamente excluyente con --value)

`--qos`
Nivel MQTT QoS (0/1/2)
predeterminado: `0`

`--retain`
Configurar flag retain de MQTT

`--dry-run`
Previsualizar la solicitud sin ejecutarla

Notas

- `--value` y `--file` son mutuamente excluyentes.
- El valor debe ser un JSON object válido que coincida con los topic fields.

Ejemplos

```
tier0 uns write --topic demo --value '{"temp":25}'
```

```
tier0 uns write --topic demo --file payload.json
```

```
tier0 uns write --topic sensor1 --value '{"on":true}' --qos 1 --retain
```

`tier0 uns history`

Consultar datos históricos de topics

Uso

```
tier0 uns history
```

Opciones

`-t, --topic`

Nombre(s) de topic (repetible, obligatorio)
obligatorio repetible

`--start`

Hora de inicio: relativa (-1h/-30m/-7d), ISO 8601 o 'now' (obligatorio)
obligatorio

`--end`

Hora de fin: relativa, ISO 8601 o 'now' (predeterminado: now)
predeterminado: `now`

`--page`

Número de página
predeterminado: `1`

`-l, --size`

Tamaño de página (máximo de puntos de datos)
predeterminado: `100`

`--interval`

Intervalo de agregación (por ejemplo, 1m, 1h, 1d)

`--fn`

Función de agregación (avg/max/min/sum/count)

`--field`

Nombre del campo de agregación

Ejemplos

```
tier0 uns history -t demo --start -1h
```

```
tier0 uns history -t demo --start -24h --end now --fn avg --interval 1h
```

```
tier0 uns history -t demo --start 2026-01-01T00:00:00Z --end 2026-01-02T00:00:00Z
```

`tier0 uns create`

Crear nodos de namespace de UNS

Uso

```
tier0 uns create
```

Opciones

`-t, --topic`

Ruta del topic o nombre de hoja (obligatorio si no se usa --file)

`--parent`

Prefijo de ruta padre (opcional, se combina con --topic)

`--type`

Tipo de nodo: 'path' (carpeta) o 'topic' (punto de datos)

`-d, --display-name`

Nombre visible

`--description`

Descripción

`--alias`

Alias

`-f, --file`

Leer definición de namespace desde archivo JSON ({"namespace":[]} o un array directo)

`--topic-type`

Obsoleto: el topic type ahora se deriva de la ruta (carpeta Metric/Action/State antes de la hoja)
obsoleto

`--fields`

Array JSON de schema fields

`--fields-file`

Leer array JSON de schema fields desde archivo

`--dry-run`

Previsualizar la solicitud sin ejecutarla

Notas

- Si no se usa `--file`, `--topic` y `--type` son obligatorios.
- `--fields` y `--fields-file` son mutuamente excluyentes.
- En nodos topic, el segmento inmediatamente anterior a la hoja debe ser Metric, Action o State.

Ejemplos

```
tier0 uns create --topic Plant/Line1/Metric/Temperature --type topic
```

```
tier0 uns create --parent Factory1/Line1/Station1 --topic
Metric/ProductionCount --type topic
```

```
tier0 uns create --topic Plant/Line1 --type path --display-name 'Line 1'
```

```
tier0 uns create --file namespace.json
```

`tier0 uns update`

Actualizar metadatos de topic UNS

Uso

```
tier0 uns update
```

Opciones

`-p, --path`

Ruta del topic a actualizar (obligatorio)

obligatorio

`-n, --name`

Nuevo nombre

`--alias`

Nuevo alias

`--description`

Nueva descripción

`--clear-description`

Borrar la descripción sin depender de un argumento vacío del shell

`-d, --display-name`

Nuevo nombre visible

`--extend-properties`

JSON object de propiedades extendidas

`--fields`

Array JSON de schema fields

`--fields-file`

Leer array JSON de schema fields desde archivo

`--update-mask`

Campos a actualizar (repetible, por ejemplo name,description,fields)
repetible

`--dry-run`

Previsualizar la solicitud sin ejecutarla

Notas

- `--description` y `--clear-description` son mutuamente excluyentes.
- `--fields` y `--fields-file` son mutuamente excluyentes.
- Se requiere al menos un campo para actualizar.

Ejemplos

```
tier0 uns update --path Plant/Line1/Metric/Temperature --display-name
'Line 1 Temp'
```

```
tier0 uns update --path Plant/Line1 --description 'Production line 1' --
update-mask description
```

```
tier0 uns update --path Plant/Line1/Metric/Temperature --fields
'[{"name":"temp","type":"float","unit":"C"}]' --update-mask fields
```

`tier0 uns delete`

Eliminar nodo(s) UNS

Uso

```
tier0 uns delete
```

Opciones

`-p, --path`

Ruta(s) de nodo a eliminar (repetible, obligatorio)

obligatorio repetible

`--hard`

Borrado definitivo (irreversible)

`-y, --yes`

Confirmar operación de alto riesgo (obligatorio)

obligatorio

`--dry-run`

Previsualizar la solicitud sin ejecutarla

Notas

- El borrado suave es el predeterminado y se puede restaurar.
- `--hard` es irreversible.
- Las operaciones de alto riesgo requieren `--yes` salvo que se use `--dry-run` .

Ejemplos

```
tier0 uns delete --path factory/line1/sensor/temp --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --path
factory/line1/sensor/humi --yes
```

```
tier0 uns delete --path factory/line1/sensor/temp --hard --yes
```

`tier0 uns restore`

Restaurar topic desde el historial

Uso

```
tier0 uns restore
```

Opciones

`-p, --path`
Ruta del topic a restaurar (obligatorio)
obligatorio

`-y, --yes`
Confirmar operación de alto riesgo (obligatorio)
obligatorio

`--dry-run`
Previsualizar la solicitud sin ejecutarla

Notas

- Las operaciones de alto riesgo requieren `--yes` salvo que se use `--dry-run`.

Ejemplo

```
tier0 uns restore --path Plant/Line1/Metric/Temperature --yes
```

Flujo Node-RED

`tier0 flow list`

Listar flows

alias: ls

Uso

```
tier0 flow list
```

Opciones

`-k, --keyword`
Filtrar por palabra clave de nombre

`-t, --type`
Filtrar por tipo (SourceFlow/EventFlow)

`--source`
Mostrar solo SourceFlow

`--event`
Mostrar solo EventFlow

Notas

- `--source` y `--event` son mutuamente excluyentes.
- `--type` no puede combinarse con `--source` ni `--event` .

Ejemplos

```
tier0 flow list
```

```
tier0 flow list --source --json
```

`tier0 flow get`

Obtener detalles del flow

Uso

```
tier0 flow get --id <id>
```

```
tier0 flow get <id>
```

Opciones

`--id`
Flow ID
predeterminado: `0`

Notas

- Se requiere un Flow ID positivo mediante `--id` o como argumento posicional.

Ejemplos

```
tier0 flow get --id 1 --json
```

```
tier0 flow get 1 --json
```

`tier0 flow create`

Crear un nuevo flow

Uso

```
tier0 flow create
```

Opciones

`-n, --name`

Nombre del Flow (obligatorio)
obligatorio

`-t, --type`

Tipo de Flow: SourceFlow | EventFlow (obligatorio)
obligatorio

`--source`

Definir tipo como SourceFlow

`--event`

Definir tipo como EventFlow

`--desc`

Descripción

`--template`

Cadena JSON de plantilla inicial

`--template-file`

Leer plantilla inicial desde archivo

`--dry-run`

Previsualizar la solicitud sin ejecutarla

Notas

- Use exactamente uno de `--type`, `--source` o `--event` para seleccionar el tipo de Flow.
- `--template` y `--template-file` son mutuamente excluyentes.

Ejemplos

```
tier0 flow create --name "modbus-collector" --source --desc "Modbus TCP
collector"
```

```
tier0 flow create --name "alert-handler" --event --desc "Temperature
alarm processor"
```

`tier0 flow update`

Actualizar metadatos del flow

Uso

```
tier0 flow update --id <id>
```

```
tier0 flow update <id>
```

Opciones

`--id`
Flow ID (obligatorio)
predeterminado: `0`

`-n, --name`
Nuevo nombre

`--desc`
Nueva descripción

`--template`
Nueva cadena JSON de plantilla

`--template-file`
Leer nueva plantilla desde archivo

`--favorite`
Marcar como favorito

`--unfavorite`
Quitar de favoritos

`--dry-run`
Previsualizar la solicitud sin ejecutarla

Notas

- Se requiere un Flow ID positivo mediante `--id` o como argumento posicional.
- `--template` y `--template-file` son mutuamente excluyentes.

- `--favorite` y `--unfavorite` son mutuamente excluyentes.
- Se requiere al menos un campo para actualizar.

Ejemplos

```
tier0 flow update --id 1 --name "line1-collector"
```

```
tier0 flow update --id 1 --favorite
```

`tier0 flow data`

Obtener JSON del canvas de Node-RED

Uso

```
tier0 flow data --id <id>
```

```
tier0 flow data <id>
```

Opciones

`--id`
Flow ID
predeterminado: `0`

`-o, --out`
Guardar salida en archivo

Notas

- Se requiere un Flow ID positivo mediante `--id` o como argumento posicional.

Ejemplo

```
tier0 flow data --id 1 --out flows.json
```

`tier0 flow deploy`

Desplegar JSON del canvas de Node-RED

Uso

```
tier0 flow deploy --id <id>
```

```
tier0 flow deploy <id>
```

Opciones

`--id`
Flow ID (obligatorio)
predeterminado: `0`

`--flows-json`
Cadena JSON del canvas de Node-RED

`-f, --flows-file`
Leer JSON del canvas de Node-RED desde archivo (recomendado)

`-y, --yes`
Confirmar operación de alto riesgo (obligatorio)
obligatorio

`--dry-run`
Previsualizar la solicitud sin ejecutarla

Notas

- Se requiere un Flow ID positivo mediante `--id` o como argumento posicional.
- `--flows-json` y `--flows-file` son mutuamente excluyentes.
- El despliegue reemplaza todos los nodos Node-RED existentes en el Flow.

- Las operaciones de alto riesgo requieren `--yes` salvo que se use `--dry-run`.

Ejemplos

```
tier0 flow deploy --id 1 -f flows.json --dry-run --json
```

```
tier0 flow deploy --id 1 -f flows.json --yes
```

`tier0 flow delete`

Eliminar flow(s)

alias: del, rm

Uso

```
tier0 flow delete --id <id>
```

```
tier0 flow delete <id>[,<id>...]
```

Opciones

`--id`
Flow ID(s) a eliminar (repetible)
repetible

`-y, --yes`
Confirmar operación de alto riesgo (obligatorio)
obligatorio

`--dry-run`
Previsualizar la solicitud sin ejecutarla

Notas

- Los argumentos posicionales pueden contener Flow IDs separados por comas.
- La eliminación detiene los contenedores Node-RED y no se puede deshacer.
- Las operaciones de alto riesgo requieren `--yes` salvo que se use `--dry-run`.

Ejemplo

```
tier0 flow delete --id 1 --yes
```

`tier0 flow nodes`

Listar tipos de nodos Node-RED disponibles

Uso

```
tier0 flow nodes [source|event]
```

```
tier0 flow nodes --type <SourceFlow|EventFlow>
```

```
tier0 flow nodes --source
```

```
tier0 flow nodes --event
```

Opciones

`-t, --type`
Tipo de Flow (SourceFlow/EventFlow)

`--source`
Mostrar nodos SourceFlow

`--event`

Mostrar nodos EventFlow

Notas

- Acepta como máximo un tipo de Flow posicional.
- `--source` y `--event` son mutuamente excluyentes.
- No combine entrada posicional/de tipo con `--source` o `--event`.

Ejemplos

```
tier0 flow nodes source --json
```

```
tier0 flow nodes --source --json
```

```
tier0 flow nodes --event --json
```

Archivos

`tier0 assets upload`

Subir un archivo a Tier0 object storage

Uso

```
tier0 assets upload <local-file>
```

Opciones

`--business`

Escena de negocio

predeterminado: `attachment`

`--use-by`

Ámbito de uso: user|workspace|platform
predeterminado: `workspace`

`--visibility`

Visibilidad: public|private
predeterminado: `private`

`--app-instance-id`

ID de instancia de app AI

`--session-id`

ID de sesión AI

`--multipart-size`

Tamaño de parte multipart (por ejemplo 10MB, mínimo 5MB)
predeterminado: `CLI-defined default`

`--concurrency`

Concurrencia de carga multipart (número de partes paralelas)
predeterminado: `4`

`--resume`

Reanudar una carga multipart interrumpida

`--abort`

Cancelar una carga multipart interrumpida y limpiar su estado

Notas

- Se requiere exactamente un argumento de archivo local.
- Los archivos mayores de 100MB usan automáticamente carga multipart.
- El CLI no impone un tamaño máximo estricto; el servidor aplica límites de plan/cuota.

Ejemplos

```
tier0 assets upload ./report.csv
```

```
tier0 assets upload ./report.csv --visibility public --business
attachment
```

```
tier0 assets upload ./report.csv --use-by workspace --visibility private
```

```
tier0 assets upload ./large-backup.tar.gz
```

```
tier0 assets upload ./large-backup.tar.gz --resume
```

```
tier0 assets upload ./large-backup.tar.gz --abort
```

```
tier0 assets upload ./large-backup.tar.gz --multipart-size 20MB --
concurrency 8
```

`tier0 assets download`

Descargar un archivo desde Tier0 object storage

Uso

```
tier0 assets download
```

Opciones

`--file-path`

Ruta de archivo devuelta por la carga

obligatorio

`-o, --output`

Ruta del archivo de salida (predeterminado: stdout)

`--content-disposition`

Content-Disposition personalizado

Ejemplos

```
tier0 assets download --file-path workspace/.../report.csv -o
./report.csv
```

```
tier0 assets download --file-path workspace/.../report.csv
```

`tier0 assets url`

Obtener la URL de acceso del archivo en Tier0 object storage

Uso

```
tier0 assets url
```

Opciones

`--file-path`

Ruta de archivo devuelta por la carga
obligatorio

`--expired-sec`

Segundos de expiración de la URL prefirmada
predeterminado: `3600`

`--content-disposition`

Content-Disposition personalizado

Ejemplos

```
tier0 assets url --file-path workspace/.../report.csv
```

```
tier0 assets url --file-path workspace/.../report.csv --expired-sec 300
```

`tier0 assets delete`

Eliminar un archivo de Tier0 object storage

Uso

```
tier0 assets delete
```

Opciones

`--file-path`
Ruta de archivo devuelta por la carga
obligatorio

`-y, --yes`
Confirmar operación de alto riesgo (obligatorio)
obligatorio

`--dry-run`
Previsualizar la solicitud sin ejecutarla

Notas

- La eliminación no se puede deshacer.
- Las operaciones de alto riesgo requieren `--yes` salvo que se use `--dry-run`.

Ejemplos

```
tier0 assets delete --file-path workspace/.../report.csv --dry-run --
json
```

```
tier0 assets delete --file-path workspace/.../report.csv --yes
```

REFERENCIA

Skills integradas de Builder

Referencia de las AI Skills integradas en Tier0 Builder.

Tier0 App Builder incluye actualmente 7 AI Skills integradas activas. Estas Skills ayudan al agent a convertir requisitos en aplicaciones industriales, alinear aplicaciones con el scaffold de la plataforma Tier0, reparar problemas de UI y mantener estables los entornos de preview.

`app-srs`

Significado: Punto de entrada para el análisis de requisitos de nuevas aplicaciones empresariales industriales. Se usa para WMS, MES, ERP, CRM, QMS, EAM, aprobaciones, inventario, proveedores, clientes, planificación y workflows empresariales similares.

Ejemplo:

```
Usa $app-srs para diseñar un sistema de gestión de excepciones de calidad para una fábrica de autopartes.
```

También se puede activar con lenguaje natural:

```
Genera una app de gestión de almacén con recepción, ubicación, picking, envío y conteo de inventario.
```

Qué hace:

1. Analiza roles de negocio, workflows, estados y excepciones.
2. Comprueba si el requisito es suficientemente claro.
3. Genera un SRS completo.
4. Lo guarda en `specs/spec.md` de forma predeterminada.
5. Entrega el resultado a `app-builder` por defecto, salvo que el usuario pida solo los requisitos.

`app-builder`

Significado: Skill principal para construir la aplicación.

Ejemplo:

```
Usa $app-builder para construir la primera versión desde specs/spec.md.
```

Tipos internos de trabajo: El agent decide qué tipo de trabajo usar según el prompt y el estado actual del project.

- **First Release**: Construye la primera versión desde un SRS.
- **Delta Feature**: Añade un módulo o workflow importante a una app existente.
- **Light Edit**: Gestiona cambios pequeños de campos, texto, estilo o validación sin ejecutar todo el flujo de build.

Qué comprueba:

- Entradas visibles.
- Rutas de usuario ejecutables.
- Cambios de datos o estado.
- Permisos de roles.
- Verificación de build y preview.

> **Consejo**
> Si una primera construcción no tiene `specs/spec.md`, el agent lo creará primero con `app-srs`.

`align-platform-app`

Significado: Skill de migración para alinear una app existente con el scaffold de la plataforma Tier0.

```

```

```
Usa $align-platform-app para migrar este proyecto antiguo de Vite React al
scaffold de la plataforma Tier0.
```

Fuentes adecuadas:

- Apps Vite, CRA, Next.js, React Router, HTML estático y Node/React full-stack.
- Apps antiguas de Tier0.
- Apps que dependen de LocalStorage, IndexedDB, Zustand persist, mocks en memoria o datos simulados.

Qué hace:

- Alinea la app con el scaffold Tier0 TanStack Start.
- Migra datos de negocio persistentes a Drizzle/PostgreSQL.
- Alinea rutas, APIs, services y configuración de base de datos.
- Elimina flujos de login, logout y cambio de roles propios de la app.
- Conecta con identidad y roles gestionados por Gateway.
- Conserva contratos de plataforma como `/api/health`, `/api/manifest` y `artifact.toml`.
- Verifica preview y arranque en producción.

`migrate-node-red-app`

Significado: Skill de migración de Node-RED para alinear una aplicación Node-RED con el scaffold de la plataforma Tier0. Normalmente se usa junto con `align-platform-app`.

Ejemplo:

```
Usa $align-platform-app y $migrate-node-red-app para migrar este backend Node-
RED a una app Tier0 TanStack Start.
```

Qué hace:

- Inventaría flows, subflows, function nodes, change nodes y switch nodes.
- Analiza HTTP, MQTT, base de datos, tareas programadas y llamadas a APIs externas.

- Crea una tabla de mapeo de migración.
- Separa comportamiento CRUD, workflow, scheduling, integración y simulación temporal.
- Mueve la lógica de negocio a services.
- Mueve interfaces a API routes.
- Mueve el estado a PostgreSQL.
- Mueve trabajos largos e integraciones a workers, queues o servicios externos.
- Verifica por separado browser, CRUD, workflow e integration cutover.

> ⚠ **Precaución**
> No gestiona por sí sola problemas ordinarios de preview o empaquetado.

`ui-repair`

Significado: Skill de reparación para apps existentes. Se usa para problemas de calidad de UI, no para generar apps por primera vez.

Ejemplo:

```
Usa $ui-repair para corregir overflow y desalineación de formulario en el
diálogo de edición de órdenes en pantallas pequeñas.
```

Problemas adecuados:

- Formularios o marcadores obligatorios desalineados.
- Overflow en dialog o drawer.
- Scroll horizontal accidental.
- Menús select o dropdown recortados.
- Layout roto en tablas o gráficos.
- Páginas planas con jerarquía visual débil.
- Idiomas mezclados en la UI.
- Problemas de layout en PDA, kiosk, dashboards e interfaces industriales.

> ⚠ **Precaución**
> La Skill debe localizar el componente responsable más pequeño y reparar la superficie afectada sin reconstruir toda la app. Para trabajo responsive complejo, usa `responsive-craft`.

`responsive-craft`

Significado: Skill de layout responsive para auditorías, nuevas construcciones responsive y previews multi-breakpoint.

Modos de invocación:

- **audit**: Audita y repara un layout responsive existente.
- **build**: Construye una página responsive desde cero.
- **preview**: Inicia un preview en vivo multi-breakpoint.

Ejemplos:

```
Usa $responsive-craft audit para corregir el layout móvil de este dashboard.
```

```
Usa $responsive-craft build para crear un workspace de inventario para móvil,
tablet y desktop.
```

```
Usa $responsive-craft preview para mostrar breakpoints de 375, 768, 1024 y
1440 px.
```

Estilos de ejecución:

- **Adaptive**: Avanza rápido y confirma bifurcaciones de diseño cuando aparecen.
- **Guided**: Escribe especificaciones de comportamiento responsive para cada componente antes de implementar.

`preview-runtime-stability`

Significado: Skill de troubleshooting de runtime para fallos de preview durante el desarrollo de apps. No construye funcionalidades de negocio.

Ejemplo:

```
Usa $preview-runtime-stability para diagnosticar "Preview loading timed out".
```

Problemas adecuados:

- `node_modules` faltantes o parciales.
- npm `ERESOLVE`.
- Fallos de arranque de Vite o TanStack Start.
- Conflictos en el puerto `5173`.
- Fallos de `/api/health`.
- Errores de módulo SDK SSR.
- Desajuste del puerto de preview en `artifact.toml`.
- Recargas completas frecuentes durante HMR.
- Procesos antiguos de Node o Vite que no salieron.

Flujo estándar:

1. Comprueba la instalación de dependencias.
2. Comprueba el puerto `5173`.
3. Prueba el health endpoint.
4. Reutiliza el preview si está sano.
5. Limpia el preview anterior si no está sano.
6. Reinicia y verifica el puerto, el health endpoint y el comportamiento de HMR.

Guía de selección de Skills

- Usa `app-srs` primero cuando el requisito no esté claro o necesites una especificación durable.
- Usa `app-builder` cuando el alcance de implementación ya esté claro.
- Usa `align-platform-app` y `migrate-node-red-app` para trabajos de migración.
- Usa `ui-repair`, `responsive-craft` o `preview-runtime-stability` cuando la app ya exista y el problema sea visual, responsive o de runtime.

REFERENCIA

Gestión de usuarios

Tier0 utiliza la pertenencia a un Workspace, las licencias y las cuentas de usuario personalizadas para que los clientes gestionen el acceso a la plataforma y los permisos de los usuarios.

Tier0 gestiona los permisos de usuario y el acceso de forma diferente según la edición.

- Cloud utiliza invitaciones a miembros para gestionar quién puede acceder a un Workspace.
- En Enterprise, el administrador asigna cuentas de usuario y roles para determinar las funciones a las que se puede acceder.

Control de acceso

PESTAÑA: CLOUD

- El propietario del Workspace o los miembros existentes pueden invitar a nuevos miembros mediante su número de teléfono o correo electrónico.
- Tras recibir un código o enlace de invitación, los nuevos miembros registran una cuenta y se unen al Workspace.
- Al enviar la invitación, configura el tipo de puesto del nuevo miembro (Operator o Builder) para determinar a qué funciones puede acceder.
    - **Builder**: Puede acceder a todos los módulos funcionales.
    - **Operator**: Solo puede acceder al centro de aplicaciones.
- El registro de la cuenta requiere una verificación en dos pasos.

> ⚠ **Precaución**
> Ser miembro de un Workspace no concede automáticamente acceso a la plataforma; los usuarios deben registrar una cuenta con el número de teléfono o correo electrónico al que se envió la invitación para utilizarla.

> ⓘ **Nota**
> Después de registrar una cuenta, también puedes solicitar un nuevo Workspace.

PESTAÑA: ENTERPRISE

- Tras iniciar sesión por primera vez con la cuenta predeterminada, el administrador cambia la contraseña y, a continuación, añade cuentas de usuario y asigna roles en **Configuración**.
- El administrador determina quién puede utilizar la plataforma mediante la asignación de cuentas y a qué funciones puede acceder cada cuenta mediante la asignación de roles:
    - **Admin**: Puede acceder a todos los módulos funcionales.
    - **Operator**: Solo puede acceder al **Launchpad**, donde se muestran todas las aplicaciones a las que tiene acceso.

> ⓘ **Nota**
> Inicia sesión con la cuenta creada por el administrador y utiliza las funciones dentro de los permisos del rol asignado.

Gestión de cuentas

PESTAÑA: CLOUD

Operación	Quién	Descripción
Cambiar el tipo de puesto	Creador / Builder	• El creador puede cambiar el tipo de puesto de cualquier miembro. • Builder solo puede cambiar el de un Operator.
Eliminar el Workspace	Creador	Al eliminar el Workspace, todos los datos se borran de inmediato y no se pueden recuperar.

Operación	Quién	Descripción
Transferir el Workspace	Creador	—
Quitar a un miembro	Creador / Builder	• El creador puede quitar a cualquier miembro. • Builder solo puede quitar a un Operator.
Eliminar la cuenta	Todos los miembros	• El creador debe eliminar o transferir primero su Workspace. • La cuenta del miembro abandona de inmediato todos los Workspaces. • Los datos de la cuenta se borran de inmediato y no se pueden recuperar.

PESTAÑA: ENTERPRISE

- Un Admin puede restablecer contraseñas, editar roles de usuario, deshabilitar cuentas y eliminarlas.
- No se puede editar el rol de la cuenta predeterminada inicial, ni deshabilitar o eliminar esa cuenta.
- Al eliminar una cuenta, todos sus datos se borran de inmediato y no se pueden recuperar.

REFERENCIA

Seguridad de red y datos de Tier0 Cloud

Límites de red, aislamiento de inquilinos, protección de datos y operaciones de seguridad del servicio SaaS de producción de Tier0 Cloud.

Ámbito: El servicio SaaS de producción de Tier0 Cloud, desplegado en la región de AWS de Singapur.
Fecha del documento: 22 de octubre de 2026

Tier0 Cloud utiliza controles de acceso a la red, aislamiento de inquilinos, gestión de identidades y accesos, cifrado de datos y medidas de copia de seguridad y recuperación para reducir el riesgo de ataques de red y accesos no autorizados. Este documento describe las medidas de seguridad de la plataforma y los controles que limitan la propagación de incidentes de seguridad a otros inquilinos, a servicios internos de la plataforma y a las redes de los clientes.

> ⓘ **Edición Edge**
> Este documento solo se aplica a Tier0 Cloud. Para configurar Edge, consulta **Instalación**.

1 Límites de red y control de acceso

El acceso público a la Web y a las API de Tier0 Cloud utiliza cifrado TLS y pasa por un punto de entrada perimetral unificado y una puerta de enlace de aplicaciones. Los puntos de entrada de los servicios principales validan el acceso al servidor de origen y rechazan las solicitudes directas que no cumplen los requisitos de validación, lo que reduce el riesgo de acceder al origen evitando el punto de entrada perimetral.

El entorno de producción está aislado de los entornos de desarrollo y preproducción en las capas de cuenta de nube, red, clúster y base de datos. Las credenciales de acceso se gestionan por separado. El acceso administrativo a producción utiliza un canal de red dedicado con autenticación.

2 Aislamiento de inquilinos y contención de incidentes

2.1 Aislamiento de cargas de trabajo

Las cargas de trabajo de los inquilinos se ejecutan en espacios de nombres de Kubernetes independientes y se separan de los servicios de la plataforma y de los componentes de administración mediante grupos de nodos dedicados y restricciones de planificación. Estos controles limitan la interacción directa entre las aplicaciones de los inquilinos y los componentes de la plataforma, reduciendo el impacto en otros servicios si una aplicación sufre un incidente de seguridad.

2.2 Aislamiento de red

El tráfico entrante y saliente de los espacios de nombres de los inquilinos está sujeto a políticas de red que deniegan el tráfico de forma predeterminada y solo lo permiten mediante reglas explícitas. La plataforma configura y sincroniza estas políticas automáticamente, y los componentes de red del clúster las aplican.

El tráfico entrante solo se permite desde cargas de trabajo del mismo inquilino y desde servicios de la plataforma que realizan funciones de negocio necesarias. Las reglas de salida permiten la resolución DNS del clúster, los servicios del mismo inquilino, las puertas de enlace designadas de la plataforma, los servicios de mensajería y los puntos de conexión de base de datos asignados. El acceso a Internet está sujeto a las políticas de red. Salvo las dependencias de negocio permitidas explícitamente, estas políticas restringen el acceso a direcciones de red privadas y bloquean el acceso directo a las direcciones de metadatos de las instancias de nube.

Estas medidas limitan las conexiones no autorizadas entre inquilinos y el movimiento lateral desde las cargas de trabajo de los inquilinos hacia la red interna de la plataforma. Un incidente de seguridad en una aplicación de un inquilino no concede automáticamente acceso a otros inquilinos ni a los componentes de administración de la plataforma. Las conexiones de negocio autorizadas siguen sujetas a las reglas de red y a los controles de identidad y acceso correspondientes.

2.3 Aislamiento de datos e identidades

Los datos de las aplicaciones de los inquilinos se aíslan lógicamente en la capa de base de datos. Cuando se asigna una unidad de base de datos independiente, las reglas de acceso a la red también restringen los orígenes de las conexiones.

La plataforma asigna permisos de acceso según las responsabilidades de cada servicio y limita el acceso de cada carga de trabajo a los recursos necesarios para sus funciones. El aislamiento

de red y la autorización basada en identidades limitan conjuntamente los recursos a los que puede acceder una carga de trabajo afectada.

2.4 Límites de acceso a la red del cliente

Las medidas de aislamiento de inquilinos descritas en esta sección se aplican dentro de la plataforma Tier0 Cloud. Cuando los clientes conectan sistemas internos mediante conectores, puertas de enlace u otras integraciones, los límites de acceso a sus redes también dependen del método de conexión, el enrutamiento, las reglas del cortafuegos y los permisos concedidos a las credenciales utilizadas.

Los clientes deben permitir únicamente las direcciones de destino, los puertos y los permisos de cuenta necesarios para su actividad, y segmentar los componentes de integración respecto de otros sistemas internos. El aislamiento interno de la plataforma no sustituye los controles de acceso a la red del cliente. Ambas partes deben determinar y verificar el alcance del acceso según la integración concreta, para limitar el riesgo de propagación a otros sistemas del cliente a través de conexiones autorizadas.

3 Protección de datos

La plataforma protege los datos en tránsito, en reposo y durante el acceso mediante las siguientes medidas:

- La comunicación pública con la Web y las API, así como el acceso a las bases de datos y cachés gestionadas, utiliza cifrado TLS.
- Las bases de datos, las cachés y el almacenamiento de objetos gestionados tienen habilitado el cifrado en reposo.
- El almacenamiento de objetos bloquea el acceso público y restringe el acceso a los datos mediante requisitos de cifrado, requisitos de HTTPS y permisos sobre los recursos.
- El acceso a los datos de los inquilinos está sujeto tanto al aislamiento de bases de datos como a los controles de identidad y acceso.

4 Copias de seguridad y recuperación

Tier0 Cloud realiza copias de seguridad de las bases de datos de producción y de las bases de datos de servicios incluidas en el ámbito de las copias de seguridad. Las copias se guardan en

almacenamiento de objetos dedicado con acceso restringido y se gestionan según las políticas de retención correspondientes.

Los trabajos de copia de seguridad utilizan una identidad independiente con permisos limitados a las operaciones de lectura y escritura necesarias en las ubicaciones de almacenamiento designadas. Esta identidad no tiene permisos para eliminar los objetos de las copias de seguridad. Las reglas de ciclo de vida del almacenamiento eliminan las copias caducadas, lo que reduce el riesgo de que unas credenciales de copia de seguridad comprometidas se utilicen para eliminar datos de recuperación.

Al finalizar las copias de seguridad se comprueba su integridad, y se valida su restauración en un entorno aislado para verificar que son utilizables. Las configuraciones de las aplicaciones se mantienen bajo control de versiones y los artefactos de compilación se guardan en un registro privado de imágenes de contenedor para facilitar la recuperación del servicio y la reversión de versiones.

5 Acceso administrativo y cambios en producción

El acceso administrativo a producción utiliza identidades asignadas individualmente y permisos basados en el principio de mínimo privilegio. Las identidades de consola utilizadas para operaciones sensibles requieren autenticación multifactor.

Las inspecciones rutinarias utilizan una identidad restringida de solo lectura que permite consultar el estado de las cargas de trabajo, los registros y los eventos, pero no leer credenciales sensibles, modificar recursos ni ejecutar comandos remotos. Los cambios en producción utilizan identidades y procesos autorizados por separado.

Las configuraciones de infraestructura, los manifiestos de despliegue y los scripts de operaciones se mantienen bajo control de versiones. Los cambios en producción deben definir sus objetivos, alcance del impacto, pasos de verificación y planes de reversión. Se registran la identidad que ejecuta el cambio, las diferencias de configuración y los resultados de la verificación para facilitar la trazabilidad y la recuperación.

6 Supervisión y respuesta a incidentes de seguridad

La plataforma recopila de forma centralizada las métricas operativas, los registros y el estado de salud del entorno de producción, y envía alertas de anomalías a los canales de respuesta operativa. La supervisión incluye la disponibilidad del servicio, las tasas de error, el estado de

las bases de datos, la validez de los certificados, el estado de los trabajos de copia de seguridad y el estado del clúster.

Ante un incidente de seguridad, la respuesta prioriza contener su impacto y conservar los registros, las configuraciones y la cronología del incidente. El servicio se restablece tras completar el aislamiento y la investigación necesarios. Después se revisa el incidente y se realiza un seguimiento de las medidas correctivas. Los cambios en producción durante la respuesta a incidentes siguen los procesos establecidos de autorización y registro.

7 Contacto de seguridad

Para evaluaciones de seguridad, procesos de diligencia debida de clientes o posibles vulnerabilidades, contacta con tu gestor de cuenta de Tier0 o con el equipo de seguridad. Los detalles no públicos de la arquitectura y los controles pueden facilitarse bajo acuerdos de confidencialidad adecuados. No incluyas credenciales de acceso, claves privadas ni datos sensibles de clientes en el primer contacto.

REFERENCIA

Jerarquía de equipos ISA-95

Niveles de referencia para modelar activos físicos y sistemas de control con ISA-95.

La jerarquía de equipos ISA-95 describe cómo se estructuran los activos físicos y los sistemas de control dentro de una organización.

Se divide en los siguientes niveles:

- **Enterprise**: El nivel superior, que representa toda la organización. Incluye los procesos de negocio generales y las entidades de toma de decisiones, normalmente respaldadas por sistemas ERP.
- **Site**: Representa una ubicación geográfica o instalación específica. Cada site puede tener sus propios sistemas de gestión y control local de producción.
- **Area**: Una división operativa dentro de un site, a menudo un área funcional como empaquetado, ensamblaje o almacén.
- **Process Cell**: La unidad básica de fabricación dentro de un area. Incluye todo el equipo necesario para producir un producto específico o realizar una función específica.
- **Unit**: Una parte diferenciada del process cell, responsable de una función específica como mezclar, calentar o ensamblar.
- **Equipment Module**: Un equipo modular que realiza una tarea específica dentro de un unit. Por ejemplo, en una unidad de mezcla, el módulo podría ser un elemento calefactor.
- **Control Module**: El nivel más bajo, que representa dispositivos o controladores individuales como sensores, bombas, actuadores o lazos de control PID que gestionan el control directo de los procesos físicos.

REFERENCIA

Standard Deployment Plan

Reference four-VM standard deployment plan for Tier0 Enterprise.

Este capítulo aún no está disponible en el idioma seleccionado. Se incluye la versión en inglés a continuación.

Objective

The simplified edition does not use Docker Swarm. It deploys one `enterprise` application node with a total of `4` virtual machines:

- `1` enterprise node hosts `tier0-enterprise + EMQX`.
- `2` database nodes form a PostgreSQL + TimescaleDB primary/standby cluster.
- `1` Witness node participates in database supervision, quorum, and failover.

The simplified edition reduces deployment complexity. The tradeoff is that the application layer and EMQX are single points of failure. Only the database layer keeps `DB VIP + Witness` high availability.

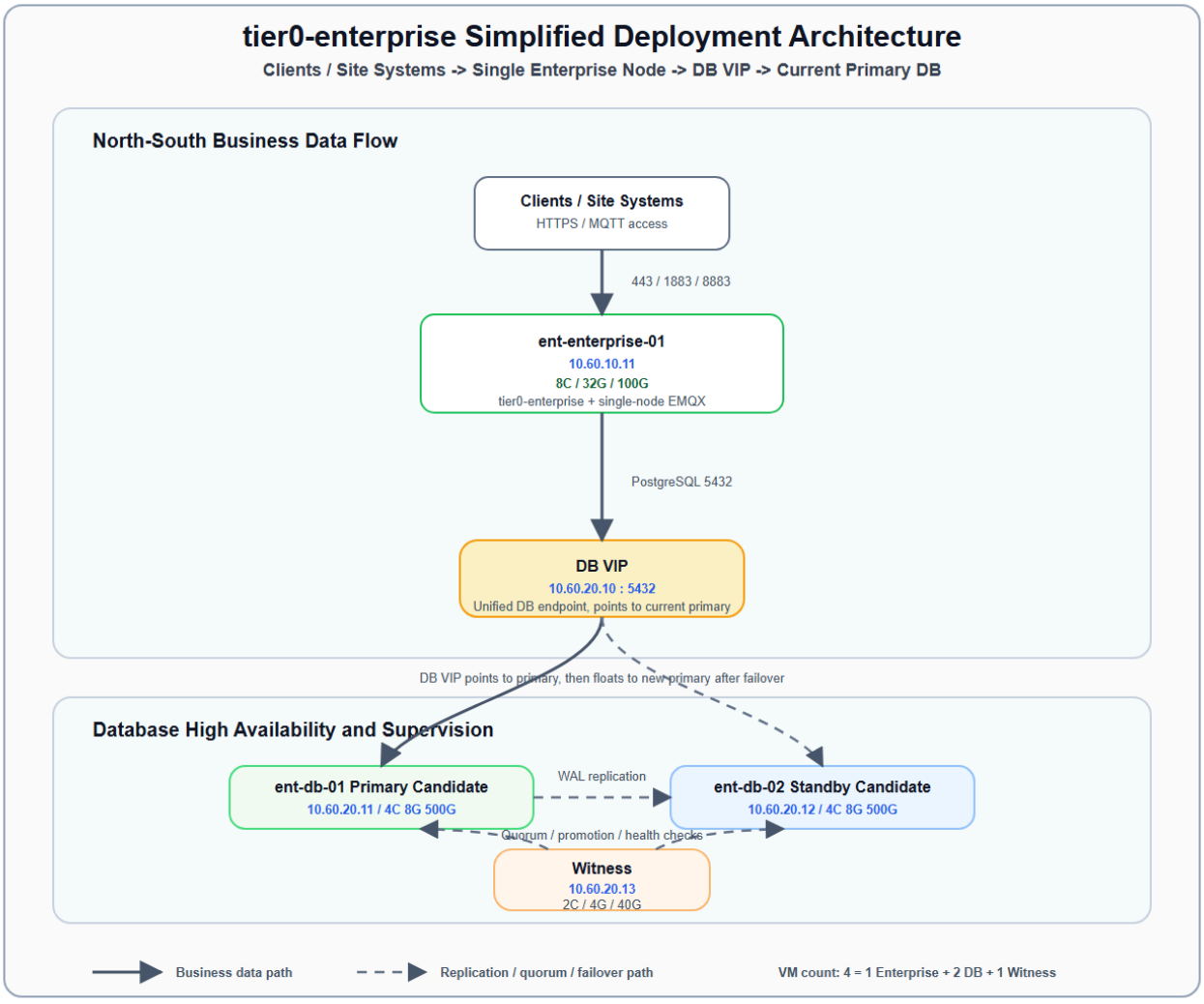
Planned IP Addresses

The following IP addresses are placeholders. Replace them with the real site network before implementation.

Type	Name	Planned IP	Description
Application node	`ent-enterprise-01`	`10.60.10.11`	Single enterprise node hosting `tier0-enterprise + EMQX`
Logical entry	`DB VIP`	`10.60.20.10`	Unified database endpoint, points only to the current primary

Type	Name	Planned IP	Description
Database node	`ent-db-01`	`10.60.20.11`	PostgreSQL + TimescaleDB primary candidate
Database node	`ent-db-02`	`10.60.20.12`	PostgreSQL + TimescaleDB standby candidate
Quorum node	`ent-db-witness-01`	`10.60.20.13`	`repmgr witness` / database quorum supervisor

Overall Topology



Core paths:

- Business path: clients / site systems -> `ent-enterprise-01` -> `DB VIP` -> current primary DB.
- `tier0-enterprise` and `EMQX` both run on `ent-enterprise-01`.
- Database access still goes through `DB VIP`; direct application access to `ent-db-01/02` is prohibited.
- `Witness` participates only in database quorum, promotion, and health checks. It is not on the business path.

Database Supervision and Failover

Phase	Path	Description
Normal access	`ent-enterprise-01 -> DB VIP -> current primary`	Application always connects to `10.60.20.10:5432`
Normal replication	`current primary -> current standby`	PostgreSQL WAL streaming replication
Normal supervision	`DB-01/DB-02 -> Witness`	`repmgrd` reports status; Witness participates in quorum
Primary failure	`standby -> Witness`	Standby requests quorum confirmation to avoid split-brain
Standby promotion	`standby promote`	Standby is promoted after Witness confirmation
VIP floating	`DB VIP -> new primary`	`10.60.20.10` binds to the new primary
Application recovery	`ent-enterprise-01 -> DB VIP -> new primary`	Application recovers after connection-pool reconnect

Node Configuration

Node	Role	vCPU	Memory	Storage	System disk recommendation	Data disk recommendation	Main components
`ent-enterprise-01`	Single enterprise node	`8`	`32G`	`100G`	Single `/` system disk, 100G	No separate data disk	tier0-enterprise, EMQX, optional Nginx/HAProxy
`ent-db-01`	PostgreSQL/TSDB primary candidate	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived
`ent-db-02`	PostgreSQL/TSDB standby candidate	`4`	`8G`	`500G`	`/` 80G	`/pgdata` 360G, `/pgwal` 40G, `/backup-staging` 20G	PostgreSQL, TimescaleDB, repmgr, keepalived
`ent-db-`	Witness / quoru	`2`	`4G`	`40G`	`/` 40G	Not planned	repmgr witness,

Node	Role	vCPU	Memory	Storage	System disk recommendation	Data disk recommendation	Main components
`witness-01`	m supervisor					separately	monitoring agent, audit logs

Configuration Summary

Category	Count	Per-node spec	vCPU subtotal	Memory subtotal	Storage subtotal
enterprise application node	`1`	`8C / 32G / 100G`	`8C`	`32G`	`100G`
PostgreSQL + TimescaleDB nodes	`2`	`4C / 8G / 500G`	`8C`	`16G`	`1000G`
Witness node	`1`	`2C / 4G / 40G`	`2C`	`4G`	`40G`
Total	`4`	-	`18C`	`52G`	`1140G`

Converted total:

- Total VMs: `4`
- Total vCPU: `18`
- Total memory: `52G`
- Total storage: `1.14T`

Connectivity Matrix

Operations Access

Target	Method	Port	Description
All VMs	`ssh -i ~/.ssh/chen-rsa ops@<IP>`	`22/TCP`	Use after unified key-based SSH is implemented; use site account before that
`ent-enterprise-01`	`https://10.60.10.11`	`443/TCP`	Business entry
`DB VIP`	`psql -h 10.60.20.10 -p 5432`	`5432/TCP`	Allow only enterprise node and operations whitelist

Application and EMQX Ports

Source	Target	Port	Purpose
Users / site systems	`ent-enterprise-01`	`80/TCP`	HTTP entry or HTTPS redirect, optional
Users / site systems	`ent-enterprise-01`	`443/TCP`	HTTPS business entry
Devices / site systems	`ent-enterprise-01`	`1883/TCP`	EMQX MQTT plaintext, optional
Devices / site systems	`ent-enterprise-01`	`8883/TCP`	EMQX MQTT TLS, recommended
Devices / site systems	`ent-enterprise-01`	`8083/TCP`	MQTT WebSocket, optional
Devices / site systems	`ent-enterprise-01`	`8084/TCP`	MQTT WebSocket TLS, optional

Source	Target	Port	Purpose
Operations network	`ent-enterprise-01`	`18083/TCP`	EMQX Dashboard, operations whitelist only
`ent-enterprise-01`	`DB VIP`	`5432/TCP`	PostgreSQL business access

Database and Supervision Communication

Source	Target	Port	Purpose
`ent-db-01`	`ent-db-02`	`5432/TCP`	PostgreSQL WAL streaming replication
`ent-db-02`	`ent-db-01`	`5432/TCP`	Reverse replication after recovery
`ent-db-01/02`	`ent-db-witness-01`	`5432/TCP`	repmgr metadata / witness communication
`ent-db-01`	`ent-db-02`	`VRRP`	DB VIP floating; allow protocol `112`

Deployment Recommendations

enterprise Application Node

- Do not deploy Docker Swarm.
- Deploy `tier0-enterprise` and `EMQX` on the same `ent-enterprise-01` node.
- `ent-enterprise-01` is an application-layer single point of failure. If it fails, recover manually or rebuild from backup.
- The node has only one `100G` system disk and is not suitable for long-term business data.
- `tier0-enterprise` business data must be stored in PostgreSQL/TimescaleDB.

- Uploads, imports/exports, and reports may be stored locally and backed up. If later upgraded to the standard edition, migrate these files to NFS/NAS or object storage.

Single-node EMQX

- EMQX is a single-node stateful service in the simplified edition.
- Pin its data directory on `ent-enterprise-01`, for example `/data/emqx`.
- Back up configuration, certificates, plugins, and rule-engine configuration.
- No message-layer HA is provided. Use the standard edition with a three-node EMQX cluster if this is unacceptable.

PostgreSQL + TimescaleDB

- PostgreSQL: `16` or the site-certified version.
- TimescaleDB: stable version compatible with the selected PostgreSQL major version.
- Replication: physical streaming replication.
- Failover: `repmgrd + witness`.
- Database entry: `DB VIP`.
- Connection pool: PgBouncer can run on `ent-enterprise-01` to reduce database connection pressure.

Witness

- Witness stores only `repmgr witness` metadata and supervision logic.
- Witness does not host business SQL.
- Witness failure does not stop the current primary, but reduces trusted automatic failover capability.

Port and Firewall Baseline

Port / protocol	Allowed scope	Purpose
`22/TCP`	Operations network to 4 VMs	SSH

Port / protocol	Allowed scope	Purpose
`80/TCP`	User network to `ent-enterprise-01`	HTTP redirect or internal plaintext entry
`443/TCP`	User network to `ent-enterprise-01`	HTTPS business entry
`1883/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX MQTT plaintext, optional
`8883/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX MQTT TLS, recommended
`8083/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX WebSocket, optional
`8084/TCP`	Devices / site systems to `ent-enterprise-01`	EMQX WebSocket TLS, optional
`18083/TCP`	Operations network to `ent-enterprise-01`	EMQX Dashboard, whitelist only
`5432/TCP`	enterprise to `DB VIP`; DB/Witness internal	PostgreSQL / repmgr
`112/VRRP`	Between DB nodes	DB VIP floating
`9100/TCP`	Monitoring system to 4 VMs	node exporter, optional

HA Capabilities and Failure Boundaries

Failure scenario	Expected behavior	Business impact
`ent-enterprise-01` fails	Application and EMQX both stop; manual recovery or rebuild from backup is required	Full application-layer outage

Failure scenario	Expected behavior	Business impact
Current DB primary fails	Standby is promoted after Witness quorum; DB VIP floats	Write interruption for seconds to minutes; application must reconnect
DB standby fails	Primary continues serving, replication alert is raised	No direct business outage, but DB hot standby is lost
Witness fails	Current primary continues serving, but automatic failover confidence is reduced	No direct business outage, restore Witness first
DB VIP floating fails	Database may have a new primary, but unified entry is unreachable	Business outage; manual VIP binding required

Pre-Go-Live Checklist

Check item	Pass criteria
4 VM hostnames are fixed	`hostnamectl` matches the plan
Unified SSH works	Operations terminal can access all 4 VMs with key-based SSH
Time synchronization	Time drift across 4 VMs is below `1s`
Firewall is open	Ports from section 9 are allowed by scope
enterprise service is healthy	`tier0-enterprise` health check passes
EMQX state is verified	EMQX node, listening ports, and persistent directory are healthy
enterprise disk usage is safe	`/` usage is below `70%`; log and cache cleanup is configured

Check item	Pass criteria
DB primary/standby is healthy	`repmgr cluster show` shows primary, standby, and witness healthy
DB VIP works	enterprise node can connect to `10.60.20.10:5432`
Primary DB failover drill	Standby can be promoted and DB VIP can float
Backup restore drill	Backup can be restored to a test database

Implementation Order

1. Create 4 VMs and fix hostnames, IPs, time synchronization, and SSH.
2. Initialize OS baseline, firewall, mounts, and kernel parameters.
3. Deploy `tier0-enterprise` on `ent-enterprise-01`.
4. Deploy single-node EMQX on `ent-enterprise-01`.
5. Deploy PostgreSQL + TimescaleDB primary/standby.
6. Deploy repmgr witness, repmgrd, and DB VIP floating scripts.
7. Verify connectivity, EMQX recovery, database failover, backup restore, and routine checks.
8. Finalize runbooks, credential archives, and routine check tasks.

Risks and Open Items

- The simplified edition has an application-layer single point. If `ent-enterprise-01` fails, the whole business service is unavailable.
- EMQX is single-node and does not provide message-layer HA.
- `ent-enterprise-01` has only a single `100G` system disk. Logs, cache, attachments, and temporary files must be controlled.
- If application-layer or EMQX HA is required later, switch to the standard edition.
- Database RPO/RTO is not confirmed. This primary/standby design cannot guarantee zero data loss.

- DB VIP floating scripts must be verified on site, especially whether the virtualization/network platform allows VRRP.

REFERENCIA

Plan de despliegue de alta disponibilidad

Plan de despliegue de referencia para un entorno Tier0 Enterprise de alta disponibilidad.

Objetivo del plan

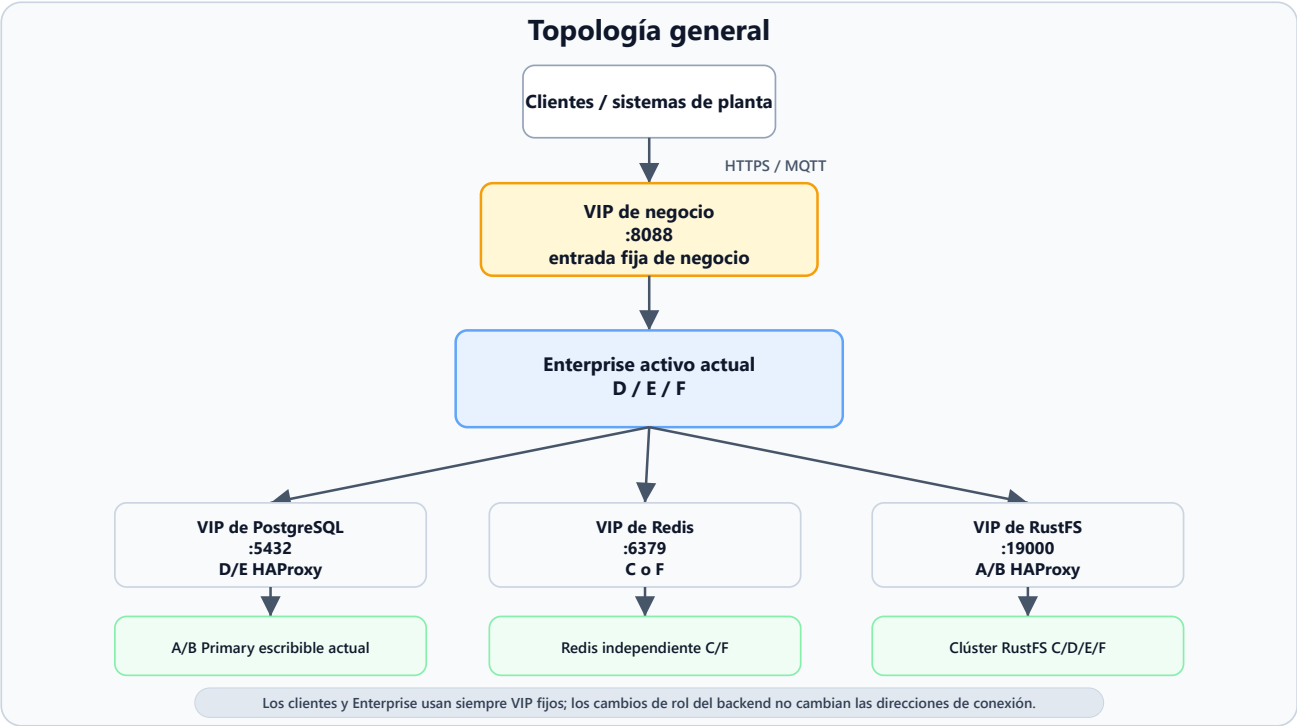
Este plan V2 despliega un Tier0 Enterprise Fleet Center lógico en un entorno sin conexión o privado. Seis máquinas comparten los roles de negocio, middleware, base de datos, almacenamiento y agente. Los usuarios y los servicios internos de Enterprise siempre se conectan mediante direcciones VIP fijas. Cuando falla una máquina o un proxy, Keepalived, HAProxy, repmgr, RustFS y Fleet Agent toman el control en sus propias capas.

El plan cubre recuperación ante fallo de una sola máquina dentro de un mismo segmento de red. Es un diseño de recuperación activa/en espera, no un clúster de alta disponibilidad entre centros de datos, con elección por consenso o con pérdida cero de datos.

- La entrada de Fleet Center puede moverse a un nodo Standby sano cuando falla el nodo de negocio actual.
- PostgreSQL, Redis y RustFS usan VIP fijas, por lo que Enterprise no necesita cambiar `.env` cuando cambian los roles backend.
- Los archivos locales de App, Flow, Marimo, License y otros activos de ejecución pueden sincronizarse de Active a Standby.
- Una Cluster License corresponde a una instalación lógica y no se activa de nuevo después de cambiar el nodo de Enterprise.
- Un deployer separado inicializa las seis máquinas, asigna roles, despliega servicios, monitorea el estado y valida el failover.
- Se separan explícitamente el failover de VIP, la sincronización de archivos, Redis independiente, la replicación de base de datos y la redundancia de almacenamiento de objetos.

Este plan aplica solo a Fleet Center. Los nodos Branch Enterprise se instalan como despliegues normales de una sola máquina y no se unen a este diseño activo/en espera de seis máquinas.

Topología general



La ruta principal de solicitudes es:

```
Clientes / sistemas del sitio
    -> Business VIP :8088
    -> Enterprise Active actual (D / E / F)
    |-- PostgreSQL VIP :5432 -> D/E HAProxy -> Primary escribible actual A/B
    |-- Redis VIP      :6379 -> Redis independiente C o F
    `-- RustFS VIP     :19000 -> A/B HAProxy -> clúster RustFS C/D/E/F
```

Las cuatro VIP son entradas fijas en la configuración de negocio. Cuando Enterprise cambia de nodo o cambian los roles backend, los clientes y Enterprise no cambian las direcciones de conexión.

Direcciones IP planificadas

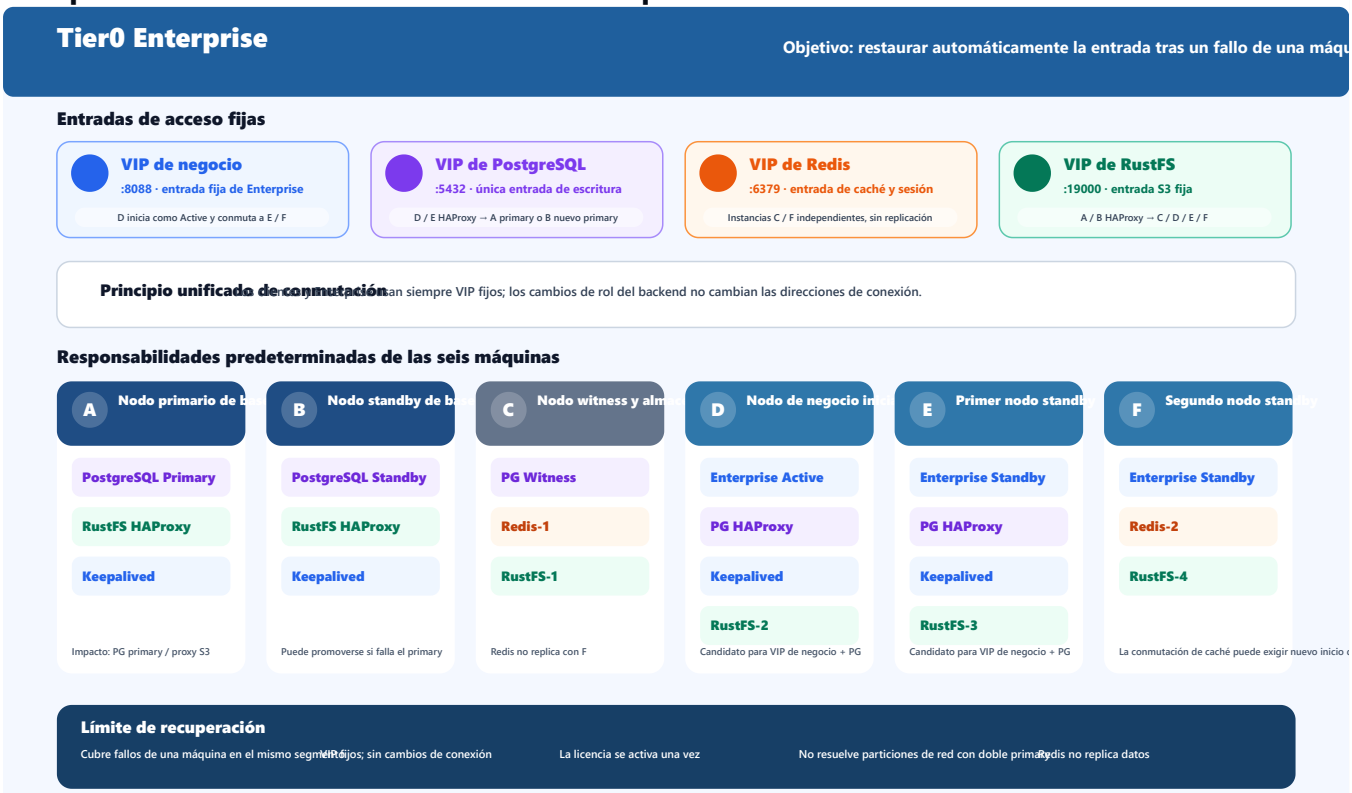
Las siguientes direcciones son solo ejemplos. Sustitúyalas por direcciones reales del mismo segmento IPv4 en sitio antes de implementar. Las cuatro VIP no deben estar ocupadas por DHCP, hosts estáticos u otras instancias VRRP.

Nombre	IP de ejemplo	Descripción
deployer-controller	`10.60.10.20`	Deployer independiente; no cuenta como una de las

Nombre	IP de ejemplo	Descripción
		seis máquinas de negocio
ent-ha-01 / A	`10.60.10.11`	PostgreSQL Primary y roles relacionados
ent-ha-02 / B	`10.60.10.12`	PostgreSQL Standby y roles relacionados
ent-ha-03 / C	`10.60.10.13`	Witness, Redis-1, RustFS-1
ent-ha-04 / D	`10.60.10.14`	Enterprise Active inicial y roles relacionados
ent-ha-05 / E	`10.60.10.15`	Enterprise Standby y roles relacionados
ent-ha-06 / F	`10.60.10.16`	Enterprise Standby y roles relacionados
Business VIP	`10.60.10.101`	Entrada unificada de Enterprise, valor predeterminado `8088`
PostgreSQL VIP	`10.60.10.102`	Entrada unificada de base de datos, valor predeterminado `5432`
Redis VIP	`10.60.10.103`	Entrada unificada de Redis, valor predeterminado `6379`
RustFS VIP	`10.60.10.104`	Entrada unificada de S3, valor predeterminado `19000`

Antes de implementar, confirme el nombre de la NIC de negocio, prefijo IPv4, gateway, DNS, NTP, hostnames, usuario SSH, puntos de montaje de disco y reglas de firewall. Los scripts no deben adivinar VIP, IP de nodos ni NIC para usarlas directamente en producción.

Responsabilidades de las seis máquinas



Máquina	Componentes desplegados por defecto	Responsabilidad clave
A	PostgreSQL Primary, RustFS HAProxy, Keepalived	Primary inicial de base de datos; primer candidato de proxy RustFS
B	PostgreSQL Standby, RustFS HAProxy, Keepalived	Standby de base de datos por streaming; segundo candidato de proxy RustFS
C	PostgreSQL Witness, Redis-1, RustFS-1	Witness/registro de estado de base de datos; nodo de datos Redis y RustFS
D	PG HAProxy, Keepalived, RustFS-2, Enterprise, Fleet Agent, rsync	Active de negocio inicial; primer candidato de proxy PostgreSQL
E	PG HAProxy, Keepalived, RustFS-3, Enterprise, Fleet Agent, rsync	Standby de negocio; segundo candidato de proxy PostgreSQL

Máquina	Componentes desplegados por defecto	Responsabilidad clave
F	Redis-2, RustFS-4, Enterprise, Fleet Agent, rsync	Standby de negocio; segunda instancia Redis

Esta asignación busca tolerar el fallo de una sola máquina y equilibrar recursos entre las seis máquinas. El deployer mapea A-F por el orden natural de hostnames/IP y selecciona roles predeterminados, pero la asignación final debe revisarse con el diseño de discos y dominios de fallo del sitio.

Lista de configuración de máquinas

La capa de negocio de Enterprise es activa/en espera. En cualquier momento, solo el nodo que mantiene la Business VIP atiende solicitudes. Escale esta capa verticalmente aumentando CPU, memoria y disco de un nodo de negocio; no estime concurrencia multiplicando el número de nodos Enterprise.

Máquina	Tipo de nodo	vCPU	Memoria	Almacenamiento base	Recomendación de disco
A	Nodo PostgreSQL Primary	8C	16G	1T	`/` aprox. 80G; espacio restante para datos PostgreSQL, WAL y staging de backups
B	Nodo PostgreSQL Standby	8C	16G	1T	`/` aprox. 80G; espacio restante para datos PostgreSQ

Máquina	Tipo de nodo	vCPU	Memoria	Almacenamiento base	Recomendación de disco
					L, WAL y staging de backups
C	Nodo Witness / middleware	4C	8G	500G	Asigne puntos de montaje fijos para sistema, Redis y datos RustFS
D	Nodo Enterprise Active inicial	8C	32G	500G	Asigne puntos de montaje fijos para sistema, contenedores, archivos sincronizados de Enterprise y datos RustFS
E	Nodo Enterprise Standby	8C	32G	500G	Igual que D
F	Nodo Enterprise Standby	8C	32G	500G	Igual que D

Categoría	Cantidad	Especificación por nodo	Subtotal vCPU	Subtotal memoria	Subtotal almacenamiento
Nodos de base de datos PostgreSQL	2	`8C / 16G / 1T`	16C	32G	2T
Nodo Witness / middleware	1	`4C / 8G / 500G`	4C	8G	500G
Nodos de negocio Enterprise	3	`8C / 32G / 500G`	24C	96G	1.5T
Total	**6**	-	**44C**	**136G**	**4T**

Resumen de configuración

Configuración unificada de conexión de Enterprise

D, E y F mantienen la misma configuración de Enterprise.

Dependencia	Principio de configuración
PostgreSQL	Conectar a PostgreSQL VIP, no a las IP de los nodos A/B
Redis	Conectar a Redis VIP, no a las IP de los nodos C/F
File storage	Usar `FILESTORE_DRIVER=s3`; apuntar el endpoint S3 a RustFS VIP
Fleet identity	Tres nodos comparten un `installationId` lógico; cada nodo tiene su propio `memberId`

Dependencia	Principio de configuración
Fleet Agent	Un Fleet Agent de nivel proceso se ejecuta en cada nodo Enterprise
License	Activar una vez mediante Business VIP; incluir el Bundle en el fileset HA

`installationId` y `memberId` son generados y persistidos por el proceso de instalación. Los operadores no deben inventarlos manualmente ni copiar `memberId` entre máquinas.

VIP y nodos candidatos

VIP	Nodos candidatos	Base de salud	Resultado de toma de control
Business VIP	D, E, F	Readiness de Fleet Agent, estado del contenedor de negocio, generación de sincronización	Solo el Owner inicia el grupo de negocio HA de Enterprise
PostgreSQL VIP	D, E	Estado de HAProxy/Keepalived; HAProxy enruta solo al Primary escribible	Enterprise mantiene la misma dirección de base de datos
Redis VIP	C, F	Redis `PING` y estado del servicio local	Cambia a la otra instancia independiente; la caché/sesión puede perderse
RustFS VIP	A, B	Salud de HAProxy y del backend RustFS	La entrada proxy se mueve mientras la dirección S3

VIP	Nodos candidatos	Base de salud	Resultado de toma de control
			permanece sin cambios

Diseño de despliegue de componentes

Enterprise, Fleet Agent y Business VIP

- D, E y F instalan el mismo paquete ZIP offline completo de Enterprise.
- Fleet Agent es un servicio de proceso del host, no un contenedor de negocio.
- Keepalived gestiona la Business VIP y llama al endpoint de readiness de Fleet Agent para decidir si el nodo local puede mantener la VIP.
- Solo el Owner de la Business VIP ejecuta el grupo de negocio; los otros dos nodos permanecen en Standby.
- Un nodo que pierde la VIP debe detener el grupo de negocio. Un nodo recuperado debe reincorporarse primero como Standby.
- EMQX se ejecuta como parte del grupo de negocio de instancia activa única. No lo reinicie fuera del control del Agent ni cree otro mecanismo de elección entre las tres máquinas.

PostgreSQL primario/en espera, Witness y entrada unificada

- A inicia como Primary, B como Streaming Standby y C ejecuta repmgr Witness.
- Witness registra el estado del clúster y participa en las decisiones de fallo. No almacena datos de negocio completos ni atiende lecturas o escrituras.
- Cuando falla el Primary, repmgr promueve un Standby sano como nuevo Primary.
- D/E HAProxy enruta el tráfico `5432` solo al Primary escribible actual.
- La PostgreSQL VIP flota entre los nodos proxy D/E. El failover del proxy y el failover Primary/Standby de la base de datos están desacoplados.
- La aceptación debe demostrar tanto “un único Primary escribible” como “el Standby reanuda streaming”.

Instancias independientes de Redis

- C y F ejecutan cada uno una instancia Redis independiente.
- Las dos instancias Redis no forman Sentinel, Cluster ni replicación primaria/réplica.
- La Redis VIP solo cambia la entrada de conexión. Cuando falla la instancia actual, la VIP se mueve a la otra instancia sana.
- Redis solo debe almacenar datos que puedan perderse o reconstruirse. Caché vacía y nuevo inicio de sesión tras el cambio forman parte del límite de diseño.
- Si Redis almacena en el futuro estado que no puede descartarse, se debe actualizar a un diseño con replicación/arbitraje en lugar de mantener la hipótesis de instancias independientes.

Clúster RustFS de cuatro nodos

- C, D, E y F forman un clúster distribuido de almacenamiento de objetos RustFS de cuatro nodos.
- HAProxy en A y B proxy todos los backends RustFS sanos.
- La RustFS VIP flota entre A y B. Enterprise siempre accede a S3 mediante esta VIP.
- El fallo de un nodo RustFS lo maneja el mecanismo del clúster; el fallo de un proxy lo maneja el failover de VIP.
- No sincronice datos de objetos con rsync, copia en caliente ni edición directa de los directorios montados de RustFS.
- Los límites de disponibilidad según cantidad de nodos, discos y erasure coding de RustFS deben seguir los resultados de comprobación del clúster de la versión desplegada.

Sincronización de archivos de App, Flow, Marimo y License

Fleet Agent solo sincroniza archivos locales de negocio que no pueden colocarse directamente en PostgreSQL/RustFS pero deben existir en el nuevo nodo Active:

- Directorios de ejecución de App y `node_modules`
- Archivos locales de ejecución de Flow, Marimo, Notebook y relacionados
- Bundle de License
- Otros filesets HA registrados explícitamente por la implementación actual

La sincronización usa `rsync` del sistema sobre confianza SSH administrada por el deployer entre nodos de negocio. Cada release crea una generation/manifest completa. Un Standby solo puede tomar el control después de confirmar que la generación está completa. Los directorios de datos de PostgreSQL, Redis y RustFS no forman parte de este alcance.

Métodos de conexión y base de red

Relaciones de acceso

Origen	Destino	Propósito
Usuarios / sistemas del sitio	Business VIP	Acceso de negocio Web/API
D/E/F Enterprise	PostgreSQL VIP	Lecturas/escrituras de base de datos de negocio
D/E/F Enterprise	Redis VIP	Caché y sesión
D/E/F Enterprise	RustFS VIP	Lecturas/escrituras de objetos S3
Controlador deployer	A-F	SSH, despliegue, recolección de estado, operaciones
D/E/F	D/E/F	Sincronización de archivos rsync/SSH
A/B/C	A/B/C	Replicación streaming PostgreSQL, repmgr, comunicación Witness
RustFS members/proxies	C/D/E/F	Comunicación del clúster RustFS y backend S3
Nodos que comparten una VIP	Candidatos correspondientes	Anuncios VRRP y movimiento de VIP

Puertos y protocolos

Permita puertos según la relación mínima necesaria entre origen y destino. No exponga todos los puertos de administración directamente a la red de usuarios.

Puerto / protocolo	Lista permitida sugerida	Propósito
`22/TCP`	Deployer -> A-F; sincronización mutua D/E/F	SSH, despliegue, rsync
`8088/TCP`	Red de usuarios -> Business VIP	Entrada de negocio Enterprise
`5432/TCP`	D/E/F -> PostgreSQL VIP; Acceso interno PG/proxy	PostgreSQL, replicación streaming, proxy
`6379/TCP`	D/E/F -> Redis VIP; nodos de health check -> C/F	Acceso Redis y health checks
`19000/TCP`	D/E/F -> RustFS VIP	Entrada S3 fija de Enterprise
`9000/TCP`	A/B -> C/D/E/F; RustFS members	Comunicación backend/clúster S3 de RustFS; confirmar por versión
`9001/TCP`	Red de operaciones controlada -> entrada admin de RustFS	Endpoint admin opcional de RustFS; no exponer a la red de usuarios
`19731/TCP`	Localhost / Keepalived / red de operaciones controlada	Interfaz HTTP de salud y control de Fleet Agent
`18080/TCP`	Red de operaciones -> controlador deployer	UI HTTPS del deployer
`112/VRRP`	Nodos candidatos de cada VIP	Movimiento de VIP de Keepalived; es un número de protocolo IP, no TCP/UDP

Puerto / protocolo	Lista permitida sugerida	Propósito
`1883/8883`	Red de dispositivos del sitio -> entrada de negocio, según configuración del producto	MQTT/MQTTS

Conmutación por error y límites de capacidad

Escenario de fallo	Comportamiento esperado	Límite
Falla el nodo de negocio Active D	La Business VIP se mueve a E o F; el nuevo Owner inicia el grupo de negocio después de pasar readiness	Requiere que la última generation del HA fileset esté completa en Standby
Falla PostgreSQL Primary A	B se promueve a Primary; PostgreSQL VIP sigue enrutando al Primary escribible mediante el proxy D/E	La pérdida de datos depende del estado de la replicación streaming
Falla el nodo Redis actual	Redis VIP se mueve a la otra instancia Redis independiente	La caché/sesión puede perderse
Falla el proxy RustFS A o B	RustFS VIP se mueve al otro proxy	Los datos RustFS siguen dependiendo de la salud del clúster de cuatro nodos
Falla un solo nodo de datos RustFS	El clúster RustFS continúa el servicio si se cumplen sus condiciones de redundancia	El límite real depende del número de discos y la configuración de erasure coding

Particiones de red, fallos simultáneos de varios nodos, prioridad VRRP incorrecta, discos llenos, deriva de reloj y cambios manuales fuera del deployer quedan fuera de la recuperación

automática normal y deben cubrirse con procedimientos operativos.

Copia de seguridad, monitoreo e inspección

- PostgreSQL requiere copias de seguridad lógicas o físicas independientes y verificación de restauración.
- RustFS requiere monitoreo de capacidad, discos y salud del clúster.
- Redis debe tratarse como caché/sesión salvo que se introduzca un diseño de replicación futuro.
- El estado de Fleet Agent y Keepalived debe monitorearse en D/E/F.
- Debe monitorearse el retraso de generation de rsync. Un Standby con una generation obsoleta no debe tomar el tráfico de negocio.
- Los simulacros regulares de failover deben incluir Business VIP, promoción de PostgreSQL, cambio de Redis VIP, cambio de proxy RustFS y restauración desde backup.

Comprobaciones antes de producción

- Las seis máquinas son accesibles por SSH desde el deployer controller.
- Los nombres de host, direcciones IP, nombres de NIC, NTP, DNS y reglas de firewall están fijados y registrados.
- Las cuatro VIP no están ocupadas y pueden moverse entre sus nodos candidatos.
- Los puntos de montaje de disco están fijados y persisten tras reinicio.
- Las configuraciones de Enterprise, PostgreSQL, Redis, RustFS, Keepalived, HAProxy, repmgr, Fleet Agent y rsync son generadas por el deployer y revisadas.
- La Cluster License se activa una sola vez mediante la Business VIP.
- Los pasos de backup y restauración se ensayan antes de introducir tráfico de producción.

Secuencia de implementación

1. Prepare el deployer controller y las seis máquinas.
2. Confirme la información de IP, VIP, NIC, disco, DNS, NTP, SSH y firewall.
3. Cargue el paquete offline de Enterprise y los materiales de configuración.
4. Despliegue PostgreSQL Primary/Standby y Witness.

5. Despliegue las instancias Redis independientes y la Redis VIP.
6. Despliegue el clúster RustFS de cuatro nodos y la RustFS VIP.
7. Despliegue Enterprise en D/E/F, Fleet Agent, los filesets rsync y la Business VIP.
8. Active la License mediante la Business VIP.
9. Ejecute la validación funcional y los simulacros de failover.
10. Registre la topología final, contraseñas/keys, política de backup, objetivos de monitoreo y procedimientos de recuperación.

Criterios de aceptación

Aceptación funcional

- Los usuarios pueden acceder a Enterprise mediante la Business VIP.
- Enterprise lee y escribe PostgreSQL solo mediante la PostgreSQL VIP.
- Enterprise usa Redis solo mediante la Redis VIP.
- Enterprise usa RustFS solo mediante la RustFS VIP.
- Los archivos de App, Flow, Marimo, Notebook y License están presentes en el nodo Active tras el failover.
- Branch Enterprise puede seguir instalándose de forma independiente y no se une a este grupo HA de seis máquinas.

Aceptación de simulacros de failover

- Detener el nodo de negocio Active: la Business VIP se mueve a un Standby sano y el grupo de negocio se inicia solo allí.
- Detener PostgreSQL Primary: Standby se promueve y la PostgreSQL VIP sigue apuntando al Primary escribible.
- Detener el owner de Redis VIP: la Redis VIP se mueve a la otra instancia, aceptando la pérdida de caché/sesión.
- Detener el owner del proxy RustFS: la RustFS VIP se mueve al otro proxy.
- Detener un nodo de datos RustFS: el acceso a objetos permanece dentro del límite real de redundancia de RustFS.
- Recuperar nodos fallidos: vuelven como Standby o miembros sanos y no sobrescriben datos o archivos más nuevos.

Riesgos y puntos a confirmar en sitio

- Si la red permite el protocolo VRRP `112`.
- Si las cuatro direcciones VIP realmente no están ocupadas.
- Si el diseño y la capacidad de montaje de discos cumplen los requisitos reales de retención de archivos y redundancia de RustFS.
- Si Redis solo almacena datos descartables.
- Si el retraso de replicación de PostgreSQL y la política de backup cumplen el RPO de negocio.
- Si el personal de operaciones entiende la diferencia entre failover de VIP, sincronización de archivos, replicación de base de datos y redundancia de almacenamiento de objetos.

Conclusión

El plan actual se centra en seis máquinas, cuatro VIP fijas y una Enterprise de instancia activa única, desacoplando respectivamente la entrada de negocio, la entrada de base de datos, la entrada de caché y la entrada de almacenamiento de objetos. Puede cubrir fallos comunes de una sola máquina dentro del mismo segmento de red y reduce significativamente el trabajo posterior al cambio: modificar la configuración de conexión, reactivar License y copiar manualmente archivos de App.

Al mismo tiempo, se deben aceptar tres límites: Redis no replica datos, los filesets de negocio se sincronizan de forma asíncrona y una partición de red no cuenta con arbitraje de consenso. Por lo tanto, este plan debe definirse como **plan de recuperación ante desastres activo/en espera de seis máquinas y una sola Enterprise activa para Fleet Center**. No debe prometerse externamente cero interrupciones, cero pérdida de datos ni alta disponibilidad estricta entre centros de datos.

REFERENCIA

SLA y límites de alta disponibilidad

Recomendaciones de SLA, límites de alta disponibilidad, criterios de aceptación, simulacros y elementos de refuerzo para el despliegue estándar de Tier0 Enterprise.

Este documento aplica al despliegue completamente offline de seis VM: tres nodos de aplicación / Swarm, dos nodos HA de PostgreSQL y un nodo witness.

Línea base de arquitectura

Capa	Diseño actual	Límite de alta disponibilidad
Runtime de aplicación	Enterprise New compose se ejecuta en el `tier0_app_active_node` fijo.	`backend`, `redis`, `EMQX`, `SourceFlow`, `EventFlow`, `Marimo` y `OPC UA Server` todavía no son servicios multi-réplica automáticos.
Entrada HTTP	Tres nodos de aplicación ejecutan keepalived APP VIP + HAProxy.	El VIP puede flotar. HAProxy reenvía al nodo activo en `8088/TCP` y comprueba `/healthz`.
MQTT / OPC UA	EMQX usa `1883/8883/8083/8084`; OPC UA usa `4840`.	Se conectan directamente al nodo activo. No están incluidos en proxy TCP VIP, por lo que no se promete HA para entradas de protocolo.
Base de datos	Replicación streaming PostgreSQL / TimescaleDB + repmgr + witness + DB VIP.	Si falla el primary, el standby puede promoverse automáticamente. La replicación asíncrona no

Capa	Diseño actual	Límite de alta disponibilidad
		puede prometer RPO cero estricto.
Backup y restore	`backup-db.sh` / `restore-db.sh`.	Proporciona recuperabilidad, pero no equivale a HA online.

SLA recomendado

Objeto de servicio	Declaración recomendada	Restricciones
Entrada HTTP de plataforma	Comienza desde disponibilidad mensual de `99.5%`.	El failover de VIP no es lo mismo que migración automática del nodo de aplicación activo.
Acceso a base de datos	Objetivo operativo: `RTO <= 2 minutes`.	El RPO depende del retraso de replicación. Pruebas históricas de failover en la misma arquitectura fueron de unos 59-64 segundos.
MQTT / OPC UA	El diseño actual no promete SLA de HA.	Primero se debe añadir proxy TCP con HAProxy, un VIP separado o clustering del servicio.
Backup y restauración	Backups diarios; backup obligatorio antes de cambios mayores.	El tiempo de restauración depende del volumen de datos y del I/O en sitio.

Antes de declarar `99.9%` o más, completa la migración automática de la aplicación o el despliegue multi-réplica, la HA de entradas de protocolo, las alertas de monitoreo y los simulacros de recuperación. No describas capacidades planificadas como capacidades ya entregadas.

Criterios de aceptación

Cada despliegue o actualización debe cumplir al menos lo siguiente:

- `verify-materials.sh` pasa y las comprobaciones SHA256 del paquete de entrega pasan.
- Los tres nodos Swarm están en estado `Ready`.
- `APP VIP /healthz` y `/readyz` devuelven `200`, y la página de inicio y `/uns` son accesibles.
- Los siete servicios compose del nodo activo están en ejecución.
- `DB VIP:5432` es accesible y la topología primary, standby y witness es normal.
- La cuenta predeterminada puede iniciar sesión realmente. No evalúes la inicialización de IAM solo por puertos, contenedores o respuestas HTTP 200.
- Pasan las pruebas smoke de HTTP, puertos EMQX, WebSocket/WSS y OPC UA TCP.

Inspección y alertas

Ejecuta lo siguiente todos los días:

```
./bin/verify.sh --site config/site.conf
./bin/smoke-business-emqx.sh --site config/site.conf
```

Configura al menos las siguientes alertas:

- Fallo de APP VIP `/healthz` o `/readyz`.
- DB VIP inaccesible, sin primary, riesgo de split-brain, interrupción de replication o lag por encima del umbral.
- Estado anormal de keepalived, HAProxy, PostgreSQL o repmgrd.
- Contenedores clave como `backend`, `redis` y `EMQX` salen o se reinician con frecuencia.
- El uso del disco de datos supera `80% / 90%`.
- El backup más reciente falló o no se generó.

Requisitos de simulacro

Simulacro	Frecuencia	Criterios de verificación
Failover de APP VIP	Antes de la entrega y trimestralmente.	Tras detener keepalived en el nodo VIP actual, `/healthz` y `/readyz` se recuperan.
Fallo de DB primary	Antes de la entrega y cada seis meses.	El standby se promociona, DB VIP se recupera y el antiguo primary se reconstruye como standby.
Restauración de base de datos	Muestra mensual y antes de cambios mayores.	El backup es restaurable. Después de restaurar, pasan `verify`, smoke checks y verificación de login.
Verificación de materiales	Cada build de paquete.	Pasan los materiales fuente, SHA del paquete y verificación de desempaquetado.

Principios de gestión de incidentes

1. Confirma primero el alcance del impacto: HTTP, MQTT, OPC UA, base de datos e impacto de nodo único.
2. Prioriza restaurar VIP, HAProxy, la aplicación activa y el database primary.
3. Conserva logs, directorios de datos y backups. No limpies ni sobrescribas directamente el sitio del incidente.
4. Para incidentes de base de datos, determina primero el primary actual y luego decide si hacer failover, reincorporar o restaurar.
5. Después de la recuperación, ejecuta `verify`, smoke checks y verificación real de login.

Elementos de refuerzo para SLA alto

- Convierte la aplicación en multi-replica o establece un proceso verificable de migración automática del nodo activo.
- Crea un cluster EMQX o enrútalo mediante HAProxy TCP / TLS con un VIP dedicado.
- Proporciona una entrada de alta disponibilidad para OPC UA mediante HAProxy TCP, un VIP dedicado o clustering del servicio.
- Usa replication síncrona de PostgreSQL o define claramente el lag de replication y RPO aceptables.
- Automatiza la programación de backups, replication fuera del sitio y simulacros regulares de restore.
- Conecta contenedores, hosts, bases de datos, logs y probes black-box a alertas unificadas.

Procedimientos relacionados: **Runbook de operaciones** y **Lista estándar de puertos**.

REFERENCIA

Lista estándar de puertos

Lista de puertos de firewall para el despliegue offline estándar de Tier0 Enterprise con seis VM.

La siguiente tabla se basa en el despliegue offline Enterprise New de seis VM. Los firewalls del cliente deben permitir tráfico solo desde los rangos de origen mínimos necesarios. No expongas estos puertos directamente a internet público.

Origen	Destino	Puerto	Protocolo	Propósito
Host de control	Seis VM destino	22	TCP	SSH / Ansible
Tres nodos de aplicación, entre sí	Tres nodos de aplicación	2377	TCP	Plano de gestión de Docker Swarm
Tres nodos de aplicación, entre sí	Tres nodos de aplicación	7946	TCP/UDP	Descubrimiento de nodos Swarm
Tres nodos de aplicación, entre sí	Tres nodos de aplicación	4789	UDP	Overlay de Swarm
Tres nodos de aplicación	Nodo de aplicación activo	8088	TCP	HAProxy hacia el backend Enterprise New; solo segmento de aplicación
Segmento de red de negocio	APP VIP	80	TCP	Entrada HTTP de plataforma, `/healthz`, `/readyz`
Segmento de red de	APP VIP / nodo de	1883	TCP	MQTT

Origen	Destino	Puerto	Protocolo	Propósito
negocio	aplicación activo			
Segmento de red de negocio	APP VIP / nodo de aplicación activo	8883	TCP	MQTT TLS
Segmento de red de negocio	APP VIP / nodo de aplicación activo	8083	TCP	MQTT WebSocket
Segmento de red de negocio	APP VIP / nodo de aplicación activo	8084	TCP	MQTT WebSocket TLS
Segmento de red de negocio	APP VIP / nodo de aplicación activo	4840	TCP	OPC UA Server
Segmento de aplicación controlado	Nodo de aplicación activo	8089	TCP	Launchpad
Segmento de acceso de aplicaciones dinámicas	Nodo de aplicación activo	30000-30199	TCP	App Gateway / puertos host de aplicaciones dinámicas
Nodos de aplicación, nodos DB y Witness	DB VIP / nodos DB	5432	TCP	PostgreSQL / TimescaleDB

Origen	Destino	Puerto	Protocolo	Propósito
DB1, DB2 y Witness	DB1, DB2 y Witness	5432	TCP	Streaming replication / repmgr
Nodos de aplicación	Servidor NFS	2049	TCP/UDP	Se usa cuando `NFS_ENABLED=true`

El despliegue compose actual no expone Redis, SourceFlow, EventFlow, Marimo ni EMQX Dashboard al exterior. Estos servicios solo son llamados por el backend dentro de la red de runtime de Docker.

REFERENCIA

Runbook de operaciones

Procedimientos de inspección diaria, failover, backup, restore, limpieza y troubleshooting para el despliegue estándar de Tier0 Enterprise.

Este documento se basa en la aplicación offline Enterprise New.

Inspección diaria

1. Ejecuta los siguientes comandos desde el directorio raíz del paquete de despliegue:

```
./bin/verify.sh --site config/site.conf
./bin/smoke-business-emqx.sh --site config/site.conf
```

2. Confirma los siguientes elementos:
   - Los 3 nodos Swarm están en estado `Ready`.
   - `APP VIP /healthz` y `/readyz` devuelven `200`.
   - `DB VIP:5432` es accesible.
   - La topología PostgreSQL primary, standby y witness es normal.
   - `backend`, `redis`, `emqx`, `sourceflow`, `eventflow`, `marimo` y `opcua-server` se ejecutan en el nodo de aplicación activo.

Comprobaciones de aplicación

1. Ejecuta los siguientes comandos en el nodo de aplicación activo:

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh ps
bash bin/compose.sh logs --tail=200 backend
curl -fsS http://127.0.0.1:8088/healthz
curl -fsS http://127.0.0.1:8088/readyz
```

> **Nota**

No omitas `bin/compose.sh` ni ejecutes comandos `docker compose` sin envolver directamente. El script wrapper carga el entorno de runtime generado por el instalador.

2. Interpretación de health-check:
   - Fallo de `healthz`: revisa primero el proceso backend, la imagen y los logs del contenedor.
   - Fallo de `readyz`: revisa también DB VIP, Redis y logs de migración de base de datos.
   - Página inicial accesible pero login no disponible: confirma `ADMIN_INITIAL_PASSWORD` en `.env`, luego revisa los logs de migración IAM del backend y las tablas de usuarios. No juzgues el éxito del despliegue solo por el puerto HTTP.

Comprobaciones de entrada de protocolo

```
nc -vz <APP_VIP> 1883
nc -vz <APP_VIP> 8883
nc -vz <APP_VIP> 8083
nc -vz <APP_VIP> 8084
nc -vz <APP_VIP> 4840
```

> ⓘ **Nota**
> EMQX y OPC UA Server están fijados al nodo de aplicación activo. El failover de APP VIP solo garantiza la entrada HTTP `APP VIP:80`. Los puertos de protocolo no se enrutan mediante proxy TCP de HAProxy, por lo que este diseño no promete alta disponibilidad para entradas de protocolo.

Comprobaciones de base de datos

Ejecuta lo siguiente en los nodos de base de datos:

```
systemctl status tier0-postgresql
systemctl status tier0-repmgrd
sudo -iu postgres psql -Atqc 'select pg_is_in_recovery();'
```

```
sudo -iu postgres /usr/local/bin/repmgr -f /etc/repmgr.conf cluster show
```

- `pg_is_in_recovery=false` significa primary.
- `pg_is_in_recovery=true` significa standby.
- DB VIP solo debe ubicarse en el primary actual.
- Si la replicación standby se interrumpe o el witness reporta errores upstream, preserva primero el sitio. No vuelvas a ejecutar directamente el despliegue completo.

Failover de APP VIP

1. Antes de probar, registra el nodo que posee actualmente el VIP y verifica:

```
curl -fsS http://<APP_VIP>/healthz
curl -fsS http://<APP_VIP>/readyz
```

2. Después de detener `keepalived` en el nodo VIP actual, confirma que otro nodo de aplicación tome el VIP y vuelve a probar ambas URL. El backend de HAProxy apunta al nodo de aplicación activo en `8088/TCP`, por lo que los tres nodos de aplicación deben poder alcanzar el puerto 8088 del nodo activo.
3. Después de restaurar `keepalived`, ejecuta de nuevo `verify.sh` y smoke checks. No detengas contenedores de aplicación activos durante el simulacro.

Conmutación por fallo del DB Primary

Antes de un simulacro de fallo, crea un backup y confirma que la replication está sana:

```
./bin/backup-db.sh --site config/site.conf
```

Después de detener `tier0-postgresql` en el primary actual, observa cómo `repmgr` promueve el standby y cómo DB VIP toma control. El RTO objetivo está dentro de 2 minutos. Pruebas históricas en la misma arquitectura fueron de unos 59-64 segundos.

> ⚠ **Precaución**

- El primary anterior no debe iniciarse directamente de vuelta en la topología de producción. Primero detén PostgreSQL, repmgrd y keepalived; elimina cualquier DB VIP restante; después ejecuta basebackup desde el nuevo primary actual, registra el nodo como standby y finalmente reconecta el witness.
- Después del failover, no vuelvas a ejecutar directamente el instalador completo. Las definiciones estáticas de primary/standby en el inventory pueden alterar la topología actual.

Backup y restore

- Backup:

```
./bin/backup-db.sh --site config/site.conf
```

- Restore:

```
./bin/restore-db.sh --site config/site.conf --restore-file /backup/tier0-db/postgres-YYYYmmddTHHMMSS.dump
```

> ⓘ **Nota**
> Antes de restaurar, confirma el primary actual. Después de restaurar, ejecuta `verify.sh`, smoke checks y verificación real de login en la plataforma para la ruta de usuario predeterminado.

Limpieza y nueva prueba

```
./full-cleanup.sh --env install.env --yes
./install.sh --auto --yes
```

> ⚠ **Precaución**

La limpieza completa elimina los datos de runtime de Tier0, los datos de Docker/Swarm y las copias de seguridad de base de datos en los 6 nodos. También realiza una limpieza final en el nodo de control después de que sale el contenedor de control. Usa esta entrada solo en entornos de prueba confirmados como descartables. Si se deben conservar datos del cliente o un sitio de incidente, usa el `--cleanup` normal y completa primero una copia de seguridad.

Problemas comunes

Fallo de arranque de Backend

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=300 backend
bash bin/compose.sh ps
```

> **Nota**
> Concéntrate en DB URL, DB VIP, Redis, migration y permisos de directorios montados.

Fallo de arranque de EMQX

```
cd /opt/tier0-enterprise/tier0-deploy
bash bin/compose.sh logs --tail=200 emqx
```

> **Nota**
> Comprueba certificados, archivos de configuración, permisos de volumen y si los puertos `1883/8883/8083/8084` están ocupados.

Reinicios repetidos de repmgrd

```
systemctl status tier0-repmgrd
tail -100 /var/log/repmgr/repmgr.log
```

```
sudo -iu postgres psql -Atqc 'show shared_preload_libraries;'
```

> **Nota**
> Confirma que `shared_preload_libraries` incluye `repmgr` y que la unidad systemd usa `repmgrd --daemonize=false`.

Fallo de verificación de materiales

```
./bin/verify-materials.sh
```

> **Precaución**
> No omitas la verificación SHA256 para continuar el despliegue. Vuelve a obtener el paquete de entrega o reconstruye los materiales siguiendo el proceso trazable.

REFERENCIA

Apéndice de referencia de OpenAPI

Tier0 OpenAPI v1 · Operaciones: 36 · Esquemas: 85

Operaciones

Los nombres de campos, ejemplos y descripciones de origen de la API siguen la especificación OpenAPI en inglés incluida.

UNS

Leer, escribir y gestionar datos de Unified Namespace.

POST `/uns/attachments`

Upload a UNS attachment
Uploads a multipart attachment for a UNS node. Supply a file and at least one of unsId or topic; a valid unsId takes precedence.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments' \
  --header 'x-api-key: <api-key>'
```

`multipart/form-data`

Campo	Tipo	Obligatorio	Descripción
`unsId`	`integer (int64)`	No	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
`topic`	`string`	No	Full owner node path, used when a valid unsId is not supplied.
`fileName`	`string`	No	Original file name.
`sha256`	`string`	No	SHA-256 checksum of the file contents.
`file`	`string (binary)`	Sí	Binary file contents. The multipart field name must be file.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

`200 · application/json`


```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "fileName": "string",
    "filePath": "string",
    "fileUrl": "string",
    "contentType": "string"
  }
}
```


Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	UnsAttachmentUploadResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/attachments/list

List UNS attachments
Lists attachments for a UNS node or topic. Temporary file URLs can expire and must then be requested again.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/attachments/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"topic":"string","page":0,"size":0,"includeFileUrl":false}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
unsId	integer (int64)	No	Numeric UNS node ID.
topic	string	No	Full UNS topic path.
page	integer (int64)	No	Page number, starting at 1.
size	integer (int64)	No	Number of attachments per page.
includeFileUrl	boolean (boolean)	No	Whether to return temporary file URLs. Defaults to true.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "unsId": 0,
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  }
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
msg	string	No	Response message
data	object	No	UnsAttachmentListResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/browse

Browse the UNS tree
Returns a UNS subtree. Use an empty path for the workspace root.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/browse' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string","max_depth":0,"include_metadata":false,"include_leaf_value":false}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
path	string	No	Root UNS path. Empty means the workspace root.
max_depth	integer (int64)	No	Maximum tree depth. Omit or use 0 for the server default.

Campo	Tipo	Obligatorio	Descripción
include_metadata	boolean (boolean)	No	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	No	Whether returned leaf nodes should include their latest payload.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "tree": [
      "value"
    ]
  }
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
msg	string	No	Response message
data	object	No	BrowseResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–

Estado	Significado	Tipo de medio
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/create

Create UNS nodes

Creates UNS nodes and nested subtrees. Check each item result and use the returned topic in subsequent calls.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"namespace":
[{"name":"value","type":"value","children":"value","topicType":"value","fields":"value"}]}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
namespace	array<object>	Sí	UNS nodes to create. Nest children to create a subtree.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
msg	string	No	Response message
data	object	No	CreateResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/delete

Delete UNS nodes

Deletes the listed nodes sequentially. By default this is a soft delete; the batch is not rolled back when a later item fails.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"hard_delete":false}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
topics	array<string>	Sí	UNS topic or path node paths to delete.

Campo	Tipo	Obligatorio	Descripción
hard_delete	boolean (boolean)	No	Permanently delete instead of moving nodes to the recycle state.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
msg	string	No	Response message
data	object	No	OpenapiEmptyResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/history

Query UNS history
Queries historical topic values over an inclusive RFC 3339 time range. Results can be aggregated or sampled.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/history' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"start_time":"string","end_time":"string","aggregation":
{"interval":"string","function":"string","field":"string","fields":["value"]},"page":0}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
topics	array<string>	Sí	UNS topic paths whose history should be queried.
start_time	string	Sí	Inclusive query start in RFC 3339 format.
end_time	string	Sí	Inclusive query end in RFC 3339 format.
aggregation	object	No	HistoryAggregation. Consulta el esquema referenciado en la sección Esquemas.
page	integer (int64)	No	Page number, starting from 1.
size	integer (int64)	No	Page size; values above the server maximum are capped.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```


Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	HistoryResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/uns/read`

Read UNS values

Returns value, quality, and timeStamp (VQT) for each requested topic. A failed topic does not discard successful items in the same batch.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/read' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"topics":["string"],"include_metadata":false,"include_leaf_value":false}'
```

`application/json`

Campo	Tipo	Obligatorio	Descripción
`topics`	`array<string>`	Sí	UNS topic paths to read.
`include_metadata`	`boolean (boolean)`	No	Whether each result should include UNS node metadata.
`include_leaf_value`	`boolean (boolean)`	No	Whether leaf nodes should include their latest payload.

Respuestas

`200` `400` `401` `403` `404` `409` `413` `500`

Ejemplo de respuesta correcta

`200 · application/json`

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	ReadResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/restore

Restore a deleted UNS node

Restores a soft-deleted UNS node. Permanently deleted nodes cannot be restored.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/restore' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"path":"string"}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
path	string	Sí	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```


Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	OpenapiEmptyResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/uns/search`

Search UNS nodes

Searches UNS node names, aliases, and paths, with optional type and path filters.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/search' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"keyword":"string","topicType":"string","path_prefix":"string","include_metadata":false,"include_
leaf_value":false}'
```

`application/json`

Campo	Tipo	Obligatorio	Descripción
`keyword`	`string`	No	Keyword matched against UNS node names, aliases, and paths.
`topicType`	`string`	No	Topic data category filter: STATE, ACTION, or METRIC.
`path_prefix`	`string`	No	Only return nodes under this UNS path prefix.
`include_metadata`	`boolean (boolean)`	No	Whether returned nodes should include metadata.
`include_leaf_value`	`boolean (boolean)`	No	Whether returned leaf nodes should include their latest payload.
`page`	`integer (int64)`	No	Page number, starting from 1.
`size`	`integer (int64)`	No	Page size.

Respuestas

`200` `400` `401` `403` `404` `409` `413` `500`

Ejemplo de respuesta correcta

`200 · application/json`

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "objects": [
      "value"
    ],
    "page": 0,
    "size": 0,
    "total": 0
  }
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
msg	string	No	Response message
data	object	No	SearchResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/unsBindFlow

Bind a flow to a UNS node
Binds a flow ID to a UNS node ID.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/unsBindFlow' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"unsId":0,"flowId":0}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
unsId	integer (int64)	Sí	UNS node ID to bind.
flowId	integer (int64)	Sí	Flow ID to bind.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json


```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "unsId": 0,
    "flowId": 0
  }
}
```


Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
msg	string	No	Response message
data	object	No	UnsBindFlowResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /uns/update

Update a UNS node
Updates an existing UNS node. Empty strings, an empty fields array, and an empty extendProperties object retain existing values.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"path":"string","name":"string","description":"string","displayName":"string","extendProperties":
{}}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
path	string	Sí	Existing UNS topic or path node to update.
name	string	No	New node name.
description	string	No	Updated node description.
displayName	string	No	Updated display name.
extendProperties	object	No	Updated custom metadata key-value pairs.
fields	array<object>	No	Updated payload schema fields.
alias	string	No	Updated topic alias.
enableHistory	boolean (boolean)	No	Supply true or false to change history storage. Omit to keep the existing setting.
updateMask	array<string>	No	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false
  }
}
```


Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	OpenapiEmptyResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/uns/write`

Write UNS values

Writes values to existing TOPIC nodes; wildcards are not supported. `value` must match the node payload schema. **Batch results:** check each `data.results` item, not only HTTP 200. **Retry:** confirm the outcome before retrying a write.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/uns/write' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"writes":
[{"topic":"value","value":"value","timeStamp":"value"}],"qos":0,"retain":false}'
```

`application/json`

Campo	Tipo	Obligatorio	Descripción
`writes`	`array<object>`	Sí	Write operations to apply.
`qos`	`integer (int64)`	No	MQTT QoS: 0, 1, or 2.
`retain`	`boolean (boolean)`	No	Whether the MQTT message should be retained.

Respuestas

`200` `400` `401` `403` `404` `409` `413` `500`

Ejemplo de respuesta correcta

`200 · application/json`

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "success": false,
    "results": [
      "value"
    ]
  }
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	WriteResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	Inspect the success flag of each `data.results` item. HTTP 200 alone does not confirm that every write succeeded.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

Archivos

Subir, descargar y gestionar archivos almacenados.

POST /assets/files

Create a direct-upload form

Issues an object-store upload form; it does not transfer file contents. Submit every returned post field unchanged with the file.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data
'{"fileName":"string","contentType":"string","size":0,"business":"string","useBy":"string"}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
fileName	string	Sí	Original file name.
contentType	string	No	MIME content type.

Campo	Tipo	Obligatorio	Descripción
`size`	`integer (int64)`	Sí	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
`business`	`string`	No	Business domain or feature associated with the file.
`useBy`	`string`	No	Optional usage category: user, workspace, or platform.
`visibility`	`string`	No	File visibility, such as private or public.
`appInstanceId`	`string`	No	Associated application instance ID.
`sessionId`	`string`	No	Caller session ID.

Respuestas

`200` `400` `401` `403` `404` `409` `413` `500`

Ejemplo de respuesta correcta

`200 · application/json`


```
{
  "code": 0,
  "data": {
    "fileId": 0,
    "filePath": "string",
    "fileUrl": "string",
    "uploadUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```


Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileUploadResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`

Estado	Significado	Tipo de medio
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/delete

Delete a file

Deletes a file identified by its server-issued storage path.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"filePath":"string"}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
filePath	string	Sí	Server-issued storage path returned by an upload API.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "data": {
    "deleted": false
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileDeleteResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

`GET` `/assets/files/download`

Download a private file
Returns file bytes directly. Do not forward x-api-key or other credentials when a client follows a redirect to a cross-origin storage URL.

Parámetros de solicitud

Nombre	Ubicación	Tipo	Obligatorio	Descripción
`filePath`	query	`string`	Sí	Server-issued storage path returned by an upload API.
`responseContentDisposition`	query	`string`	No	Optional Content-Disposition value for the download response.

Respuestas

302 400 401 403 404 409 413 500

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
302	Redirect to the final download URL.	—
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	—
409	Resource state conflict. Verify paths, IDs, and current resource state.	—
413	Request too large. Reduce the request before retrying.	—
500	Service error. Investigate the service and confirm the outcome before retrying.	—

POST /assets/files/multipart/abort

Abort a multipart upload

Cancels a multipart upload session.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/abort' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string"}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
fileKey	string	Sí	Object-store key used for the multipart upload.
uploadId	string	Sí	Object-store multipart upload session ID.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "aborted": 1
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileMultipartAbortResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/assets/files/multipart/complete`

Complete a multipart upload
Completes a multipart upload after every uploaded part number and ETag have been supplied.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/complete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","parts":[{"partNumber":"value","etag":"value"}]}'
```

`application/json`

Campo	Tipo	Obligatorio	Descripción
`fileKey`	`string`	Sí	Object-store key used for the multipart upload.
`uploadId`	`string`	Sí	Object-store multipart upload session ID.
`parts`	`array<object>`	Sí	Uploaded part numbers and their ETags.

Respuestas

`200` `400` `401` `403` `404` `409` `413` `500`

Ejemplo de respuesta correcta

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "filePath": "string",
    "fileUrl": "string",
    "sizeBytes": 0,
    "expiresAt": 0
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileMultipartCompleteResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/init

Initialize a multipart upload

Creates a multipart upload session and returns its ID, part size, and part count.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/init' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","contentType":"string","size":0,"partSize":0,"business":"string"}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
fileName	string	Sí	Original file name.
contentType	string	No	File MIME type, for example application/octet-stream.
size	integer (int64)	Sí	Total file size in bytes, greater than 0 and at most 1 GiB.
partSize	integer (int64)	No	Preferred part size in bytes. The server may normalize it.
business	string	No	Business domain or feature associated with the file.
useBy	string	No	Optional usage category: user, workspace, or platform.
visibility	string	No	File visibility: private or public.

Campo	Tipo	Obligatorio	Descripción
appInstanceId	string	No	Associated application instance ID.
sessionId	string	No	Caller session ID.
bizId	string	No	Associated business identifier.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileKey": "string",
    "filePath": "string",
    "uploadId": "string",
    "partSize": 0,
    "partCount": 0
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
data	object	No	AssetFileMultipartInitResp. Consulta el esquema referenciado en la sección Esquemas.
msg	string	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain

Estado	Significado	Tipo de medio
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /assets/files/multipart/part-urls

Create multipart upload URLs
Returns presigned PUT URLs for the requested one-based part numbers.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/files/multipart/part-urls' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileKey":"string","uploadId":"string","partNumbers":[0]}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
fileKey	string	Sí	Object-store key used for the multipart upload.
uploadId	string	Sí	Object-store multipart upload session ID.
partNumbers	array<integer (int32)>	Sí	One-based part numbers to sign.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "data": {
    "partUrls": [
      "value"
    ]
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileMultipartPartUrlsResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

`GET` `/assets/files/public`

Download a public file

Downloads bytes for a file explicitly marked public. An unguessable path is not authorization; do not mark sensitive files public.

Parámetros de solicitud

Nombre	Ubicación	Tipo	Obligatorio	Descripción
`filePath`	query	`string`	Sí	Server-issued storage path returned by an upload API.
`responseContentDisposition`	query	`string`	No	Optional Content-Disposition value for the download response.

Respuestas

200 400 404 409 413 500

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	Returns file bytes directly. Content-Type depends on the file type; no JSON envelope is used.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

GET /assets/files/url

Create a temporary file URL
Creates a temporary URL for a file.

Parámetros de solicitud

Nombre	Ubicación	Tipo	Obligatorio	Descripción
filePath	query	string	Sí	Server-issued storage path returned by an upload API.
expiredSec	query	integer (int64)	No	Requested temporary URL lifetime in seconds.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "data": {
    "fileUrl": "string",
    "expiresAt": 0
  },
  "msg": "string"
}
```


Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	AssetFileUrlResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/assets/upload`

Create a legacy asset upload URL
Legacy compatibility endpoint. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/assets/upload' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"fileName":"string","sizeBytes":0,"contentType":"string","size":0,"sha256":"string"}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
fileName	string	Sí	Original file name.
sizeBytes	integer (int64)	Sí	File size in bytes.
contentType	string	No	File MIME type, for example application/octet-stream.
size	integer (int64)	No	File size in bytes.
sha256	string	No	Optional SHA-256 checksum of the file content.
storageKey	string	No	Optional object-store key. Omit it to let the server generate one.
isPublic	boolean (boolean)	No	Whether the file should be accessible through the public download endpoint.
business	string	No	Business domain or feature that owns the uploaded file.
useBy	string	No	Usage scenario or owner category for the uploaded file.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "msg": "string",
  "data": {
    "uploadUrl": "string",
    "filePath": "string",
    "fileUrl": "string"
  }
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`msg`	`string`	No	Response message
`data`	`object`	No	AssetUploadResp. Consulta el esquema referenciado en la sección Esquemas.

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	Legacy upload response. The current Enterprise implementation returns an empty uploadUrl; use /assets/files for new integrations.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

Autenticación

Verificar la identidad y los permisos actuales de OpenAPI.

POST `/auth/whoami`

Inspect the current API key identity

Returns the current API-key identity and its effective permissions. Call this before integration work to verify access.

Respuestas

`200` `400` `401` `403` `404` `409` `413` `500`

Ejemplo de respuesta correcta

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "userID": 0,
    "userName": "string",
    "email": "string",
    "workspaceID": 0,
    "workspaceName": "string"
  },
  "msg": "string"
}
```


Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	AuthWhoamiResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

Flow

Gestionar las definiciones de Source Flow y Event Flow.

POST /flow/create

Create a flow
Creates a SourceFlow or EventFlow and returns its numeric resource ID.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowName":"string","flowType":"string","description":"string","template":"string"}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
flowName	string	Sí	Flow name, up to 63 Unicode characters.
flowType	string	Sí	Flow category: SourceFlow or EventFlow.
description	string	No	Flow description.
template	string	No	Optional flow template JSON.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "brokerID": "string"
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
data	object	No	FlowCreateResp. Consulta el esquema referenciado en la sección Esquemas.

Campo	Tipo	Obligatorio	Descripción
msg	string	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/delete

Delete flows

Deletes the listed flows sequentially; earlier deletions are not rolled back when a later item fails.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"ids":[0]}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
ids	array<integer (int64)>	Sí	Flow IDs to delete.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	FlowEmptyResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/flow/deploy`

Deploy a flow
Deploys a complete Node-RED canvas JSON string to a flow.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/deploy' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowsJson":"string"}'
```

`application/json`

Campo	Tipo	Obligatorio	Descripción
`id`	`integer (int64)`	Sí	Flow ID to deploy.
`flowsJson`	`string`	Sí	Node-RED canvas JSON to deploy.

Respuestas

`200` `400` `401` `403` `404` `409` `413` `500`

Ejemplo de respuesta correcta

`200 · application/json`

```
{
  "code": 0,
  "data": {
    "flowId": "string"
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	FlowDeployResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`

Estado	Significado	Tipo de medio
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/flowdata

Get flow canvas data
Returns the Node-RED canvas revision and node objects for a flow.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/flowdata' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
id	integer (int64)	Sí	Flow ID.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "data": {
    "rev": "string",
    "flows": [
      "value"
    ]
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	FlowDataResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/flow/get`

Get a flow
Returns flow metadata for a numeric flow ID.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/get' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

`application/json`

Campo	Tipo	Obligatorio	Descripción
`id`	`integer (int64)`	Sí	Flow ID.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json


```
{
  "code": 0,
  "data": {
    "id": 0,
    "flowId": "string",
    "flowName": "string",
    "flowType": "string",
    "flowStatus": "string"
  },
  "msg": "string"
}
```


Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
data	object	No	FlowInfo. Consulta el esquema referenciado en la sección Esquemas.
msg	string	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/list

List flows
Lists flows, optionally filtered by a keyword or flow category.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/list' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","flowType":"string"}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
keyword	string	No	Keyword matched against flow names and descriptions.
flowType	string	No	Flow category filter: SourceFlow or EventFlow.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 200,
  "msg": "success",
  "data": {
    "list": [
      {
        "id": 101,
        "flowId": "flow-101",
        "flowName": "Line 1 collector",
        "flowType": "source",
        "flowStatus": "draft",
        "template": "[]",
        "description": "Collects line telemetry",
        "isFavorite": 2,
        "currentVersionId": 0,
        "currentVersionName": "",
        "currentVersionType": "",
        "createdTime": 1788134400000,
        "updatedTime": 1788134400000
      }
    ]
  }
}
```

Campos de la respuesta correcta

`200 · application/json`

Campo	Tipo	Obligatorio	Descripción
`code`	`integer`	No	Response code
`data`	`object`	No	FlowListResp. Consulta el esquema referenciado en la sección Esquemas.
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

POST `/flow/nodes`

List available flow node types
Returns the Node-RED node sets available for a flow category.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/nodes' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"flowType":"string"}'
```

`application/json`

Campo	Tipo	Obligatorio	Descripción
`flowType`	`string`	Sí	Flow category whose Node-RED node catalog should be returned.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json


```
{
  "code": 0,
  "data": {
    "nodes": [
      "value"
    ]
  },
  "msg": "string"
}
```


Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
data	object	No	FlowNodesResp. Consulta el esquema referenciado en la sección Esquemas.
msg	string	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /flow/update

Update a flow
Updates flow metadata and settings for a numeric flow ID.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/flow/update' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0,"flowName":"string","description":"string","template":"string","isFavorite":0}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
id	integer (int64)	Sí	Flow ID to update.
flowName	string	No	Updated flow name.
description	string	No	Updated flow description.
template	string	No	Updated flow template JSON.
isFavorite	integer (int64)	No	Favorite state: 1 for favorite, 2 for not favorite.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
data	object	No	FlowEmptyResp. Consulta el esquema referenciado en la sección Esquemas.

Campo	Tipo	Obligatorio	Descripción
`msg`	`string`	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
`200`	A successful response.	`application/json`
`400`	Correct the request parameters or Content-Type before retrying.	`application/json`
`401`	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	`text/plain`
`403`	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	`text/plain`
`404`	Resource not found. Verify paths, IDs, and current resource state.	–
`409`	Resource state conflict. Verify paths, IDs, and current resource state.	–
`413`	Request too large. Reduce the request before retrying.	–
`500`	Service error. Investigate the service and confirm the outcome before retrying.	–

Plataforma y proyectos

Leer información de la plataforma y gestionar proyectos.

`POST` **`/info`**

Get service information
Returns the service name, version, reported capabilities, and configured MQTT broker.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/info' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{}'
```

`application/json`

Este cuerpo de solicitud no tiene campos de objeto declarados en OpenAPI.

Respuestas

`200` `400` `401` `403` `404` `409` `413` `500`

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	–
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /launchpad/{projectName}/getMembers

List project members

Lists Launchpad project members. URL-encode projectName when it contains special characters.

Parámetros de solicitud

Nombre	Ubicación	Tipo	Obligatorio	Descripción
projectName	path	string	Sí	Project name in the Launchpad URL. URL-encode special characters.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/launchpad/{projectName}/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"roleKey":"string","roles":
["string"],"updatedAtStart":"string","updatedAtEnd":"string","page":0}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
roleKey	string	No	Role key filter.
roles	array<string>	No	Role filters or the returned role list, depending on the model.

Campo	Tipo	Obligatorio	Descripción
updatedAtStart	string	No	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	No	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	No	Page number, starting at 1.
size	integer (int64)	No	Number of members per page.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json


```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```


Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
data	object	No	LaunchpadMembersResp. Consulta el esquema referenciado en la sección Esquemas.
msg	string	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json

Estado	Significado	Tipo de medio
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /platform/getMembers

List platform members

Lists platform members with optional keyword, role, status, and update-time filters.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/platform/getMembers' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"keyword":"string","roleKey":"string","roles":["string"],"statuses":
["string"],"updatedAtStart":"string"}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
keyword	string	No	Search keyword.
roleKey	string	No	Role key filter.
roles	array<string>	No	Role filters or the returned role list, depending on the model.
statuses	array<string>	No	Member status filters: active or disabled.
updatedAtStart	string	No	Inclusive update-time lower bound in RFC 3339 format.
updatedAtEnd	string	No	Inclusive update-time upper bound in RFC 3339 format.
page	integer (int64)	No	Page number, starting at 1.
size	integer (int64)	No	Number of members per page.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json


```
{
  "code": 0,
  "data": {
    "list": [
      "value"
    ],
    "total": 0,
    "page": 0,
    "size": 0
  },
  "msg": "string"
}
```


Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
data	object	No	PlatformMembersResp. Consulta el esquema referenciado en la sección Esquemas.
msg	string	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

Autenticación MQTT

Crear y eliminar credenciales MQTT.

POST /mqtt-auth/create

Create an MQTT credential
Creates a normal MQTT credential. The returned password is available only once and must be stored securely immediately.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/create' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"name":"string","description":"string","clientIDRandomSuffixEnabled":false}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
name	string	Sí	MQTT credential name.
description	string	No	Description of the MQTT credential.
clientIDRandomSuffixEnabled	boolean	No	Whether to append a random suffix to the MQTT client ID.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "data": {
    "id": 0,
    "name": "string",
    "clientID": "string",
    "username": "string",
    "password": "string"
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
data	object	No	MqttAuthCreateResp. Consulta el esquema referenciado en la sección Esquemas.
msg	string	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

POST /mqtt-auth/delete

Delete an MQTT credential
Idempotently deletes a normal MQTT credential. System credentials are not deleted.

Cuerpo de la solicitud

```
curl --request POST 'https://<tier0-host>/openapi/v1/mqtt-auth/delete' \
  --header 'x-api-key: <api-key>' \
  --header 'Content-Type: application/json' \
  --data '{"id":0}'
```

application/json

Campo	Tipo	Obligatorio	Descripción
id	integer (int64)	Sí	Resource ID.

Respuestas

200 400 401 403 404 409 413 500

Ejemplo de respuesta correcta

200 · application/json

```
{
  "code": 0,
  "data": {
    "success": false
  },
  "msg": "string"
}
```

Campos de la respuesta correcta

200 · application/json

Campo	Tipo	Obligatorio	Descripción
code	integer	No	Response code
data	object	No	MqttAuthDeleteResp. Consulta el esquema referenciado en la sección Esquemas.
msg	string	No	Response message

Significado de los estados de respuesta

Estado	Significado	Tipo de medio
200	A successful response.	application/json
400	Correct the request parameters or Content-Type before retrying.	application/json
401	Authentication middleware returns plain-text unauthorized. Check API key validity, scope, and owner roles.	text/plain
403	Authentication middleware returns plain-text forbidden. Check API key validity, scope, and owner roles.	text/plain
404	Resource not found. Verify paths, IDs, and current resource state.	–
409	Resource state conflict. Verify paths, IDs, and current resource state.	–
413	Request too large. Reduce the request before retrying.	–
500	Service error. Investigate the service and confirm the outcome before retrying.	–

Esquemas

Los nombres de campos, ejemplos y descripciones de origen de la API siguen la especificación OpenAPI en inglés incluida.

AssetFileDeleteReq

Campo	Tipo	Obligatorio	Descripción
`filePath`	`string`	Sí	Server-issued storage path returned by an upload API.

AssetFileDeleteResp

Campo	Tipo	Obligatorio	Descripción
`deleted`	`boolean`	Sí	Whether the file was deleted.

AssetFileMultipartAbortReq

Campo	Tipo	Obligatorio	Descripción
`fileKey`	`string`	Sí	Object-store key used for the multipart upload.
`uploadId`	`string`	Sí	Object-store multipart upload session ID.

AssetFileMultipartAbortResp

Campo	Tipo	Obligatorio	Descripción
`aborted`	`integer (int32)`	Sí	Abort result: 1 = aborted (including idempotent replay success), 2 = not aborted.

AssetFileMultipartCompleteReq

Campo	Tipo	Obligatorio	Descripción
`fileKey`	`string`	Sí	Object-store key used for the multipart upload.
`uploadId`	`string`	Sí	Object-store multipart upload session ID.
`parts`	`array<object>`	Sí	Uploaded part numbers and their ETags.

AssetFileMultipartCompleteResp

Campo	Tipo	Obligatorio	Descripción
`filePath`	`string`	Sí	Server-issued storage path returned by an upload API.

Campo	Tipo	Obligatorio	Descripción
`fileUrl`	`string`	No	File access URL.
`sizeBytes`	`integer (int64)`	Sí	File size in bytes.
`expiresAt`	`integer (int64)`	No	Expiration time, in Unix milliseconds.

AssetFileMultipartInitReq

Campo	Tipo	Obligatorio	Descripción
`fileName`	`string`	Sí	Original file name.
`contentType`	`string`	No	File MIME type, for example application/octet-stream.
`size`	`integer (int64)`	Sí	Total file size in bytes, greater than 0 and at most 1 GiB.
`partSize`	`integer (int64)`	No	Preferred part size in bytes. The server may normalize it.
`business`	`string`	No	Business domain or feature associated with the file.
`useBy`	`string`	No	Optional usage category: user, workspace, or platform.
`visibility`	`string`	No	File visibility: private or public.
`appInstanceId`	`string`	No	Associated application instance ID.
`sessionId`	`string`	No	Caller session ID.
`bizId`	`string`	No	Associated business identifier.

AssetFileMultipartInitResp

Campo	Tipo	Obligatorio	Descripción
`fileKey`	`string`	Sí	Object-store key used for the multipart upload.
`filePath`	`string`	Sí	Server-issued storage path returned by an upload API.
`uploadId`	`string`	Sí	Object-store multipart upload session ID.
`partSize`	`integer (int64)`	Sí	Part size in bytes; the server may normalize it.
`partCount`	`integer (int64)`	Sí	Total number of upload parts.
`expiresAt`	`integer (int64)`	Sí	Expiration time, in Unix milliseconds.

AssetFileMultipartPart

Campo	Tipo	Obligatorio	Descripción
`partNumber`	`integer (int32)`	Sí	Part number, starting at 1.
`etag`	`string`	Sí	ETag returned after uploading this part.

AssetFileMultipartPartUrl

Campo	Tipo	Obligatorio	Descripción
`partNumber`	`integer (int32)`	Sí	Part number, starting at 1.
`url`	`string`	Sí	Presigned PUT URL for this part.
`expiresAt`	`integer (int64)`	Sí	Expiration time, in Unix milliseconds.

AssetFileMultipartPartUrlsReq

Campo	Tipo	Obligatorio	Descripción
`fileKey`	`string`	Sí	Object-store key used for the multipart upload.
`uploadId`	`string`	Sí	Object-store multipart upload session ID.
`partNumbers`	`array<integer (int32)>`	Sí	One-based part numbers to sign.

AssetFileMultipartPartUrlsResp

Campo	Tipo	Obligatorio	Descripción
`partUrls`	`array<object>`	Sí	Presigned URLs for the requested parts.

AssetFileUploadReq

Campo	Tipo	Obligatorio	Descripción
`fileName`	`string`	Sí	Original file name.
`contentType`	`string`	No	MIME content type.
`size`	`integer (int64)`	Sí	File size in bytes. Business validation requires a value greater than 0 and at most 1 GiB.
`business`	`string`	No	Business domain or feature associated with the file.
`useBy`	`string`	No	Optional usage category: user, workspace, or platform.
`visibility`	`string`	No	File visibility, such as private or public.

Campo	Tipo	Obligatorio	Descripción
appInstanceId	string	No	Associated application instance ID.
sessionId	string	No	Caller session ID.

AssetFileUploadResp

Campo	Tipo	Obligatorio	Descripción
fileId	integer (int64)	No	File ID.
filePath	string	Sí	Server-issued storage path returned by an upload API.
fileUrl	string	No	File access URL.
uploadUrl	string	Sí	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
expiresAt	integer (int64)	No	Expiration time, in Unix milliseconds.

AssetFileUrlResp

Campo	Tipo	Obligatorio	Descripción
fileUrl	string	Sí	File access URL.
expiresAt	integer (int64)	No	Expiration time, in Unix milliseconds.

AssetUploadReq

AssetUploadReq

Campo	Tipo	Obligatorio	Descripción
fileName	string	Sí	Original file name.
sizeBytes	integer (int64)	Sí	File size in bytes.
contentType	string	No	File MIME type, for example application/octet-stream.
size	integer (int64)	No	File size in bytes.
sha256	string	No	Optional SHA-256 checksum of the file content.
storageKey	string	No	Optional object-store key. Omit it to let the server generate one.
isPublic	boolean (boolean)	No	Whether the file should be accessible through the public download endpoint.

Campo	Tipo	Obligatorio	Descripción
business	string	No	Business domain or feature that owns the uploaded file.
useBy	string	No	Usage scenario or owner category for the uploaded file.

AssetUploadResp

AssetUploadResp

Campo	Tipo	Obligatorio	Descripción
uploadUrl	string	Sí	Upload URL; the legacy Enterprise endpoint currently returns an empty string.
filePath	string	Sí	Server-issued storage path returned by an upload API.
fileUrl	string	No	File access URL.

AuthWhoamiResp

Campo	Tipo	Obligatorio	Descripción
userID	integer (int64)	Sí	Numeric ID of the API key owner.
userName	string	Sí	User name.
email	string	Sí	Email address.
workspaceID	integer (int64)	Sí	Platform workspace ID.
workspaceName	string	Sí	Workspace name; may be empty in Enterprise.
apiKeyName	string	Sí	Name of the current API key.
keyPrefix	string	Sí	API key prefix.
permissions	array<string>	Sí	Effective permissions of the current identity.
roles	array<string>	Sí	Role filters or the returned role list, depending on the model.
keyType	string	Sí	API key ownership type, for example personal.

BrowseReq

BrowseReq

Campo	Tipo	Obligatorio	Descripción
path	string	No	Root UNS path. Empty means the workspace root.
max_depth	integer (int64)	No	Maximum tree depth. Omit or use 0 for the server default.

Campo	Tipo	Obligatorio	Descripción
include_metadata	boolean (boolean)	No	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	No	Whether returned leaf nodes should include their latest payload.

BrowseResp

BrowseResp

Campo	Tipo	Obligatorio	Descripción
tree	array<object>	Sí	UNS node tree.

CreateResp

CreateResp

Campo	Tipo	Obligatorio	Descripción
success	boolean (boolean)	Sí	Whether the operation succeeded.
results	array<object>	Sí	Per-item results. Check success and error for each item.

FlowCreateReq

Campo	Tipo	Obligatorio	Descripción
flowName	string	Sí	Flow name, up to 63 Unicode characters.
flowType	string	Sí	Flow category: SourceFlow or EventFlow.
description	string	No	Flow description.
template	string	No	Optional flow template JSON.

FlowCreateResp

Campo	Tipo	Obligatorio	Descripción
id	integer (int64)	Sí	Resource ID.
brokerID	string	Sí	ID of the MQTT broker configuration node injected when the flow is created.

FlowDataResp

Campo	Tipo	Obligatorio	Descripción
`rev`	`string`	Sí	Node-RED canvas revision; may be an empty string.
`flows`	`array<object>`	Sí	Array of Node-RED canvas node objects.

FlowDeleteReq

Campo	Tipo	Obligatorio	Descripción
`ids`	`array<integer (int64)>`	Sí	Flow IDs to delete.

FlowDeployReq

Campo	Tipo	Obligatorio	Descripción
`id`	`integer (int64)`	Sí	Flow ID to deploy.
`flowsJson`	`string`	Sí	Node-RED canvas JSON to deploy.

FlowDeployResp

Campo	Tipo	Obligatorio	Descripción
`flowId`	`string`	Sí	Flow identifier.

FlowEmptyResp

Campo	Tipo	Obligatorio	Descripción
`success`	`boolean`	Sí	Whether the operation succeeded.

FlowGetReq

Campo	Tipo	Obligatorio	Descripción
`id`	`integer (int64)`	Sí	Flow ID.

FlowInfo

Campo	Tipo	Obligatorio	Descripción
`id`	`integer (int64)`	Sí	Resource ID.
`flowId`	`string`	Sí	Flow identifier.

Campo	Tipo	Obligatorio	Descripción
flowName	string	Sí	Flow name.
flowType	string	Sí	Response flow type: source or event.
flowStatus	string	Sí	Current flow status, for example draft.
template	string	Sí	Flow template JSON, encoded as a string.
description	string	Sí	Description.
isFavorite	integer (int64)	Sí	Favorite state: 1 for favorite, 2 for not favorite.
currentVersionId	integer (int64)	Sí	Current version ID; 0 if unset.
currentVersionName	string	Sí	Current version name; empty if unset.
currentVersionType	string	Sí	Current version type; empty if unset.
createdTime	integer (int64)	Sí	Creation time, in Unix milliseconds.
updatedTime	integer (int64)	Sí	Last update time, in Unix milliseconds.

FlowListReq

Campo	Tipo	Obligatorio	Descripción
keyword	string	No	Keyword matched against flow names and descriptions.
flowType	string	No	Flow category filter: SourceFlow or EventFlow.

FlowListResp

Campo	Tipo	Obligatorio	Descripción
list	array<object>	Sí	Result list.

FlowNodesReq

Campo	Tipo	Obligatorio	Descripción
flowType	string	Sí	Flow category whose Node-RED node catalog should be returned.

FlowNodesResp

Campo	Tipo	Obligatorio	Descripción
nodes	array<object>	Sí	Available Node-RED node sets.

FlowUpdateReq

Campo	Tipo	Obligatorio	Descripción
`id`	`integer (int64)`	Sí	Flow ID to update.
`flowName`	`string`	No	Updated flow name.
`description`	`string`	No	Updated flow description.
`template`	`string`	No	Updated flow template JSON.
`isFavorite`	`integer (int64)`	No	Favorite state: 1 for favorite, 2 for not favorite.

HistoryAggregation

HistoryAggregation

Campo	Tipo	Obligatorio	Descripción
`interval`	`string`	Sí	Positive aggregation window, such as 1m, 5m, 1h, or 1d.
`function`	`string`	No	Aggregation function: avg, min, max, sum, count, first, or last.
`field`	`string`	No	First-level payload field to aggregate.
`fields`	`array<object>`	No	Per-field aggregation settings.

HistoryAggregationField

HistoryAggregationField

Campo	Tipo	Obligatorio	Descripción
`name`	`string`	Sí	Payload field to aggregate.
`function`	`string`	No	Function for this field; defaults to the outer aggregation function.

HistoryQueryMeta

HistoryQueryMeta

Campo	Tipo	Obligatorio	Descripción
`aggregated`	`boolean (boolean)`	Sí	Whether the result was aggregated.
`sampled`	`boolean (boolean)`	Sí	Whether the result was sampled.
`rawTotal`	`integer (int64)`	Sí	Total number of raw records.
`returnedTotal`	`integer (int64)`	Sí	Number of records returned in this response.

Campo	Tipo	Obligatorio	Descripción
requestedIntervalMs	integer (int64)	No	Requested aggregation interval in milliseconds.
actualIntervalMs	integer (int64)	No	Actual aggregation interval in milliseconds.
strategy	string	No	Processing strategy used for the query.

HistoryReq

HistoryReq

Campo	Tipo	Obligatorio	Descripción
topics	array<string>	Sí	UNS topic paths whose history should be queried.
start_time	string	Sí	Inclusive query start in RFC 3339 format.
end_time	string	Sí	Inclusive query end in RFC 3339 format.
aggregation	object	No	HistoryAggregation. Consulta el esquema referenciado en la sección Esquemas.
page	integer (int64)	No	Page number, starting from 1.
size	integer (int64)	No	Page size; values above the server maximum are capped.

HistoryResp

HistoryResp

Campo	Tipo	Obligatorio	Descripción
success	boolean (boolean)	Sí	Whether the operation succeeded.
results	array<object>	Sí	Per-item results. Check success and error for each item.
page	integer (int64)	Sí	Page number, starting at 1.
size	integer (int64)	Sí	File size in bytes.
total	integer (int64)	Sí	Total number of matching records.

InfoReq

```
{
  "type": "object",
  "properties": {}
}
```

LaunchpadGetMembersReq

Campo	Tipo	Obligatorio	Descripción
`roleKey`	`string`	No	Role key filter.
`roles`	`array<string>`	No	Role filters or the returned role list, depending on the model.
`updatedAtStart`	`string`	No	Inclusive update-time lower bound in RFC 3339 format.
`updatedAtEnd`	`string`	No	Inclusive update-time upper bound in RFC 3339 format.
`page`	`integer (int64)`	No	Page number, starting at 1.
`size`	`integer (int64)`	No	Number of members per page.

LaunchpadMemberResp

Campo	Tipo	Obligatorio	Descripción
`memberId`	`string`	Sí	Member identifier.
`userId`	`string`	Sí	User ID.
`userName`	`string`	No	User name.
`email`	`string`	No	Email address.
`roles`	`array<object>`	Sí	Role filters or the returned role list, depending on the model.
`updatedAt`	`string`	No	Last update time.

LaunchpadMemberRoleAppResp

Campo	Tipo	Obligatorio	Descripción
`appId`	`string`	Sí	Application ID, represented as a string.
`appName`	`string`	Sí	Application name.

LaunchpadMemberRoleResp

Campo	Tipo	Obligatorio	Descripción
`roleId`	`string`	Sí	Role ID, represented as a string.
`roleKey`	`string`	Sí	Role key filter.
`roleName`	`string`	No	Role display name.

Campo	Tipo	Obligatorio	Descripción
description	string	No	Description.
memberCount	integer (int64)	Sí	Number of members assigned to the role.
apps	array<object>	No	Applications associated with the role.

LaunchpadMembersResp

Campo	Tipo	Obligatorio	Descripción
list	array<object>	Sí	Result list.
total	integer (int64)	Sí	Total number of matching records.
page	integer (int64)	Sí	Page number, starting at 1.
size	integer (int64)	Sí	File size in bytes.

MqttAuthCreateReq

Campo	Tipo	Obligatorio	Descripción
name	string	Sí	MQTT credential name.
description	string	No	Description of the MQTT credential.
clientIDRandomSuffixEnabled	boolean	No	Whether to append a random suffix to the MQTT client ID.

MqttAuthCreateResp

Campo	Tipo	Obligatorio	Descripción
id	integer (int64)	Sí	Resource ID.
name	string	Sí	Name.
clientID	string	Sí	MQTT client ID.
username	string	Sí	MQTT user name.
password	string	Sí	Plaintext password. Save it now; it cannot be retrieved again.
clientIDRandomSuffixEnabled	boolean	Sí	Whether to append a random suffix to the MQTT client ID.

MqttAuthDeleteReq

Campo	Tipo	Obligatorio	Descripción
id	integer (int64)	Sí	Resource ID.

MqttAuthDeleteResp

Campo	Tipo	Obligatorio	Descripción
success	boolean	Sí	Whether the operation succeeded.

NamespaceNode

NamespaceNode

Campo	Tipo	Obligatorio	Descripción
name	string	Sí	Node name, up to 63 Unicode characters. In a nested request, this is the name under its parent.
type	string	No	Node type: PATH or TOPIC.
children	array<object>	No	Child nodes to create under this node.
topicType	string	No	Topic data category: STATE, ACTION, or METRIC.
fields	array<object>	No	Payload schema fields for TOPIC nodes.
description	string	No	Node description, up to 512 Unicode characters.
displayName	string	No	Display name shown in clients.
extendProperties	object	No	Custom metadata key-value pairs.
alias	string	No	Optional topic alias.
enableHistory	boolean (boolean)	No	Whether history storage is enabled for this topic.

NodeCreateReq

NodeCreateReq

Campo	Tipo	Obligatorio	Descripción
namespace	array<object>	Sí	UNS nodes to create. Nest children to create a subtree.

NodeDeleteReq

NodeDeleteReq

Campo	Tipo	Obligatorio	Descripción
`topics`	`array<string>`	Sí	UNS topic or path node paths to delete.
`hard_delete`	`boolean (boolean)`	No	Permanently delete instead of moving nodes to the recycle state.

NodeRedNodeSet

Campo	Tipo	Obligatorio	Descripción
`id`	`string`	Sí	Resource ID.
`name`	`string`	Sí	Name.
`types`	`array<string>`	Sí	Node-RED node types provided by the node set.
`enabled`	`boolean`	Sí	Whether the node set is enabled.
`module`	`string`	Sí	Node-RED module name.
`version`	`string`	Sí	Service or node module version.

NodeRestoreReq

NodeRestoreReq

Campo	Tipo	Obligatorio	Descripción
`path`	`string`	Sí	Full UNS path of the node to restore. Use its path before deletion, not a node ID or HTTP URL.

NodeUpdateReq

NodeUpdateReq

Campo	Tipo	Obligatorio	Descripción
`path`	`string`	Sí	Existing UNS topic or path node to update.
`name`	`string`	No	New node name.
`description`	`string`	No	Updated node description.
`displayName`	`string`	No	Updated display name.
`extendProperties`	`object`	No	Updated custom metadata key-value pairs.

Campo	Tipo	Obligatorio	Descripción
`fields`	`array<object>`	No	Updated payload schema fields.
`alias`	`string`	No	Updated topic alias.
`enableHistory`	`boolean (boolean)`	No	Supply true or false to change history storage. Omit to keep the existing setting.
`updateMask`	`array<string>`	No	Compatibility field ignored by the current Enterprise implementation. It cannot be used to clear fields.

OpenapiCreateResult

OpenapiCreateResult

Campo	Tipo	Obligatorio	Descripción
`success`	`boolean (boolean)`	Sí	Whether the operation succeeded.
`topic`	`string`	Sí	Full UNS topic path.
`error`	`object`	No	OpenapiItemError. Consulta el esquema referenciado en la sección Esquemas.

OpenapiEmptyResp

OpenapiEmptyResp

Campo	Tipo	Obligatorio	Descripción
`success`	`boolean (boolean)`	Sí	Whether the operation succeeded.

OpenapiHistoryItem

OpenapiHistoryItem

Campo	Tipo	Obligatorio	Descripción
`success`	`boolean (boolean)`	Sí	Whether the operation succeeded.
`topic`	`string`	Sí	Full UNS topic path.
`result`	`object`	No	OpenapiHistoryResult. Consulta el esquema referenciado en la sección Esquemas.
`error`	`object`	No	OpenapiItemError. Consulta el esquema referenciado en la sección Esquemas.

OpenapiHistoryResult

OpenapiHistoryResult

Campo	Tipo	Obligatorio	Descripción
values	array<object>	Sí	Historical VQT records.
meta	object	No	HistoryQueryMeta. Consulta el esquema referenciado en la sección Esquemas.

OpenapiHistoryValue

OpenapiHistoryValue

Campo	Tipo	Obligatorio	Descripción
value	object	No	Payload value.
quality	string	Sí	Data quality: Good, GoodNoData, Uncertain, or Bad.
timeStamp	integer (int64)	Sí	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiItemError

OpenapiItemError

Campo	Tipo	Obligatorio	Descripción
code	integer (int64)	Sí	Status code; in an item error, identifies the failure.
message	string	Sí	Item-level error message.

OpenapiNodeInfo

OpenapiNodeInfo

Campo	Tipo	Obligatorio	Descripción
id	integer (int64)	Sí	Resource ID.
path	string	Sí	Full UNS node path.
name	string	Sí	Name.
type	string	Sí	Node type: PATH or TOPIC.
topicType	string	Sí	Topic category: STATE, ACTION, or METRIC.
description	string	No	Description.

Campo	Tipo	Obligatorio	Descripción
displayName	string	No	Display name.
extendProperties	object	No	Custom node metadata as key-value pairs.
fields	array<object>	No	Payload field definitions.
children	array<object>	No	Child nodes under this node.
alias	string	No	Node alias.
enableHistory	boolean (boolean)	No	Whether history storage is enabled.
bindFlowID	integer (int64)	No	Bound Source Flow ID. 0 means no Source Flow is bound.
bindEventFlowID	integer (int64)	No	Bound Event Flow ID. 0 means no Event Flow is bound.
payload	object	No	OpenapiVQT. Consulta el esquema referenciado en la sección Esquemas.

OpenapiReadItem

OpenapiReadItem

Campo	Tipo	Obligatorio	Descripción
success	boolean (boolean)	Sí	Whether the operation succeeded.
topic	string	Sí	Full UNS topic path.
result	object	No	OpenapiVQT. Consulta el esquema referenciado en la sección Esquemas.
error	object	No	OpenapiItemError. Consulta el esquema referenciado en la sección Esquemas.
metadata	object	No	OpenapiNodeInfo. Consulta el esquema referenciado en la sección Esquemas.

OpenapiVQT

OpenapiVQT

Campo	Tipo	Obligatorio	Descripción
value	object	No	Payload value.
quality	string	Sí	Data quality: Good, GoodNoData, Uncertain, or Bad.

Campo	Tipo	Obligatorio	Descripción
`timeStamp`	`integer (int64)`	Sí	Unix timestamp in milliseconds. Writes use server time when omitted.

OpenapiWriteResult

OpenapiWriteResult

Campo	Tipo	Obligatorio	Descripción
`success`	`boolean (boolean)`	Sí	Whether the operation succeeded.
`topic`	`string`	Sí	Full UNS topic path.
`error`	`object`	No	OpenapiItemError. Consulta el esquema referenciado en la sección Esquemas.

PlatformGetMembersReq

Campo	Tipo	Obligatorio	Descripción
`keyword`	`string`	No	Search keyword.
`roleKey`	`string`	No	Role key filter.
`roles`	`array<string>`	No	Role filters or the returned role list, depending on the model.
`statuses`	`array<string>`	No	Member status filters: active or disabled.
`updatedAtStart`	`string`	No	Inclusive update-time lower bound in RFC 3339 format.
`updatedAtEnd`	`string`	No	Inclusive update-time upper bound in RFC 3339 format.
`page`	`integer (int64)`	No	Page number, starting at 1.
`size`	`integer (int64)`	No	Number of members per page.

PlatformMemberResp

Campo	Tipo	Obligatorio	Descripción
`userId`	`string`	Sí	User ID.
`userName`	`string`	No	User name.
`nickName`	`string`	No	User nickname.

Campo	Tipo	Obligatorio	Descripción
email	string	No	Email address.
status	string	Sí	Platform member status: active or disabled.
roles	array<object>	Sí	Role filters or the returned role list, depending on the model.
updatedAt	string	Sí	Last update time.

PlatformMemberRoleResp

Campo	Tipo	Obligatorio	Descripción
roleId	string	Sí	Role ID, represented as a string.
roleKey	string	Sí	Role key filter.
roleName	string	No	Role display name.
description	string	No	Description.

PlatformMembersResp

Campo	Tipo	Obligatorio	Descripción
list	array<object>	Sí	Result list.
total	integer (int64)	Sí	Total number of matching records.
page	integer (int64)	Sí	Page number, starting at 1.
size	integer (int64)	Sí	File size in bytes.

ReadReq

ReadReq

Campo	Tipo	Obligatorio	Descripción
topics	array<string>	Sí	UNS topic paths to read.
include_metadata	boolean (boolean)	No	Whether each result should include UNS node metadata.
include_leaf_value	boolean (boolean)	No	Whether leaf nodes should include their latest payload.

ReadResp

ReadResp

Campo	Tipo	Obligatorio	Descripción
success	boolean (boolean)	Sí	Whether the operation succeeded.
results	array<object>	Sí	Per-item results. Check success and error for each item.

SchemaField

SchemaField

Campo	Tipo	Obligatorio	Descripción
name	string	Sí	Payload field name.
type	string	Sí	Payload field type: INTEGER, LONG, DOUBLE, FLOAT, STRING, BOOLEAN, or DATETIME.
unit	string	No	Optional engineering unit.

SearchReq

SearchReq

Campo	Tipo	Obligatorio	Descripción
keyword	string	No	Keyword matched against UNS node names, aliases, and paths.
topicType	string	No	Topic data category filter: STATE, ACTION, or METRIC.
path_prefix	string	No	Only return nodes under this UNS path prefix.
include_metadata	boolean (boolean)	No	Whether returned nodes should include metadata.
include_leaf_value	boolean (boolean)	No	Whether returned leaf nodes should include their latest payload.
page	integer (int64)	No	Page number, starting from 1.
size	integer (int64)	No	Page size.

SearchResp

SearchResp

Campo	Tipo	Obligatorio	Descripción
objects	array<object>	Sí	Matching UNS nodes.
page	integer (int64)	Sí	Page number, starting at 1.

Campo	Tipo	Obligatorio	Descripción
size	integer (int64)	Sí	File size in bytes.
total	integer (int64)	Sí	Total number of matching records.

UnsAttachmentInfo

UnsAttachmentInfo

Campo	Tipo	Obligatorio	Descripción
unsId	integer (int64)	Sí	Numeric UNS node ID.
fileName	string	No	Original file name.
filePath	string	Sí	Server-issued storage path returned by an upload API.
fileUrl	string	No	File access URL.
contentType	string	No	File MIME type, for example application/octet-stream.
size	integer (int64)	No	File size in bytes.
sha256	string	No	SHA-256 checksum of the file contents.
createdAt	integer (int64)	No	Creation time, in Unix milliseconds.
fileUrlExpiresAt	integer (int64)	No	File URL expiration time, in Unix milliseconds.

UnsAttachmentListReq

UnsAttachmentListReq

Campo	Tipo	Obligatorio	Descripción
unsId	integer (int64)	No	Numeric UNS node ID.
topic	string	No	Full UNS topic path.
page	integer (int64)	No	Page number, starting at 1.
size	integer (int64)	No	Number of attachments per page.
includeFileUrl	boolean (boolean)	No	Whether to return temporary file URLs. Defaults to true.

UnsAttachmentListResp

UnsAttachmentListResp

Campo	Tipo	Obligatorio	Descripción
unsId	integer (int64)	Sí	Numeric UNS node ID.
list	array<object>	Sí	Result list.
total	integer (int64)	Sí	Total number of matching records.
page	integer (int64)	Sí	Page number, starting at 1.
size	integer (int64)	Sí	File size in bytes.

UnsAttachmentUploadReq

UnsAttachmentUploadReq

Campo	Tipo	Obligatorio	Descripción
unsId	integer (int64)	No	Numeric owner node ID. Supply unsId or topic; a valid unsId takes precedence.
topic	string	No	Full owner node path, used when a valid unsId is not supplied.
fileName	string	No	Original file name.
sha256	string	No	SHA-256 checksum of the file contents.
file	string (binary)	Sí	Binary file contents. The multipart field name must be file.

UnsAttachmentUploadResp

UnsAttachmentUploadResp

Campo	Tipo	Obligatorio	Descripción
unsId	integer (int64)	Sí	Numeric UNS node ID.
fileName	string	No	Original file name.
filePath	string	Sí	Server-issued storage path returned by an upload API.
fileUrl	string	No	File access URL.
contentType	string	No	File MIME type, for example application/octet-stream.
size	integer (int64)	No	File size in bytes.
sha256	string	No	SHA-256 checksum of the file contents.
createdAt	integer (int64)	No	Creation time, in Unix milliseconds.
fileUrlExpiresAt	integer (int64)	No	File URL expiration time, in Unix milliseconds.

UnsBindFlowReq

UnsBindFlowReq

Campo	Tipo	Obligatorio	Descripción
`unsId`	`integer (int64)`	Sí	UNS node ID to bind.
`flowId`	`integer (int64)`	Sí	Flow ID to bind.

UnsBindFlowResp

UnsBindFlowResp

Campo	Tipo	Obligatorio	Descripción
`success`	`boolean (boolean)`	Sí	Whether the operation succeeded.
`unsId`	`integer (int64)`	Sí	Numeric UNS node ID.
`flowId`	`integer (int64)`	Sí	Flow identifier.

WriteItem

WriteItem

Campo	Tipo	Obligatorio	Descripción
`topic`	`string`	Sí	UNS topic path to write.
`value`	`object`	No	Payload value to write.
`timeStamp`	`integer (int64)`	No	Optional Unix timestamp in milliseconds. Omit it to use server time.

WriteReq

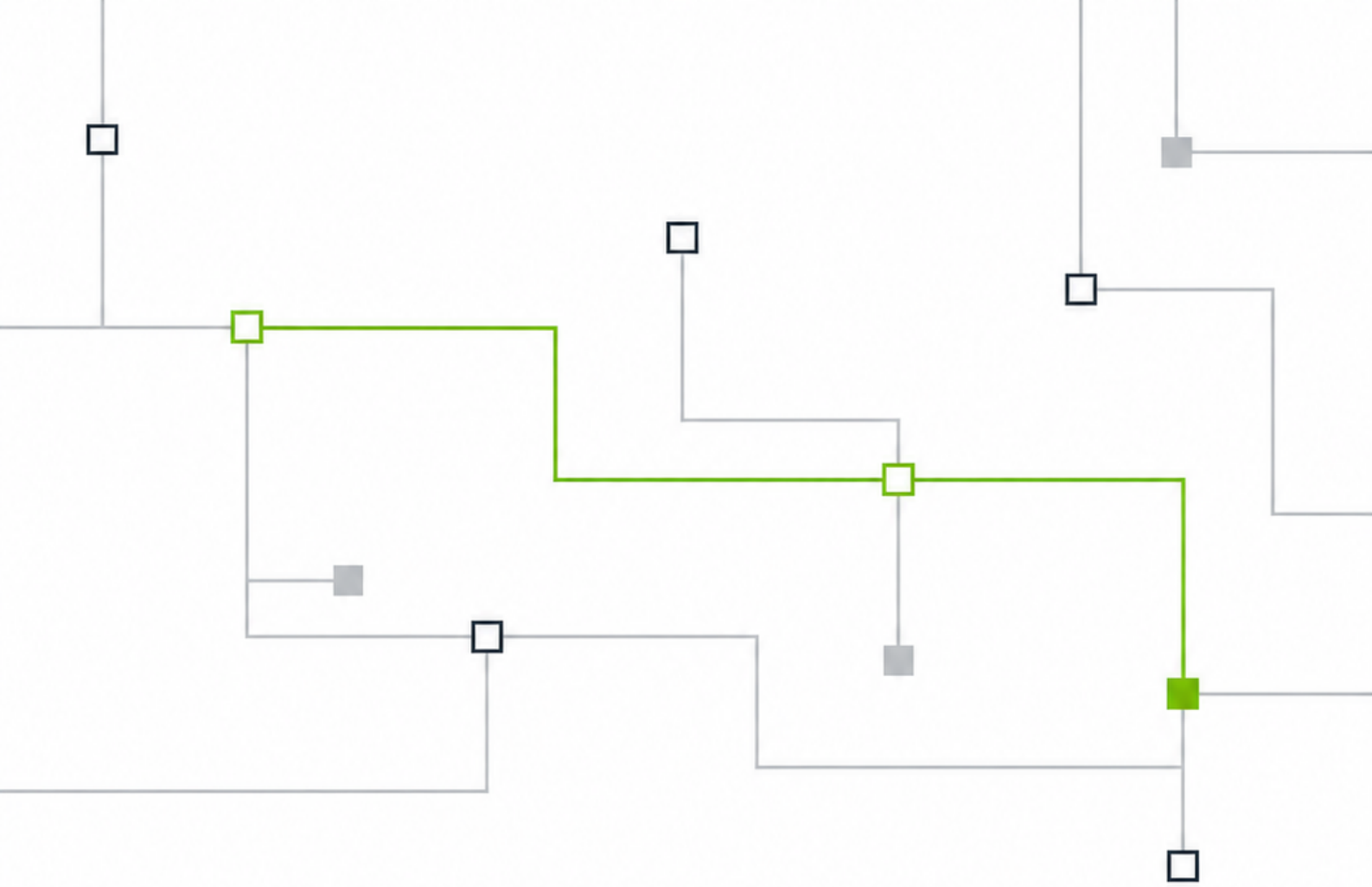
WriteReq

Campo	Tipo	Obligatorio	Descripción
`writes`	`array<object>`	Sí	Write operations to apply.
`qos`	`integer (int64)`	No	MQTT QoS: 0, 1, or 2.
`retain`	`boolean (boolean)`	No	Whether the MQTT message should be retained.

WriteResp

WriteResp

Campo	Tipo	Obligatorio	Descripción
`success`	`boolean (boolean)`	Sí	Whether the operation succeeded.
`results`	`array<object>`	Sí	Per-item results. Check success and error for each item.



TIER 0

tier0.app